

Competitive Programming

Section 62 Editorial: Codeforces Course Ladder

Competitive Programming Course

2026-07-18

Table of contents

1	How to use these editorials	19
2	1. 71A – Way Too Long Words	20
2.1	Restatement and recognition	20
2.2	Algorithm, proof, and tests	20
3	2. 4C – Registration System	21
3.1	State meaning	21
3.2	Why no extra occupied-name set is needed	21
4	3. 363B – Fence	22
4.1	Observation	22
4.2	Algorithm and proof	22
5	4. 279B – Books	23
5.1	Invariant	23
5.2	Algorithm, proof, and complexity	23
6	5. 158B – Taxi	24
6.1	Forced greedy packing	24
6.2	Proof route	24
7	6. 706B – Interesting Drink	25
7.1	Recognition and algorithm	25
7.2	Correctness and traps	25
8	7. 520B – Two Buttons	26
8.1	State graph	26
8.2	Algorithm and proof	26
9	8. 455A – Boredom	27
9.1	Compress the story by value	27
9.2	Correctness and implementation	27
10	9. 300C – Beautiful Numbers	28
10.1	Count by number of a digits	28
10.2	Proof and complexity	28
11	10. 166B – Polygons	29
11.1	Reduce containment to convex point location	29
11.2	Why this is sufficient	29
12	11. 20C – Dijkstra?	30
12.1	Model and algorithm	30
12.2	Correctness proof	30
13	12. 161D – Distance in Tree	31
13.1	Bounded-depth tree DP	31

13.2 Merge, proof, and complexity	31
14 13. 580D – Kefa and Dishes	32
14.1 State captures the only relevant history	32
14.2 Proof and costs	32
15 14. 1245D – Shichikuji and Power Grid	33
15.1 Virtual-source reduction	33
15.2 Algorithm and proof	33
16 15. 271D – Good Substrings	34
16.1 Enumerate starts, identify contents exactly	34
16.2 Correctness and testing	34
17 16. 61E – Enemy Is Weak	35
17.1 Split each decreasing triple at its middle	35
17.2 Proof and complexity	35
18 17. 86D – Powerful Array	36
18.1 Maintain the statistic while moving a range	36
18.2 Why it works	36
19 18. 498C – Array and Operations	37
19.1 Decompose operations by prime	37
19.2 Reduction proof	37
20 19. 319C – Kalila and Dimna in the Logging Industry	38
20.1 Derive lines before choosing a hull	38
20.2 Hull invariant and proof	38
21 20. 786B – Legacy	39
21.1 Reduction	39
22 21. 191C – Fools and Roads	40
22.1 Tree difference invariant	40
23 22. 208E – Blood Cousins	41
23.1 Lift, group by depth, count an interval	41
24 23. 375D – Tree and Queries	42
24.1 Keep the heavy subtree’s frequency table	42
25 24. 600E – Lomsat gelral	43
25.1 Small-to-large maps	43
26 25. 877E – Danil and a Part-time Job	44
26.1 Euler-tour range toggles	44
27 26. 242E – XOR on Segment	45
27.1 Store bit counts, not only the sum	45
28 27. 103D – Time to Raid Cowavans	46
28.1 Split queries by step size	46
29 28. 1000F – One Occurrence	47
29.1 Sweep by right endpoint	47

30	29. 240F – TorCoder	48
	30.1 Count letters, then rebuild the range	48
31	30. 343D – Water Tree	49
	31.1 One tree, two interval families	49
32	31. 1209E2 – Rotate Columns	50
	32.1 Keep only useful columns, then subset DP	50
33	32. 1416C – XOR Inverse	51
	33.1 Decide each xor bit independently	51
34	33. 13E – Holes	52
	34.1 Sqrt-decompose jump chains	52
35	34. 342E – Xenia and Tree	53
	35.1 Centroid ancestors summarize red updates	53
36	35. 895C – Square Subsets	54
	36.1 Parity masks of prime exponents	54
37	36. 868F – Yet Another Minimization Problem	55
	37.1 Divide-and-conquer DP with a movable cost window	55
38	37. 1292C – Xenon's Attack on the Gangs	56
	38.1 DP on ordered endpoint pairs	56
39	38. 613D – Kingdom and its Cities	57
	39.1 Virtual tree preserves only relevant ancestry	57
40	39. 1110F – Nearest Leaf	58
	40.1 Reroot distances as range additions	58
41	40. 587C – Duff in the Army	59
	41.1 Merge only the smallest needed identifiers	59
42	41. 786C – Till I Collapse	60
	42.1 Divide answer values by their first segment endpoint	60
43	42. 1093G – Multidimensional Queries	61
	43.1 Manhattan distance is a maximum over sign masks	61
44	43. 321E – Ciel and Gondolas	62
	44.1 Prefix costs plus monotone divide-and-conquer DP	62
45	44. 896C – Willem, Chtholly and Seniorious	63
	45.1 Ordered disjoint intervals, with stated randomness assumptions	63
46	45. 833B – The Bakery	64
	46.1 Divide-and-conquer DP with dynamic distinct cost	64
47	46. 311B – Cats Transport	65
	47.1 Remove station travel before partitioning waiting times	65
48	47. 55D – Beautiful Numbers	66
	48.1 Digit DP over divisibility state	66

49 48. 840D – Destiny	67
49.1 Random sampling plus exact range counts	67
50 49. 911G – Mass Change Queries	68
50.1 Push value transitions through a segment tree	68
51 50. 1100F – Ivan and Burgers	69
51.1 Offline ranges with a position-aware xor basis	69
52 D. Ribbon Cuts (Offline Blind Capstone)	70
52.1 Restatement and state	70
52.2 Correctness proof	70
52.3 Complexity and tests	70
52.4 Full C++ solution	70
52.5 Full Python solution	70
53 Stress and carry-forward	72
54 Offline Companion Editorials	73
54.1 1. 71A – Way Too Long Words (800): local companion	73
54.2 Restatement	73
54.3 Constraints and traps	73
54.4 Sample walk-through	73
54.5 Algorithm	73
54.6 Correctness proof	74
54.7 Complexity	74
54.8 C++ implementation notes	74
54.9 Python implementation notes	74
54.10 Full C++ solution	74
54.11 Full Python solution	75
54.12 Common wrong answers and tests	75
55 Stress testing strategy	76
56 What to carry forward	77
56.1 2. 4C – Registration System (1300): local companion	77
56.2 Restatement	77
56.3 Constraints and traps	77
56.4 Walk-through	77
56.5 Algorithm	78
56.6 Correctness proof	78
56.7 Complexity and language notes	78
56.8 Full C++ solution	78
56.9 Full Python solution	79
56.10 Tests that matter	79
56.11 3. 363B – Fence (1100): local companion	79
56.12 Restatement	79
56.13 Constraints and traps	79
56.14 Observation and trace	80
56.15 Algorithm	80
56.16 Correctness proof	80
56.17 Complexity	80
56.18 Implementation notes and tests	80
57 Stress testing strategy	81

58 What to carry forward	82
58.1 Observation	82
58.2 Correctness and complexity	82
58.3 Tests that matter	82
58.4 Full C++ solution	82
58.5 Full Python solution	83
58.6 4. 279B – Books (1400): local companion	84
58.7 Restatement and recognition	84
58.8 Trace and algorithm	84
58.9 Correctness proof	84
58.10 Complexity and implementation notes	84
58.11 Full C++ solution	84
58.12 Full Python solution	85
58.13 Tests that matter	85
59 Stress testing strategy	86
60 What to carry forward	87
60.1 5. 158B – Taxi (1100): local companion	87
60.2 Restatement and constraints	87
60.3 Observation and walk-through	87
60.4 Algorithm	87
60.5 Correctness proof	88
60.6 Complexity and implementation notes	88
60.7 Full C++ solution	88
60.8 Full Python solution	88
60.9 Tests that matter	89
60.10 6. 706B – Interesting Drink (1100): local companion	89
60.11 Restatement	89
60.12 Constraints and traps	89
60.13 Observation and trace	89
60.14 Algorithm	90
60.15 Correctness proof	90
60.16 Complexity	91
60.17 Full C++ solution	91
60.18 Full Python solution	91
60.19 Implementation traps and tests	92
61 Stress testing strategy	93
62 What to carry forward	94
62.1 7. 520B – Two Buttons (1400): local companion	94
62.2 Restatement and traps	94
62.3 Trace and algorithm	94
62.4 Correctness proof	94
62.5 Complexity and language notes	95
62.6 Full C++ solution	95
62.7 Full Python solution	96
62.8 Tests that matter	96
63 Contest testing strategy	97
64 What to carry forward	98
64.1 8. 455A – Boredom (1500): local companion	98
64.2 Restatement and predicate	98

64.3	Bounds, trace, and algorithm	98
64.4	Correctness proof	98
64.5	Complexity and implementation notes	99
64.6	Full C++ solution	99
64.7	Full Python solution	100
64.8	9. 300C – Beautiful Numbers (1800): local companion	100
64.9	Restatement	101
64.10	Observation and trace	101
64.11	Algorithm	101
64.12	Correctness proof	101
64.13	Complexity	102
64.14	Implementation traps and tests	102
65	Stress testing greedy	103
66	What to carry forward	104
66.1	Reference implementations	104
66.2	Full C++ solution	104
66.3	Full Python solution	105
66.4	10. 166B – Polygons (2100, geometry challenge): local companion	105
66.5	Restatement and feasibility	106
66.6	Greedy choice and trace	106
66.7	Algorithm	106
66.8	Correctness proof	106
66.9	Complexity and tests	106
66.10	Full C++ solution	106
66.11	Full Python solution	106
67	Stress testing constructive greedy	108
68	What to carry forward	109
68.1	11. 20C – Dijkstra? (1900): local companion	109
68.2	Restatement	109
68.3	Constraints and traps	109
68.4	Closed-interval event model	109
68.5	Correctness and complexity	110
68.6	Full C++ solution	110
68.7	Full Python solution	110
68.8	12. 161D – Distance in Tree (1800): local companion	110
68.9	Restatement	111
68.10	Observation and trace	111
68.11	Algorithm	111
68.12	Correctness proof	112
68.13	Complexity	112
68.14	Implementation traps and tests	112
68.15	Full C++ solution	112
68.16	Full Python solution	113
69	Stress testing	114
70	What to carry forward	115
70.1	13. 580D – Kefa and Dishes (1800): local companion	115
70.2	Restatement	115
70.3	Constraints and traps	115
70.4	Observation and walk-through	115

70.5	Algorithm	116
70.6	Correctness proof	116
70.7	Complexity	116
70.8	Full C++ solution	116
70.9	Full Python solution	117
70.10	Tests that matter	117
70.11	14. 1245D – Shichikuji and Power Grid (1900): local companion	117
70.12	Restatement	117
70.13	Constraints and traps	117
70.14	Observation and walk-through	118
70.15	Algorithm	118
70.16	Correctness proof	118
70.17	Complexity	118
70.18	Full C++ solution	118
70.19	Full Python solution	119
70.20	Tests that matter	119
70.21	15. 271D – Good Substrings (1800): local companion	119
70.22	Restatement	119
70.23	Key idea: start BFS from every station	120
70.24	Walk-through	120
70.25	Algorithm	120
70.26	Correctness proof	121
70.27	Complexity	121
70.28	Reference implementation in C++	121
70.29	Reference implementation in Python	122
70.30	Tests worth checking	123
70.31	16. 61E – Enemy Is Weak (1900): local companion	123
70.32	Restatement	123
70.33	Constraints and traps	123
70.34	Observation and walk-through	124
70.35	Algorithm	124
70.36	Correctness proof	124
70.37	Complexity	125
70.38	C++ implementation notes	125
70.39	Python implementation notes	125
70.40	Full C++ solution	125
70.41	Full Python solution	126
70.42	Common wrong answers	127
70.43	Tests that matter	128
71	Stress testing	129
72	What to carry forward	130
72.1	17. 86D – Powerful Array (2200): local companion	130
72.2	Restatement	130
72.3	Observation: available choices determine uniqueness	130
72.4	Walk-through	131
72.5	Algorithm	131
72.6	Correctness proof	132
72.7	Complexity	132
72.8	Implementation notes	133
72.9	Full C++ solution	133
72.10	Full Python solution	134
72.11	Common wrong answers	135

72.12	Tests that matter	135
73	Stress testing	136
74	What to carry forward	137
74.1	18. 498C – Array and Operations (2100): local companion	137
74.2	Restatement	137
74.3	Observation: keep a minimum spanning forest	137
74.4	Walk-through	138
74.5	Algorithm	138
74.6	Correctness proof	138
74.7	Complexity	139
74.8	Implementation notes	139
74.9	Full C++ solution	139
74.10	Full Python solution	140
74.11	Common wrong answers	141
74.12	Tests that matter	142
75	Stress testing	143
76	What to carry forward	144
76.1	19. 319C – Kalila and Dimna in the Logging Industry (2100): local companion	144
76.2	Restatement	144
76.3	Constraints and traps	144
76.4	Observation and walk-through	144
76.5	Algorithm	145
76.6	Correctness proof	145
76.7	Complexity	146
76.8	C++ implementation notes	146
76.9	Python implementation notes	146
76.10	Full C++ solution	146
76.11	Full Python solution	147
76.12	Common wrong answers	148
76.13	Tests that matter	148
77	Common wrong answers	149
78	Stress testing	150
79	What to carry forward	151
79.1	20. 786B – Legacy: local companion	151
79.2	Restatement	151
79.3	Model the implicit graph	151
79.4	Expanding one portal group once	151
79.5	Walk-through	152
79.6	Algorithm	152
79.7	Correctness proof	153
79.8	Complexity	153
79.9	Implementation notes	153
79.10	Full C++ solution	153
79.11	Full Python solution	155
79.12	Common wrong answers	156
79.13	Tests that matter	156
80	Stress testing	157

81 What to carry forward	158
81.1 21. 191C – Fools and Roads: local companion	158
81.2 Restatement	158
81.3 Threshold viewpoint	158
81.4 Walk-through	159
81.5 Algorithm	159
81.6 Correctness proof	159
81.7 Complexity	160
81.8 Implementation notes	160
81.9 Full C++ solution	160
81.10 Full Python solution	161
81.11 Common wrong answers	162
81.12 Tests that matter	162
82 Stress testing	163
83 What to carry forward	164
83.1 22. 208E – Blood Cousins: local companion	164
83.2 Restatement	164
83.3 Constraints and traps	164
83.4 Observation and walk-through	164
83.5 Algorithm	165
83.6 Correctness proof	165
83.7 Complexity	166
83.8 C++ implementation notes	166
83.9 Python implementation notes	166
83.10 Full C++ solution	166
83.11 Full Python solution	167
83.12 Common wrong answers	167
83.13 Tests that matter	167
84 Testing and stress strategy	168
85 What to carry forward	169
85.1 23. 375D – Tree and Queries: local companion	169
85.2 Restatement and reduction	169
85.3 Constraints and traps	169
85.4 Reachability trace	170
85.5 Algorithm	170
85.6 Correctness proof	170
85.7 Complexity	171
85.8 Full C++ solution	171
85.9 C++ implementation notes	171
85.10 Full Python solution	171
85.11 Python implementation notes	172
85.12 Common wrong answers	172
85.13 Tests that matter	172
86 Stress testing	173
87 What to carry forward	174
87.1 24. 600E – Lomsat gelral: local companion	174
87.2 Restatement	174
87.3 Constraints and traps	174
87.4 Key observation and trace	174

87.5	State and algorithm	175
87.6	Correctness proof	175
87.7	Complexity	176
87.8	Full C++ solution	176
87.9	Full Python solution	176
87.10	Common wrong answers and tests	177
87.11	Stress testing	177
88	What to carry forward	179
88.1	25. 877E – Danil and a Part-time Job: local companion	179
88.2	Restatement	179
88.3	Constraints and traps	179
88.4	State and transition	179
88.5	Small trace	180
88.6	Algorithm	180
88.7	Correctness proof	180
88.8	Complexity	180
88.9	Full C++ solution	180
88.10	C++ implementation notes	181
88.11	Full Python solution	181
88.12	Python implementation notes	181
88.13	Traps and tests	182
89	Stress testing	183
90	What to carry forward	184
90.1	26. 242E – XOR on Segment: local companion	184
90.2	Restatement	184
90.3	Constraints and traps	184
90.4	State and walk-through	184
90.5	Algorithm	185
90.6	Correctness proof	185
90.7	Complexity	186
90.8	C++ implementation notes	186
90.9	Python implementation notes	186
90.10	Full C++ solution	186
90.11	Full Python solution	187
90.12	Common wrong answers	188
90.13	Tests that matter	188
91	Testing and carry-forward	189
91.1	27. 103D – Time to Raid Cowavans: local companion	189
91.2	Restatement	189
91.3	Constraints and traps	189
91.4	Observations	190
91.5	Sample walk-through	190
91.6	Algorithm	190
91.7	Correctness proof	191
91.8	Complexity	192
91.9	C++ implementation notes	192
91.10	Python implementation notes	192
91.11	Full C++ solution	192
91.12	Full Python solution	193
91.13	Common wrong answers	194

91.14Tests that matter	194
92 Testing discussion	195
93 What to carry forward	196
93.1 28. 1000F – One Occurrence: local companion	196
93.2 Restatement and reduction	196
93.3 Constraints and traps	196
93.4 Worked trace	197
93.5 Algorithm	197
93.6 Correctness proof	197
93.7 Complexity	197
93.8 Full C++ solution	198
93.9 C++ implementation notes	198
93.10Full Python solution	198
93.11Python implementation notes	199
93.12Traps and tests	199
94 Stress testing	200
95 What to carry forward	201
95.1 29. 240F – TorCoder: local companion	201
95.2 Restatement and observation	201
95.3 Constraints and traps	201
95.4 Worked trace	202
95.5 Algorithm	202
95.6 Correctness proof	202
95.7 Complexity	202
95.8 Full C++ solution	202
95.9 C++ implementation notes	203
95.10Full Python solution	203
95.11Python implementation notes	204
95.12Traps and tests	204
96 Stress testing	205
97 What to carry forward	206
97.1 Constraints and traps	206
97.2 Observations	206
97.3 Sample walk-through	207
97.4 Algorithm	207
97.5 Correctness proof	208
97.6 Complexity	208
97.7 C++ implementation notes	208
97.8 Python implementation notes	209
97.9 Full C++ solution	209
97.10Full Python solution	210
97.11Common wrong answers	210
97.12Tests that matter	210
97.13D-specific stress testing	211
98 Formative upsolve exercises	212
99 Local workflow	213

100	Carry-forward checklist	214
101	Full reference implementations	215
101.1A.	GCD LCM Queries: C++	215
101.2A.	GCD LCM Queries: Python	215
101.3B.	General Inverse: C++	216
101.4B.	General Inverse: Python	217
101.5C.	Linear Diophantine: C++	217
101.6C.	Linear Diophantine: Python	218
101.7D.	Clock Offset: C++	219
101.8D.	Clock Offset: Python	220
101.931.	1209E2 – Rotate Columns: local companion	220
101.10	Restatement	220
101.11	Constraints and traps	220
101.12	Observation and walk-through	221
101.13	Algorithm	221
101.14	Correctness proof	222
101.15	Complexity	222
101.16	C++ implementation notes	222
101.17	Python implementation notes	222
101.18	Full C++ solution	222
101.19	Full Python solution	223
101.20	Common wrong answers	224
101.21	Tests that matter	224
102	Testing and stress discussion	225
103	What to carry forward	226
103.132.	1416C – XOR Inverse: local companion	226
103.2	Restatement	226
103.3	Constraints and traps	226
103.4	Observations	226
103.5	Sample walk-through	227
103.6	Algorithm	227
103.7	Correctness proof	228
103.8	Complexity	228
103.9	C++ implementation notes	228
103.10	Python implementation notes	229
103.11	Full C++ solution	229
103.12	Full Python solution	230
103.13	Common wrong answers	230
103.14	Tests that matter	231
104	Testing and stress discussion	232
105	What to carry forward	233
105.133.	13E – Holes: local companion	233
105.2	Restatement	233
105.3	Constraints and traps	233
105.4	Bad events and reduction	233
105.5	Worked intersection trace	234
105.6	Algorithm	234
105.7	Correctness proof	234
105.8	Complexity	234

105.9	Full C++ solution	234
105.10	C++ implementation notes	235
105.11	Full Python solution	235
105.12	Python implementation notes	236
105.13	Traps and tests	236
106	Stress testing	237
107	What to carry forward	238
107.134.	342E – Xenia and Tree: local companion	238
107.2	Restatement	238
107.3	Constraints and traps	238
107.4	Observation and trace	238
107.5	Algorithm	239
107.6	Correctness proof	239
107.7	Complexity	239
107.8	Full C++ solution	239
107.9	Full Python solution	239
107.10	Implementation notes and tests	240
107.11	Stress testing	240
108	Local workflow	241
108.1A.	Expected Prize: C++	241
108.2A.	Expected Prize: Python	242
108.3B.	First Success: C++	242
108.4B.	First Success: Python	243
108.5C.	Coupon Collector: C++	243
108.6C.	Coupon Collector: Python	244
109	What to carry forward	245
109.135.	895C – Square Subsets: local companion	245
109.2	Restatement and constraints	245
109.3	Observation and worked trace	245
109.4	Algorithm	245
109.5	Correctness proof	246
109.6	Complexity, implementation notes, and tests	246
109.7	Full C++ solution	246
109.8	Full Python solution	247
109.9	D-specific stress testing	248
110	Testing discussion	249
111	What to carry forward	250
112	Problem-local upsolve checklist	251
112.1A.	Blocked Path	251
112.2B.	Coprime Pairs	251
112.3C.	At Least One	252
112.436.	868F – Yet Another Minimization Problem: local companion	252
112.5	Restatement	253
112.6	Constraints and traps	253
112.7	Observations	253
112.8	Sample walk-through	254
112.9	Algorithm	254
112.10	Correctness proof	254

112.1C	Complexity	255
112.1C++	implementation notes	255
112.1P	Python implementation notes	255
112.1F	Full C++ solution	255
112.1P	Full Python solution	256
112.1C	Common wrong answers	258
112.1T	Tests that matter	258
113	Testing discussion	259
114	Problem-specific upsolve notes	260
114.1A.	Point Add Range Sum	260
114.2B.	Inversion Count	260
114.3C.	Range Add Point Query	260
115	Stress-testing workflow	262
115.137.	1292C – Xenon’s Attack on the Gangs: local companion	262
115.2	Restatement and constraint implications	262
115.3	Node invariant and trace	262
115.4	Algorithm	262
115.5	Correctness proof	263
115.6	Complexity, implementation notes, and tests	263
115.7	Full C++ solution	263
115.8	Full Python solution	264
116	Testing discussion	266
117	Problem-specific upsolve notes	267
117.1A.	Range Minimum Updates	267
117.2B.	Maximum Subarray Updates	267
117.3C.	Lazy Range Add Sum	267
117.4D.	First Available	268
118	Stress-testing workflow	269
118.138.	613D – Kingdom and its Cities: local companion	269
118.2	Restatement	269
118.3	Constraints and traps	269
118.4	Observations	269
118.5	Sample walk-through	270
118.6	Algorithm	270
118.7	Correctness proof	270
118.8	Complexity	271
118.9C++	implementation notes	271
118.1P	Python implementation notes	271
118.1F	Full C++ solution	271
118.1P	Full Python solution	272
118.1C	Common wrong answers	273
118.1T	Tests that matter	274
119	Testing discussion	275
120	Problem-specific upsolve notes	276
120.1A.	Static Range Minimum	276
120.2B.	Static Range GCD	276
120.3C.	Kth Ancestor	276

121Stress-testing workflow	277
121.139. 1110F – Nearest Leaf: local companion	277
121.2Restatement and traps	277
121.3Observation and trace	277
121.4Algorithm	277
121.5Correctness proof	278
121.6Complexity	278
121.7Full C++ solution	278
121.8Full Python solution	278
121.9Implementation notes and tests	278
121.10Stress testing	278
122What to carry forward	279
122.140. 587C – Duff in the Army: local companion	279
122.2Restatement and constraint implications	279
122.3Coordinate collection and trace	279
122.4Algorithm	279
122.5Correctness proof	280
122.6Complexity, implementation notes, and tests	280
122.7Full C++ solution	280
122.8Full Python solution	282
123Testing discussion	284
124Detailed upsolve workshop	285
124.1A. Hotel Queries	285
124.2B. List Removals	285
124.3C. Mo Distinct Queries	286
125What to carry forward	287
125.141. 786C – Till I Collapse: local companion	287
125.2Restatement	287
125.3Constraints and traps	287
125.4Observations	287
125.5Sample walk-through	288
125.6Algorithm	288
125.7Correctness proof	288
125.8Complexity	288
125.9C++ implementation notes	289
125.10Python implementation notes	289
125.11Full C++ solution	289
125.12Full Python solution	289
125.13Common wrong answers	290
125.14Tests that matter	290
126Testing discussion	291
127What to carry forward	292
127.142. 1093G – Multidimensional Queries: local companion	292
127.2Restatement and constraints	292
127.3Ordering and worked trace	292
127.4Algorithm	293
127.5Correctness proof	293
127.6Complexity, implementation notes, and tests	293
127.7Full C++ solution	294

127.8	Full Python solution	295
128	What to carry forward	298
129	Testing discussion	299
130	What to carry forward	300
130.143.	321E – Ciel and Gondolas: local companion	300
130.2	Restatement and traps	300
130.3	Observation and trace	300
130.4	Correctness proof	301
130.5	Complexity	301
130.6	Full C++ solution	301
130.7	Full Python solution	301
130.8	Tests and stress testing	302
130.944.	896C – Willem, Chtholly and Seniorious: local companion	302
130.10	Restatement	302
130.11	Constraints and traps	302
130.12	Observations and trace	302
130.13	Algorithm	302
130.14	Correctness proof	303
130.15	Complexity	303
130.16	C++ implementation notes	303
130.17	Python implementation notes	303
130.18	Full C++ solution	303
130.19	Full Python solution	304
130.20	Common wrong answers	304
130.21	Tests that matter	304
131	What to carry forward	305
132	Testing discussion	306
132.145.	833B – The Bakery: local companion	306
132.2	Restatement and reduction target	306
132.3	Network construction and trace	306
132.4	Algorithm	307
132.5	Correctness proof	307
132.6	Complexity, implementation notes, and tests	307
132.7	Full C++ solution	307
132.8	Full Python solution	309
133	What to carry forward	312
134	Testing discussion	313
134.146.	311B – Cats Transport: local companion	313
134.2	Restatement	313
134.3	Constraints and traps	313
134.4	Observations and trace	313
134.5	Algorithm	314
134.6	Correctness proof	314
134.7	Complexity and implementation notes	315
134.8	Full C++ solution	315
134.9	Full Python solution	316
134.10	Common wrong answers	317
134.11	Tests that matter	317

135	Testing discussion	318
135.1	47. 55D – Beautiful Numbers: local companion	318
135.2	Restatement and constraint implications	318
135.3	State and worked trace	318
135.4	Algorithm	318
135.5	Correctness proof	319
135.6	Complexity, implementation notes, and tests	319
135.7	Full C++ solution	319
135.8	Full Python solution	320
136	Testing discussion	322
136.1	48. 840D – Destiny: local companion	322
136.2	Restatement and constraints	322
136.3	Reduction and trace	322
136.4	Monotone deque algorithm	322
136.5	Correctness proof	323
136.6	Complexity and implementation notes	323
136.7	Full C++ solution	323
136.8	Full Python solution	324
136.9	Tests that matter	325
137	Testing discussion	326
137.1	49. 911G – Mass Change Queries: local companion	326
137.2	Restatement	326
137.3	Observations	326
137.4	Algorithm	326
137.5	Correctness proof	327
137.6	Complexity	327
137.7	Implementation notes	327
137.8	Full C++ solution	327
137.9	Full Python solution	327
137.10	Tests that matter	328
138	Testing discussion	329
138.1	50. 1100F – Ivan and Burgers: local companion	329
138.2	Restatement and observation	329
138.3	Algorithm	329
138.4	Correctness proof	329
138.5	Complexity and implementation	330
138.6	Full C++ solution	330
138.7	Full Python solution	330
138.8	Tests that matter	330
139	Testing discussion	331

1 How to use these editorials

Each block is an original derivation, not a substitute for attempting the linked task. Read the restatement only to compare your interpretation. Then look for the recognition signal, reproduce the proof route, and finally check the implementation traps. For external problems, the course teaches the tools; these notes teach why that particular statement calls for them.

2 1. 71A – Way Too Long Words

Official: <https://codeforces.com/problemset/problem/71/A>

2.1 Restatement and recognition

Transform each independent word according to one length threshold. There is no interaction between cases, so the correct model is a direct input/output loop, not a string algorithm.

2.2 Algorithm, proof, and tests

For a word `s`, leave it unchanged when `len(s) <= 10`; otherwise output `s[0]`, `len(s)-2`, and `s.back()`. The two branches partition all legal word lengths. In the long branch, exactly the first and last characters are kept, so the count between them is exactly `len(s)-2`; therefore the output matches the requested abbreviation. The time is $O(\text{total input characters})$.

Test lengths 1, 10, 11, and 100. In C++, concatenate `string` and `to_string`; in Python use an f-string. Do not treat the threshold as `>= 10`.

3 2. 4C – Registration System

Official: <https://codeforces.com/problemset/problem/4/C>

3.1 State meaning

Maintain `next[name]`: the smallest suffix not yet suggested for this base name, with zero meaning that the unsuffixed name has never appeared. On a new base name, print `OK` and set `next[name] = 1`. Otherwise print `name + next[name]` and increment it.

3.2 Why no extra occupied-name set is needed

The requests themselves are lowercase base names, while every generated suffix for one base is issued in increasing order. Once suffix `t` is printed, all smaller positive suffixes for that base were printed earlier, and `t+1` is the next free one. The map invariant therefore yields the smallest allowed suffix. Expected time is $O(\text{total name length})$ with a hash map, and memory is linear in the distinct requested names.

Test the same base three times, two interleaved bases, and a request that looks like another generated name. Use `unordered_map<string,int>` or `dict`; never scan suffixes from 1 on every duplicate.

4 3. 363B – Fence

Official: <https://codeforces.com/problemset/problem/363/B>

4.1 Observation

Every candidate is a fixed-length window. After computing the first sum, moving the window one step removes exactly its leftmost height and adds exactly its new rightmost height.

4.2 Algorithm and proof

Set $\text{sum} = a[0] + \dots + a[k-1]$, remember it as the current best, and for each right endpoint $r \geq k$ update $\text{sum} += a[r] - a[r-k]$. Compare the resulting window $[r-k+1, r]$ to the best. By induction, just before each comparison sum equals the sum of that window: it is true initially, and the update removes/adds precisely the changed elements. Every length- k window appears once, so the minimum remembered sum is globally minimum. Time $O(n)$, memory $O(1)$ besides input.

Output the stored left endpoint plus one. Test $k=1$, $k=n$, and equal window sums; the problem accepts any minimizer, but your comparison decides the tie.

5 4. 279B – Books

Official: <https://codeforces.com/problemset/problem/279/B>

5.1 Invariant

All reading times are positive. Keep a window $[l, r]$ with sum at most t after repeatedly advancing l while it is too expensive. Then it is the longest feasible window ending at r : moving l farther right only shortens it, and moving it left would violate the budget.

5.2 Algorithm, proof, and complexity

Extend r from left to right and add $a[r]$. While the sum exceeds t , remove $a[l++]$. Maximize $r-l+1$. The invariant above proves each recorded length is feasible and no feasible window ending at r is longer. Each endpoint advances at most n times, yielding $O(n)$ time and $O(1)$ extra memory.

The positivity is essential: with negative times, a removed prefix could later become useful and this proof fails. Test a single over-budget book, an exact budget, and an optimal window beginning after index zero.

6 5. 158B – Taxi

Official: <https://codeforces.com/problemset/problem/158/B>

6.1 Forced greedy packing

Count groups of sizes 1, 2, 3, 4. Every size-4 group is alone. Every size-3 group must occupy a taxi and can take at most one size-1 group, so pair those first. Pair size-2 groups together; one remaining size-2 can take up to two ones. Finally pack remaining ones four per taxi.

6.2 Proof route

The size-3 decision is forced: no group other than a one can share its taxi, so using a one there cannot hurt a future fuller packing. Size-2 groups only fit with another size-2 or ones; pairing twos maximizes the remaining capacity for ones. After those forced classes, only ones remain, and four per taxi is optimal by capacity. Thus the construction uses the minimum possible taxis.

All operations are $O(n)$. Test no ones for size-3 groups, an odd number of twos, and one/two/three/four leftover ones. Do not simulate individual taxis.

7 6. 706B – Interesting Drink

Official: <https://codeforces.com/problemset/problem/706/B>

7.1 Recognition and algorithm

The same static set of prices serves many threshold queries. Sort it once. For a budget x , the answer is the first index strictly greater than x , namely `upper_bound(prices.begin(), prices.end(), x) - prices.begin()` in C++ or `bisect_right(prices, x)` in Python.

7.2 Correctness and traps

Sorting places every price $\leq x$ before every price $> x$; the first position after the former group is therefore exactly their count. Preprocessing costs $O(n \log n)$ and each query $O(\log n)$. Test a budget below every price, equal to duplicate prices, and above every price. `lower_bound` counts $< x$ and is wrong at equality.

8 7. 520B – Two Buttons

Official: <https://codeforces.com/problemset/problem/520/B>

8.1 State graph

An integer is a vertex, and each button press is a unit-cost directed edge. BFS finds the fewest presses once the state space is bounded. If $n \geq m$, only decrementing is useful, so answer $n-m$. Otherwise no optimal route needs to visit a value above $2m$: after first exceeding m , a decrement-only suffix reaches m ; overshooting past $2m$ cannot beat stopping at or below $2m$.

8.2 Algorithm and proof

Run BFS on $[0, 2m]$ from n , visiting $x-1$ when nonnegative and $2x$ when within the bound. BFS explores paths by increasing length, so its first visit to m is optimal. The bound argument ensures an optimal path is represented. Time and memory are $O(m)$ in the $n < m$ branch.

Test $n=m$, $n>m$, $m=1$, and a case where doubling first is worse than a decrement. Mark visited when enqueueing, not when dequeuing.

9 8. 455A – Boredom

Official: <https://codeforces.com/problemset/problem/455/A>

9.1 Compress the story by value

Choosing value x eventually earns $\text{gain}[x] = x * \text{frequency}[x]$; taking all copies is always at least as good as taking fewer. It forbids only neighboring values. Thus $\text{dp}[x]$ is the best score using values at most x .

```
dp[x] = max(dp[x-1], dp[x-2] + gain[x])
```

9.2 Correctness and implementation

An optimal solution either omits x , giving $\text{dp}[x-1]$, or takes it, forcing the rest to use values at most $x-2$, giving the second term. These exhaustive, disjoint cases prove the recurrence by induction. Sweep through the maximum input value using two rolling `long long` states. Complexity is $O(\text{max value} + n)$ time and $O(\text{max value})$ memory for frequencies.

Test repeated equal values, values separated by gaps, and a choice where one large frequency beats a neighboring value. Do not sort individual occurrences and apply the recurrence to positions; the conflict is between values.

10 9. 300C – Beautiful Numbers

Official: <https://codeforces.com/problemset/problem/300/C>

10.1 Count by number of a digits

If exactly i positions contain digit a , the digit sum is $s = i*a + (n-i)*b$, and there are $C(n,i)$ strings with that composition. Check whether every decimal digit of s is either a or b ; if so, add the binomial coefficient.

10.2 Proof and complexity

Every legal string has one unique i , and all strings with that i have the same digit sum. The digit check selects exactly the legal compositions, while the binomial coefficient counts their distinct position choices. Precompute factorials and inverse factorials modulo the prime, using Fermat's inverse. The work is $O(n + \text{number of digits checked across } n+1 \text{ sums})$, effectively $O(n \log n)$.

The input guarantees $1 \leq a < b \leq 9$, so the digit sum is positive. Test $a=1$, $b=9$, a sum containing a forbidden interior digit, and a composition where only one value of i succeeds. Use modular arithmetic only for counts, never for the decimal digit sum.

11 10. 166B – Polygons

Official: <https://codeforces.com/problemset/problem/166/B>

11.1 Reduce containment to convex point location

Because A is convex, if every vertex of B is strictly inside A , every edge and every interior point of B is also inside A : they are convex combinations of points of A 's interior. Thus test every B vertex against A . For one point P , use $A[0]$ as a fan pivot. First reject P unless it is strictly between rays $A[0] \rightarrow A[1]$ and $A[0] \rightarrow A[n-1]$; then binary-search the fan sector $(A[0], A[i], A[i+1])$ containing P and require strict inclusion in that triangle using cross-product signs.

11.2 Why this is sufficient

A convex polygon is partitioned by its pivot fan into triangles. The ray tests select a unique possible sector, and the triangle test is equivalent to strict membership in that sector. Therefore every accepted B vertex is strictly in A . Convexity then proves the whole of B is inside; any rejected vertex proves it is not. The time is $O(m \log n)$ and memory is $O(1)$ beyond the polygons.

Use `long long` for cross products. Test clockwise input, an inner vertex on A 's edge, a B edge that comes close to A , and a point just beyond one fan boundary. Never use floating point.

12 11. 20C – Dijkstra?

Official: <https://codeforces.com/problemset/problem/20/C>

12.1 Model and algorithm

All edge weights are positive, so Dijkstra applies. Store `dist[v]` and `parent[v]`; on a strict relaxation of $u \rightarrow v$, set `parent[v]=u`. Use a min-heap of (`distance`, `vertex`) pairs and ignore stale entries. If `n` remains infinite, print `-1`; otherwise follow parents from `n` to 1 and reverse.

12.2 Correctness proof

When Dijkstra extracts a non-stale vertex, no unexplored path can improve it: such a path first crosses from an extracted prefix through a nonnegative edge, so it cannot arrive cheaper. Thus every relaxation uses a correct shortest prefix. A parent is assigned only when an edge creates the current best distance, so following parents reconstructs a shortest path to vertex 1. Complexity is $O((n+m) \log n)$.

Test disconnected graphs, parallel edges, and a route whose first discovered path is later improved. Distances need `long long`; a boolean `visited` on heap insertion would incorrectly freeze an improvable vertex.

13 12. 161D – Distance in Tree

Official: <https://codeforces.com/problemset/problem/161/D>

13.1 Bounded-depth tree DP

Root the tree. Let $\text{cnt}[u][d]$ be the number of vertices in u 's subtree at distance d from u , for $0 \leq d \leq k$; initialize $\text{cnt}[u][0]=1$. Process a child v after its table is ready. Before merging it, every pair with one end in the accumulated part of u and the other in v contributes when $d_1 + d_2 + 1 = k$.

13.2 Merge, proof, and complexity

For each child and each $0 \leq d < k$, add $\text{cnt}[v][d] * \text{cnt}[u][k-d-1]$ to the answer, then shift $\text{cnt}[v][d]$ into $\text{cnt}[u][d+1]$. Every pair has a unique lowest common ancestor; it is counted exactly when the second relevant child subtree merges there. No pair in one child is recounted because that child already handled it. The bounded $k \leq 500$ gives $O(nk)$ time and memory.

Test a path, a star, $k=1$, and k larger than the diameter. Count in `long long`; merge cross-pairs before shifting the child table.

14 13. 580D – Kefa and Dishes

Official: <https://codeforces.com/problemset/problem/580/D>

14.1 State captures the only relevant history

Use `dp[mask][last]`: maximum satisfaction for exactly the dishes in `mask` ordered with `last` final. Append an unused dish `next`:

```
dp[mask | bit(next)][next] = max(current,
                                   dp[mask][last] + base[next] + bonus[last][next])
```

Start each one-dish mask with its base value, and maximize states with `popcount(mask)=m`.

14.2 Proof and costs

The future bonus depends only on the final dish and the used set; earlier order details have already contributed their adjacent bonuses. Inductively, each transition appends every possible final dish to every valid shorter ordering, so the table considers every legal ordered selection exactly by its best value. There are $2^n * n$ states and $O(2^n * n^2)$ transitions.

Test asymmetric bonuses, `m=1`, and a bonus that makes a lower-base dish worth choosing. The common wrong state is `dp[mask]`: it loses the predecessor needed to price the next append.

15 14. 1245D – Shichikuji and Power Grid

Official: <https://codeforces.com/problemset/problem/1245/D>

15.1 Virtual-source reduction

Create vertex 0. Edge $(0, i)$ means build a station in city i , with the given station cost. Edge (i, j) means wire the two cities, with the supplied Manhattan-based cost. Any plan that powers all cities corresponds to a connected subgraph containing 0, and vice versa. The minimum plan is therefore an MST.

15.2 Algorithm and proof

Generate all $O(n^2)$ city edges plus source edges and run Kruskal, recording source edges as stations and the others as wires. An MST is no heavier than any connected subgraph because removing cycles never disconnects it or increases cost. The correspondence proves its output is an optimal power plan. With $n \leq 2000$, $O(n^2 \log n)$ is acceptable.

Use 64-bit costs. Test that a very expensive station is replaced by a wire, that two cheap stations remain separate, and that positive coordinates have a large Manhattan difference.

16 15. 271D – Good Substrings

Official: <https://codeforces.com/problemset/problem/271/D>

16.1 Enumerate starts, identify contents exactly

The length limit permits $O(n^2)$ substring enumeration. For each start, extend the end while maintaining the bad-character count; stop once it exceeds k . Insert the traversed characters into a trie. A trie node represents one unique prefix from the root and hence one unique substring; creating a node counts a new distinct valid substring. With at most $n(n+1)/2$ nodes and 26 lowercase transitions per node, an `array<int,26>` trie fits the stated 512 MB limit in C++ at $n=1500$.

16.2 Correctness and testing

The maintained bad count is exactly the count in the current extension, so the loop inserts every and only valid occurrences. Equal contents follow the same trie edges and end at the same node; distinct contents diverge at their first different character and end at different nodes. Thus the created-node count is exactly the desired number. Work is $O(n^2)$ time and $O(n^2 * \text{alphabet})$ integer storage in the fixed-alphabet implementation.

Test repeated characters, all-good/all-bad masks, $k=0$, and repeated equal substrings from different starts. Do not insert `(start,end)`: that counts occurrences, not distinct strings. A double hash is a smaller-memory optional Python strategy, but it is probabilistic and is not the proof-backed default.

17 16. 61E – Enemy Is Weak

Official: <https://codeforces.com/problemset/problem/61/E>

17.1 Split each decreasing triple at its middle

For each j , multiply the number of earlier values greater than $a[j]$ by the number of later values smaller than $a[j]$. Sum the products. Coordinate compress values; a Fenwick tree scans left-to-right for earlier counts, and a second scan right-to-left obtains later counts.

17.2 Proof and complexity

Every valid triple has one unique middle index j , and choosing either valid endpoint is independent once $a[j]$ is fixed. Thus the product counts exactly the triples centered at j , with no overlap between different middles. Fenwick updates and prefix queries make the time $O(n \log n)$ and memory $O(n)$.

For duplicates, strict comparisons require `seen - prefix(rank)` and `prefix(rank-1)`; this task gives distinct values, but writing the strict forms prevents a bad habit. Test sorted increasing, sorted decreasing, and $n < 3$.

18 17. 86D – Powerful Array

Official: <https://codeforces.com/problemset/problem/86/D>

18.1 Maintain the statistic while moving a range

Order queries by Mo blocks. Keep `freq[x]` and current answer `sum x * freq[x]^2`. Adding value `x` changes its contribution from `x*f^2` to `x*(f+1)^2`; remove the old contribution, increment, and add the new one. Removal is the reverse. Move `L` and `R` one step at a time to each next query, then store the current answer at its original query index.

18.2 Why it works

The invariant is exact: `freq` represents precisely the current interval and the answer is the sum of each value's contribution from those frequencies. Each primitive move preserves it by replacing only the changed value's term. Mo ordering changes visit order, not query contents, so each saved answer is correct. Typical complexity is $O((n+q) \sqrt{n})$ moves with $O(n)$ memory.

Use `long long`. Test repeated one value, moving both endpoints backward, and queries returned in a different order than their sorted processing order.

19 18. 498C – Array and Operations

Official: <https://codeforces.com/problemset/problem/498/C>

19.1 Decompose operations by prime

Factor every number. Fix a prime p : each exponent at an odd-index vertex is supply, each exponent at an even-index vertex is demand, and each allowed odd-even pair is an infinite-capacity edge. The maximum number of p -units removed is a max flow. Sum independently over all primes that occur.

19.2 Reduction proof

Any allowed division by a composite can be split into one operation per prime factor on the same pair. This preserves feasibility and increases (or at least preserves) the number of counted operations, so some optimum uses prime-only operations. A flow unit travels from an odd exponent through an allowed pair to an even exponent, exactly representing one removable common prime unit. Capacities prevent using an exponent unit twice. Conversely every sequence of prime-unit operations yields such a flow. Therefore max flow equals the optimum count for that prime, and primes do not interact. Factorization plus Dinic per occurring prime is feasible for $n, m \leq 100$.

Normalize each pair orientation by index parity. Test no shared prime, one large repeated prime exponent, and a path where flow must choose between two neighbors.

20 19. 319C – Kalila and Dimna in the Logging Industry

Official: <https://codeforces.com/problemset/problem/319/C>

20.1 Derive lines before choosing a hull

The recurrence has form

$$dp[i] = \min_{j < i} (dp[j] + b[j] * a[i])$$

For fixed j , this is the line $y = b[j] * x + dp[j]$ queried at $x=a[i]$. The base case is $dp[1]=0$: insert line $y=b[1]x$ before querying for $i=2$. After computing $dp[i]$, insert $y=b[i]x+dp[i]$. The stated monotonicity gives query $a[i]$ values increasing and slopes $b[j]$ decreasing, so a monotone lower hull with a moving query pointer works; a Li Chao tree is a robust alternative.

20.2 Hull invariant and proof

Keep candidate lines in slope order. When a new line makes the intersection of the last two candidates lie no later than the prior intersection, the middle line is never minimal for any future increasing query and can be removed. The front pointer advances only while its successor is no worse at the current query. Thus the queried line is minimum among all inserted lines, which equals the recurrence minimum by induction. The monotone hull runs in $O(n)$ time; Li Chao gives $O(n \log \text{coordinate range})$.

Use cross multiplication with wide integers for redundancy tests, avoiding floating point. Test equal-looking line values at a query, a line optimal only for a short interval, and values near 10^9 .

21 20. 786B – Legacy

21.1 Reduction

An interval edge is not one edge: expanding it naively can be quadratic. Build two segment-tree graphs. An “out” tree has zero-cost edges from a parent to its children, so one vertex can reach every leaf in an interval through $O(\log n)$ nodes. An “in” tree reverses those zero-cost edges, so every leaf in an interval can reach one vertex through $O(\log n)$ nodes. Connect the original vertex to the canonical out-tree cover, or the in-tree cover to the original vertex, with the requested weight. Dijkstra on this expanded nonnegative graph is then ordinary shortest path. Prove both directions: every range operation has a zero-cost tree route plus exactly one weighted edge, and every such route represents an allowed range operation.

22 21. 191C – Fools and Roads

22.1 Tree difference invariant

Root the tree. For a path (u, v) , add one at u and v , subtract two at $\text{lca}(u, v)$. Process vertices in postorder, adding each child's accumulated value into its parent. The final value stored at a child equals the number of query paths crossing that child's parent edge: a path contributes one net unit to exactly the two branches leaving its LCA. This turns all paths into $O(1)$ updates plus LCA queries. Use binary lifting and 64-bit counts.

23 22. 208E – Blood Cousins

23.1 Lift, group by depth, count an interval

Find the p -th ancestor a of v by binary lifting. All requested cousins are exactly vertices at $\text{depth}[v]$ inside a 's subtree. A DFS gives every subtree a contiguous entry-time interval; for every depth, store sorted entry times of vertices at that depth. Two binary searches count that depth list inside $[\text{tin}[a], \text{tout}[a]]$, then subtract one for v . If the ancestor does not exist, the answer is zero. The definition, rather than the family story, is what makes the reduction precise.

24 23. 375D – Tree and Queries

24.1 Keep the heavy subtree's frequency table

Use DSU on tree. Solve light children with a cleared frequency table, solve the heavy child and keep its table, then add every vertex of each light subtree. Maintain `count[color]` and a second frequency-of-frequencies structure so a query asks how many colors have count at least `k`. The heavy child is kept because every vertex is rebuilt only for light-edge ancestors, giving $O(n \log n)$ additions. The important invariant is that the maintained table contains exactly the current subtree when its query is answered.

25 24. 600E – Lomsat gelral

25.1 Small-to-large maps

For each subtree, keep a frequency map and the current largest frequency plus the sum of colors achieving it. Merge the smaller child's map into the larger map; when a color's count rises, update the maximum and sum carefully. Every map entry moves only into a map at least twice as large, so total map work is $O(n \log n)$ expected with hash maps. The answer for a vertex is read after all children are merged. Reset the sum when a strictly larger frequency appears.

26 25. 877E – Danil and a Part-time Job

26.1 Euler-tour range toggles

An Euler tour maps every subtree to one contiguous interval. A segment tree stores the number of ones in an interval; a lazy flip turns a segment sum s into $\text{length} - s$ and toggles one lazy bit. Range flip and range sum are then standard. The lazy flag means “every value under this node has been inverted an odd number of times not yet pushed”; applying a second flip cancels it. Test two flips of the same subtree and a leaf.

27 26. 242E – XOR on Segment

27.1 Store bit counts, not only the sum

For every segment and bit position, store how many values have that bit set. The segment sum is the weighted sum of these counts. Xoring a mask flips every selected bit, replacing its count by `length - count`; the mask is therefore a lazy tag combined by xor. A plain range sum cannot be updated after xor because carry interactions hide which bits changed. The bit-count representation makes each flipped bit independent. Use enough bits for the stated value range.

28 27. 103D – Time to Raid Cowavans

28.1 Split queries by step size

For small step b , precompute suffix-style sums $dp[i][b] = a[i] + dp[i+b][b]$; each query is then $O(1)$. For large b , directly jump through the array, which takes only $O(n / b)$ positions. Choose a threshold near $\sqrt{n}$: precomputation costs $O(n \sqrt{n})$, and every large-step query is also $O(\sqrt{n})$. This is a standard time-memory tradeoff; indexing must match the problem's one-based positions.

29 28. 1000F – One Occurrence

29.1 Sweep by right endpoint

When processing a right endpoint r , record for each value its latest and previous occurrence. A value is unique in $[1, r]$ exactly when its latest occurrence is at least 1 and its previous occurrence is before 1. Maintain at the latest position a score equal to its previous occurrence, removing the old latest position when the value repeats. A segment tree over positions can find any position in $[1, r]$ whose score is below 1. Answer queries sorted by right endpoint. The score is a compact certificate of uniqueness.

30 29. 240F – TorCoder

30.1 Count letters, then rebuild the range

Use 26 segment trees or one tree storing a 26-entry frequency vector. Query a range's counts, clear each letter count in that range, then assign contiguous blocks back in alphabetic or reverse-alphabetic order. Lazy assignment tags are better than moving characters one by one: assigning a node to one letter sets one count to its length and all others to zero. The queried multiset is preserved and the reconstructed blocks are exactly its sorted order. The factor 26 is constant.

31 30. 343D – Water Tree

31.1 One tree, two interval families

Euler order makes a subtree one interval. Heavy-light decomposition makes a root-to-vertex path a logarithmic number of intervals. Store binary fill state in a lazy segment tree supporting range assignment to zero or one and point query. Filling a subtree is one Euler interval; emptying a path is a sequence of HLD intervals. Assignment tags overwrite prior assignments, which matches the operation semantics. Keep the two coordinate systems distinct in your proof: both refer to the same base-array positions.

32 31. 1209E2 – Rotate Columns

32.1 Keep only useful columns, then subset DP

Only the largest few columns can matter: a selected value for any row must be among the strongest candidates for that row, so retain at most n promising columns. For one retained column, try every rotation and compute, for every row subset, the best sum this column can contribute to that subset. DP over row masks combines columns, assigning each row once. The state is small because the row count is small; the large column count is removed before DP. Verify the rotation is chosen once per column, not separately per row.

33 32. 1416C – XOR Inverse

33.1 Decide each xor bit independently

Build a binary trie recursively over value bits. At one bit, all cross-pairs between the zero and one groups have their relative order determined by whether the corresponding bit of x is zero or one. Count inversions for both choices while recursively solving inside groups; lower bits cannot affect pairs already separated at this bit. Choose the cheaper option independently and set that bit of x accordingly. The divide proof is lexicographic: the highest differing bit decides an inversion before any lower bit is examined.

34 33. 13E – Holes

34.1 Sqrt-decompose jump chains

Divide indices into blocks. For every position, precompute the first position outside its block reached by repeatedly jumping, the number of jumps needed, and the last in-block position. A query follows only block exits, so it visits $O(\sqrt{n})$ blocks. After a point update, recompute its block backward because each precomputed state depends only on later positions in that block. This is faster than recomputing all jump paths and avoids recursion depth issues.

35 34. 342E – Xenia and Tree

35.1 Centroid ancestors summarize red updates

Build a centroid decomposition. For every original vertex, keep its list of centroid ancestors and distances to them. A red update at v minimizes `best[centroid]` with that distance along all its centroid ancestors. A query at u minimizes `best[c] + dist(u,c)` over u 's centroid ancestors. Any path from a queried vertex to a red vertex meets at their relevant centroid split, which makes this summary complete. Both operations are $O(\log n)$.

36 35. 895C – Square Subsets

36.1 Parity masks of prime exponents

The product is a square exactly when every prime exponent is even. Factor each bounded value into a bitmask of exponent parities. DP over masks counts ways to choose processed values; selecting a value xors its mask. Equal values are grouped, and their even/odd subset counts multiply transitions efficiently. Value one has mask zero but still doubles subset choices, so handle it carefully. Subtract the empty subset from the final zero-mask count.

37 36. 868F – Yet Another Minimization Problem

37.1 Divide-and-conquer DP with a movable cost window

Let $dp[g][i]$ be the minimum cost to partition the first i values into g segments. Its transition minimizes $dp[g-1][j] + cost(j+1, i)$. The optimal split points are monotone for this cost, enabling divide-and-conquer optimization. Maintain $cost(l, r)$ with two pointers and frequency counts: when adding a value with current frequency f , it creates f new equal pairs. Compute each DP row recursively over index intervals with narrowed candidate split ranges. Prove the monotonicity theorem before applying this optimization.

38 37. 1292C – Xenon's Attack on the Gangs

38.1 DP on ordered endpoint pairs

Root the tree once for subtree sizes. For an ordered pair (u, v) , move u toward v and move v toward u ; the removed-side component sizes form the current product, while future choices reduce one endpoint path at a time. Memoize the best product reachable from each pair. The recursion terminates as their distance decreases. LCA and binary lifting identify the child on a path, so each transition knows the relevant component size. Do not confuse subtree size with the component on the opposite side of an edge.

39 38. 613D – Kingdom and its Cities

39.1 Virtual tree preserves only relevant ancestry

Sort marked vertices by DFS entry time, add LCAs of adjacent marked vertices, and connect them with a stack to form the virtual tree. Every path interaction between marked vertices occurs on this compressed tree; unrelated original vertices only form edge lengths. A bottom-up DP decides whether a virtual-tree node carries an unresolved marked requirement or must be removed. If a marked vertex has a marked parent adjacent in the virtual tree with no removable vertex between them, report impossibility. Building and solving the virtual tree is $O(k \log k)$ per query.

40 39. 1110F – Nearest Leaf

40.1 Reroot distances as range additions

Euler-order leaves so each subtree is an interval. For a fixed query vertex, distances to leaves can be represented in a segment tree over that order. When rerooting across an edge of weight w , distances to leaves inside the child's subtree decrease by w while all other leaf distances increase by w ; these are lazy range additions. DFS carries this representation, answers interval minimum queries, and undoes updates on return. The invariant is that the tree stores distance from the current DFS root to every leaf.

41 40. 587C – Duff in the Army

41.1 Merge only the smallest needed identifiers

Precompute binary-lifting ancestors. Alongside every jump, store the sorted list of its smallest at most ten person identifiers. Merge two such lists by a two-pointer merge truncated to ten. Decompose a path through its LCA and merge the stored lists of its upward jumps, including the LCA's local people. Larger identifiers can be discarded because a query never asks for more than the fixed limit. This truncation is the key state-compression proof.

42 41. 786C – Till I Collapse

42.1 Divide answer values by their first segment endpoint

For a fixed k , greedy segmentation is optimal because extending a segment until the next item would introduce a $(k+1)$ -st distinct value leaves the longest possible first segment. Computing this independently for all k is too slow. Use divide and conquer over k values: evaluate a midpoint greedily with a data structure that finds the next position where distinct-count exceeds the limit, then use monotonicity of segment endpoints/counts to restrict work. The implementation is demanding; first prove why the greedy partition is unique in its earliest-cut form.

43 42. 1093G – Multidimensional Queries

43.1 Manhattan distance is a maximum over sign masks

For points p, q , $|p - q|_1$ equals the maximum over all 2^d sign vectors of $\text{sign} \cdot p - \text{sign} \cdot q$. Store for every segment-tree node the minimum and maximum projection for every sign mask. Merge nodes componentwise; a query's answer is the maximum $\max[\text{mask}] - \min[\text{mask}]$. Point updates rebuild one leaf and its ancestors. With $d \leq 5$, the mask factor is small and exact.

44 43. 321E – Ciel and Gondolas

44.1 Prefix costs plus monotone divide-and-conquer DP

Precompute every group cost from two-dimensional prefix sums. Let $dp[g][i]$ be the best cost for the first i people in g groups. The recurrence tries the last cut j ; the problem's quadrangle/Monge structure makes optimal cuts monotone as i grows. Divide-and-conquer computes a DP row while searching only the inherited cut interval. The common bug is counting each pair twice: derive the exact group-cost formula against a two-person example first.

45 44. 896C – Willem, Chtholly and Seniorious

45.1 Ordered disjoint intervals, with stated randomness assumptions

Keep maximal equal-value intervals in an ordered map. Split intervals at query boundaries, operate on whole intervals, then merge adjacent equal intervals when appropriate. Assignment replaces a range by one interval; addition changes each covered interval; kth and power-sum scan the interval multiset. This is an ordered-disjoint-tree technique whose practical complexity relies on the problem's pseudorandom operations, not a universal worst-case guarantee. Do not use it when adversarial updates are possible.

46 45. 833B – The Bakery

46.1 Divide-and-conquer DP with dynamic distinct cost

The recurrence is $dp[g][i] = \max_j dp[g-1][j] + \text{distinct}(j+1, i)$. Maintain the distinct count of a moving interval using last occurrences: when extending the right endpoint, the score changes according to whether that value was already present. The required optimization proves monotonicity of optimal cut positions, then computes each row by divide and conquer while moving the cost window. This is the maximization counterpart of the previous segmentation DP; write the exact interval convention before coding.

47 46. 311B – Cats Transport

47.1 Remove station travel before partitioning waiting times

Convert every cat's arrival to its effective time at the final station by subtracting its station's prefix travel distance. Sort these values. If one bus serves a consecutive group, its best departure time is the last effective arrival in that group, making group cost a prefix-sum expression. The remaining problem is partition DP with monotone optimal cuts, solved by divide and conquer optimization. The reduction from physical stations to effective times is the important modeling step.

48 47. 55D – Beautiful Numbers

48.1 Digit DP over divisibility state

Every nonzero decimal digit divides 2520, so the least common multiple of the digits seen so far can be represented by one of the divisors of 2520. Use digit DP with state (`position`, `remainder modulo 2520`, `lcm_state`, `tight`). Appending a digit updates the remainder and, for a nonzero digit, the LCM. At the end, accept exactly when the remainder is divisible by that LCM. This state is sufficient because future digits need only the current remainder and the set-of-divisibility requirements compressed into their LCM. Count up to each endpoint and subtract.

49 48. 840D – Destiny

49.1 Random sampling plus exact range counts

If a value appears more than length / k times, there can be fewer than k such heavy values. Randomly sample several positions from the query range; each heavy value has noticeable chance to be sampled. Pre-store sorted occurrence positions for every value, so two binary searches verify a sampled candidate's exact frequency. This is a randomized algorithm: the proof is a failure-probability bound over independent samples, not a guarantee from one sample. Always verify candidates; sampling alone can return a false answer.

50 49. 911G – Mass Change Queries

50.1 Push value transitions through a segment tree

Each node stores a mapping from every possible input value to its current output value. A range operation replacing x by y composes this mapping at covered nodes by redirecting outputs equal to x . Lazy composition means an operation need not visit individual array positions. After all operations, push mappings down to leaves and apply each leaf's mapping to its original value. The invariant is functional composition: a node tag represents exactly the sequence of replacements applied to its entire segment.

51 50. 1100F – Ivan and Burgers

51.1 Offline ranges with a position-aware xor basis

Sweep right endpoints. Maintain a linear basis of xor vectors, but attach to each basis vector the latest array position needed to form it. When inserting a new value, Gaussian-eliminate by highest bit and keep the representation with the larger minimum position. For a query $[l, r]$ answered at r , use only basis vectors whose stored position is at least l ; greedily add them when they increase the xor. The position annotation is the invariant that makes one global basis answer many left boundaries exactly.

52 D. Ribbon Cuts (Offline Blind Capstone)

Official companion: <https://codeforces.com/problemset/problem/189/A>

52.1 Restatement and state

Use the ribbon exactly and maximize the number of pieces whose lengths belong to $\{a, b, c\}$. Greedily taking the shortest length can leave an uncuttable remainder, so record the best answer for every exact used length.

Let $dp[x]$ be the maximum number of pieces that form total length x , or an unreachable sentinel. Set $dp[0]=0$. For every x from one through n , append any allowed cut d when $d \leq x$:

```
dp[x] = max(dp[x-d] + 1 for d in {a,b,c} with d <= x)
```

If $dp[n]$ remains unreachable, output -1 .

52.2 Correctness proof

Every valid cutting of length x has a final piece of length d ; removing it leaves a valid cutting of $x-d$. By induction, $dp[x-d]$ stores the largest piece count for that remainder, so appending d considers the best cutting with that final piece. Taking the maximum considers all possible final pieces. Conversely every transition appends an allowed piece to an exact cutting, so it is valid. Thus $dp[n]$ is exactly the requested optimum, or unreachable.

52.3 Complexity and tests

There are n states and three transitions per state: $O(n)$ time and $O(n)$ memory. Test an impossible remainder, equal cut lengths, and a case where the shortest cut is not part of an optimum. The local tiny oracle is a recursive exact-cut search; the scale case has $n=200000$.

52.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,a,b,c;if(!(cin>>n>>a>>b>>c))retu
```

52.5 Full Python solution

```

import sys
def main() -> None:
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data:return
    n,*cuts=data;dp=[-10**9]*(n+1);dp[0]=0
    for length in range(1,n+1):
        for cut in cuts:
            if cut<=length:dp[length]=max(dp[length],dp[length-cut]+1)
    print(dp[n] if dp[n]>=0 else -1)
if __name__=='__main__':main()

```

53 Stress and carry-forward

For Gates 1–2, compare random small cases to brute force: enumerate windows, paths, subsets, or point classifications. For Gates 3–4, first validate the model on tiny instances: exhaustive dish orders, a brute MST, exact substring sets, all triples, a direct range score, and a small flow implementation. The reusable lesson is not the task name. It is the chain:

```
statement signal -> state/invariant -> proof obligation -> update rule -> test
```


54 Offline Companion Editorials

The linked contest tasks above are external judge problems. The following entries are the full editorials for their original local companions. Each contains a complete C++ and Python reference implementation, a correctness argument, complexity analysis, and hostile-test guidance. The mapping is deliberate: use the companion to rehearse the course skill offline, then return to the linked contest task without treating the companion as a replacement for its statement.

54.1 1. 71A – Way Too Long Words (800): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/71/A> Local package: [problems/01_way_too_long_words](#)

54.2 Restatement

For each print job, split n records into pages of capacity k . Print the number of pages and the unused capacity of the final page. The $n=0$ case has no final page, so both answers are zero.

54.3 Constraints and traps

There are up to 200000 queries and both values reach 10^{18} , so simulating records or pages is impossible. The familiar expression $(n+k-1)/k$ can overflow in C++ when $n+k-1$ exceeds signed 64-bit range. Use $(n-1)/k+1$ only after separating zero.

54.4 Sample walk-through

For $n=25, k=6$, four pages hold 24 records, so one more page is required:

```
pages = (25-1)/6 + 1 = 5
empty = 5*6 - 25 = 5
```

For $n=8, k=7$, two pages are needed and the final page has six empty slots.

54.5 Algorithm

```

for each n, k:
    if n == 0: print 0 0
    otherwise:
        pages = (n-1) // k + 1
        empty = pages*k - n
        print pages, empty

```

54.6 Correctness proof

For $n > 0$, integer division $(n-1)//k$ counts the number of complete capacity blocks before the final record. Adding one therefore gives the unique smallest positive number of pages whose total capacity is at least n . Those pages have capacity $\text{pages} * k$; subtracting the n occupied slots gives exactly the empty slots on the last page. For $n=0$, the statement defines both values as zero. Thus every printed pair is correct. $\square$

54.7 Complexity

Each query uses $O(1)$ time and $O(1)$ extra memory; total time is $O(Q)$.

54.8 C++ implementation notes

Use `long long`. Here $\text{pages} * k$ is at most $n+k-1$ in mathematical value and fits under the stated bounds, but do not rewrite the ceiling division as a potentially overflowing addition.

54.9 Python implementation notes

Python integers are arbitrary precision, but $(n-1)//k+1$ keeps the same zero-case semantics and mirrors the proof.

54.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q; cin >> q;
    while (q--) {
        long long n, k; cin >> n >> k;
        if (n == 0) { cout << "0 0\n"; continue; }
        long long pages = (n - 1) / k + 1;
        cout << pages << ' ' << pages * k - n << '\n';
    }
}

```

54.11 Full Python solution

```
for n, k in queries:
    if n == 0:
        print(0, 0)
    else:
        pages = (n - 1) // k + 1
        print(pages, pages * k - n)
```

54.12 Common wrong answers and tests

Do not print one empty page for $n=0$; do not use floating-point ceiling; do not loop one page at a time. Test $n=0$, $n=k$, $n=k+1$, $k=1$, and values near 10^{18} .

55 Stress testing strategy

For this section, stress testing means generating many small and boundary-heavy inputs and comparing the submitted program against a trusted implementation. The local judge does that through each problem's `tests/random_cases.py`.

The pattern is:

```
random input
|
+-- trusted solution computes expected output
|
+-- student solution computes actual output
|
+-- judge compares the two outputs
```

This is not a replacement for proof. It is a way to catch parser mistakes, off-by-one errors, and overflow cases that samples may miss.

Useful random generation is not uniformly random only. It should deliberately include edges:

- smallest values;
- largest values;
- exact thresholds;
- values just below and just above thresholds;
- negative values when allowed;
- many test cases with short arrays;
- one test case with a larger array.

56 What to carry forward

The algorithms in this section are simple. The habits are not:

- read constraints before coding;
- distinguish count limits from value limits;
- choose the simplest algorithm that fits;
- prove loop invariants even for easy loops;
- test boundaries, not only typical samples;
- be explicit about integer size.

These same habits will be used in the next sections for simulation, prefix sums, sorting, two pointers, and eventually harder contest problems.

56.1 2. 4C – Registration System (1300): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/4/C> Local package: [problems/02_registration_system](#)

56.2 Restatement

A circular carousel starts at a numbered position. Process left and right commands with wraparound, then output the final position and the smallest position with the maximum visit count. The starting position is already one visit.

56.3 Constraints and traps

Across the input, $n + \text{len}(s)$ is at most 200000, so one direct command scan and one scan of the visit array are intended. Do not forget the starting visit. The wrap rules are asymmetric at the endpoints: left from one reaches n , while right from n reaches one. A tie for most visits is resolved by the smallest index, so only replace the current best when a strictly larger count appears.

56.4 Walk-through

For $n=5$, start 1, and commands RRLLR:

```
position: 1(start), 2, 3, 2, 1, 2
visits:   v[1]=2, v[2]=3, v[3]=1, v[4]=0, v[5]=0
```

The output is 2 2: final position two, and slot two has the most visits.

56.5 Algorithm

```
visits[1..n] = 0
visits[position] += 1
for command in s:
    move one circular step left or right
    visits[position] += 1
best = smallest index with maximum visits[index]
print position, best
```

56.6 Correctness proof

After processing the first i commands, maintain: `position` is the exact carousel location after those commands, and `visits[j]` is the number of times position j has occurred among the start and those i moves. Initialization records the starting position. Each command applies exactly its stated circular move and increments exactly the newly reached position, preserving the invariant. After all commands, the first scan value is the required final position. Scanning indices in increasing order and replacing `best` only for a strictly larger count returns the smallest maximum-count index. Thus both printed values are correct.

56.7 Complexity and language notes

Time is $O(n + \text{len}(s))$ and memory is $O(n)$. C++ uses `vector<int>` indexed one through n ; Python uses a list of length $n+1$. Keep the endpoint cases explicit rather than relying on negative array indexing.

56.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int t; if (!(cin >> t)) return 0;
    while (t--) {
        int n, pos; string s; cin >> n >> pos >> s;
        vector<int> visits(n + 1); ++visits[pos];
        for (char move : s) {
            if (move == 'R') pos = pos == n ? 1 : pos + 1;
            else pos = pos == 1 ? n : pos - 1;
            ++visits[pos];
        }
        int best = 1;
        for (int i = 2; i <= n; ++i) if (visits[i] > visits[best]) best = i;
        cout << pos << ' ' << best << '\n';
    }
}
```

56.9 Full Python solution

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)
out = []
for _ in range(int(next(it))):
    n, pos = int(next(it)), int(next(it))
    visits = [0] * (n + 1)
    visits[pos] += 1
    for move in next(it).decode():
        pos = 1 if move == "R" and pos == n else pos + 1 if move == "R" else n if pos == 1 else pos
        visits[pos] += 1
    best = max(range(1, n + 1), key=lambda slot: (visits[slot], -slot))
    out.append(f"{pos} {best}")
print("\n".join(out))
```

56.10 Tests that matter

Test $n=1$, a move across each wrap boundary, a path returning to its start, and a tie where every position is visited once. The random generator varies small circular sizes and compares every command against a direct reference.

56.11 3. 363B – Fence (1100): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/363/B> Local package: [problems/03_fence](#)

56.12 Restatement

Count index pairs (i, j) with $i < j$ whose two array values are equal. This is a blind-practice problem: it uses the frequency-table idea from the section, but the lesson does not give this reduction or its update order.

56.13 Constraints and traps

The total input length is at most 200000, while values may be negative or as large as 10^9 . A frequency array indexed by value is therefore unsafe. The answer can be as large as $n(n-1)/2$, so C++ needs `long long`.

The phrase “pairs of indices” matters. Four equal values contribute six pairs, not one pair of values.

56.14 Observation and trace

Process the array left to right. Before a current value x is inserted into the table, `freq[x]` is exactly the number of earlier indices holding x . Each such index forms one new equal pair ending at the current position.

For 1 2 1 2 1:

current value	earlier equal values	new pairs	running answer
1	0	0	0
2	0	0	0
1	1	1	1
2	1	1	2
1	2	2	4

56.15 Algorithm

```
freq = empty map
answer = 0
for x in the array from left to right:
    answer += freq[x]
    freq[x] += 1
print answer
```

56.16 Correctness proof

Before processing position j , the invariant is that `freq[x]` equals the number of indices $i < j$ with value x , and `answer` counts all valid pairs whose right endpoint is before j . The current position forms a valid pair exactly with those `freq[a[j]]` earlier equal positions. Adding that number counts every new pair once, then increasing the frequency restores the invariant for the next position. By induction, the final answer is the number of all required pairs.

56.17 Complexity

Expected time is $O(n)$ per total input length with a hash map, and memory is $O(d)$ where d is the number of distinct values.

56.18 Implementation notes and tests

In C++, reserve an `unordered_map` near $2*n$; in Python use `dict.get(x, 0)` or `defaultdict(int)`. Update the answer **before** incrementing the frequency. Test one element, all equal values, all distinct values, and negative repeated values. The fixed tests include the six-pair all-equal boundary; the random generator compares to a quadratic index-pair oracle.

57 Stress testing strategy

Frequency problems are easy to stress test because slow reference solutions are short.

For Problem A, the slow reference can scan the array for every query.

For Problem B, the slow reference can sort values and count runs, or use a map and then compare all keys.

For Problem C, the slow reference is the direct double loop:

```
answer = 0
for i from 0 to n - 1:
    for j from i + 1 to n - 1:
        if a[i] + a[j] == k:
            answer += 1
```

That is too slow for full constraints but perfect for random tests with $n \leq 100$. The local generator uses this idea for pair counting.

Good random tests should include:

- small value ranges to create duplicates;
- negative values;
- absent query values;
- all-equal arrays;
- targets that are likely and unlikely.

58 What to carry forward

The main lesson is that counts are often a compressed form of the input. Once you know what must be counted, many problems become a map lookup plus a careful update.

This pattern will return in:

- coordinate compression;
- two-sum and k-sum variants;
- prefix-sum frequency problems;
- graph degree counting;
- combinatorics over repeated values.

58.1 Observation

Scan the array from left to right. Before processing position j , let $\text{freq}[x]$ be the number of earlier positions with value x . Exactly those $\text{freq}[a[j]]$ positions form a valid pair ending at j ; add that number before increasing the frequency. This counts every valid pair once, at its later endpoint, and no invalid pair is counted because only equal values contribute.

58.2 Correctness and complexity

The invariant is that `answer` equals the number of valid pairs entirely in the processed prefix. Adding $\text{freq}[a[j]]$ accounts for all and only pairs (i, j) with $i < j$ and equal values, then updating the map restores the invariant. By induction the final answer is correct. Expected time is $O(n)$; the map stores at most one entry per distinct value. Use 64-bit storage: an all-equal array has $n(n-1)/2$ pairs.

58.3 Tests that matter

Test all values equal, all values distinct, negative values, and several test cases whose lengths sum to the stated limit.

58.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
```

```

if (!(cin >> t)) return 0;
while (t--) {
    int n;
    cin >> n;
    unordered_map<long long, long long> freq;
    freq.reserve(2 * n + 1);
    long long answer = 0;
    for (int i = 0; i < n; ++i) {
        long long value;
        cin >> value;
        answer += freq[value];
        ++freq[value];
    }
    cout << answer << '\n';
}
return 0;
}

```

58.5 Full Python solution

```

"""Reference solution for D. Equal Index Pairs."""

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    index = 1
    out: list[str] = []
    for _ in range(t):
        n = data[index]
        index += 1
        freq: dict[int, int] = {}
        answer = 0
        for value in data[index:index + n]:
            answer += freq.get(value, 0)
            freq[value] = freq.get(value, 0) + 1
        index += n
        out.append(str(answer))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

58.6 4. 279B – Books (1400): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/279/B> Local package: [problems/04_books](#)

58.7 Restatement and recognition

Apply many inclusive range additions to an initially zero array, then report the largest final value and its smallest position. With n, q both 200000, updating every covered cell per operation is too slow. The output is needed only after all updates, which is the signal for a difference representation.

58.8 Trace and algorithm

For an add x on $[l, r]$, record $+x$ at l and $-x$ at $r+1$. Prefixing these changes reconstructs the final value. In the sample, the reconstructed values are 4, 2, 2, 7, -2, so the answer is 7 4.

```
diff[1..n+1] = 0
for each l,r,x: diff[l] += x; diff[r+1] -= x
current = 0; best = negative infinity
for i=1..n:
    current += diff[i]
    if current > best: best=current, position=i
```

58.9 Correctness proof

Each update changes the running prefix total by x exactly from l through r : the $+x$ begins it and the $-x$ cancels it just after r . Thus after prefixing, `current` equals the final signal at each scanned position. The scan records a position exactly when it has a strictly larger value than every earlier one; therefore it finishes with the maximum and its smallest position. $\square$

58.10 Complexity and implementation notes

The algorithm uses $O(n+q)$ time and $O(n)$ memory. Use `long long` in C++: many signed updates can exceed 32-bit range. Initialize the maximum below every legal final value because all updates may be negative. Use strict $>$ rather than $\geq$ to preserve the smallest tied index.

58.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
```

```

if (!(cin >> n >> q)) return 0;
vector<long long> diff(n + 2);
while (q--) {
    int left, right;
    long long delta;
    cin >> left >> right >> delta;
    diff[left] += delta;
    diff[right + 1] -= delta;
}
long long current = 0, best = LLONG_MIN;
int position = 1;
for (int i = 1; i <= n; ++i) {
    current += diff[i];
    if (current > best) best = current, position = i;
}
cout << best << ' ' << position << '\n';
}

```

58.12 Full Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
n, q = data[0], data[1]
diff = [0] * (n + 2)
index = 2
for _ in range(q):
    left, right, delta = data[index:index + 3]
    index += 3
    diff[left] += delta
    diff[right + 1] -= delta

current = 0
best = -10**30
position = 1
for i in range(1, n + 1):
    current += diff[i]
    if current > best:
        best, position = current, i
print(best, position)

```

58.13 Tests that matter

Test one position, a full-range negative update, overlapping positive and negative updates, and a tie for the maximum. The random generator uses a direct small-array update oracle and includes a near-limit $n=q=200000$ range-update case.

59 Stress testing strategy

Prefix-sum code is easy to test with a slow oracle.

For range sums, compare each query to a direct sum of the slice.

For range updates, apply each update directly to a small array and compare the final result with the difference-array result.

For subarray sums, use the quadratic loop:

```
answer = 0
for l:
    total = 0
    for r from l to n - 1:
        total += a[r]
        if total == k:
            answer += 1
```

This slow oracle caught incorrect hand-written expected values while this section was being built. That is exactly why stress tests are worth having.

60 What to carry forward

Prefix sums turn range sums into subtraction. Difference arrays turn range updates into boundary marks. Prefix-frequency maps turn subarray conditions into complement lookups.

These ideas reappear in:

- two-dimensional prefix sums;
- Fenwick trees and segment trees;
- offline range queries;
- subarray counting with modular conditions;
- dynamic programming transitions over intervals.

60.1 5. 158B – Taxi (1100): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/158/B> Local package: [problems/05_taxi](#)

60.2 Restatement and constraints

Choose exactly k values whose maximum minus minimum is as small as possible. With total n up to 200000, enumerating all k -subsets is impossible even though it is a useful tiny-input oracle. Values may be negative, and the answer can exceed 32-bit range.

60.3 Observation and walk-through

Sort the values. In an optimal chosen team, consider its smallest and largest chosen values. Every sorted value between them can replace an interior chosen value without increasing the spread, so some optimum is a consecutive block of exactly k sorted values.

For $[4, 1, 9, 7, 5]$ and $k=3$, sorting gives $[1, 4, 5, 7, 9]$:

```
[1,4,5] spread 4  
[4,5,7] spread 3  
[5,7,9] spread 4
```

The answer is three.

60.4 Algorithm

```

sort a
answer = a[k-1] - a[0]
for left from 1 through n-k:
    answer = min(answer, a[left+k-1] - a[left])
print answer

```

60.5 Correctness proof

Let S be an optimal team and let its minimum and maximum values occupy sorted positions L and R . The interval $[L, R]$ contains at least k values because it contains S . Choose any k consecutive values inside that interval including its endpoints when necessary; their minimum is no smaller than S 's minimum and their maximum is no larger than S 's maximum. Their spread is therefore at most the optimal spread, hence equal to it. The algorithm evaluates every consecutive block of k sorted values, including this optimal block, and returns the smallest evaluated spread. Thus it is correct.

60.6 Complexity and implementation notes

Sorting costs $O(n \log n)$ and the scan costs $O(n)$; memory is $O(n)$. Use `long long` in C++ for subtraction. Python integers are exact. For $k=1$, every block has spread zero and the same loop handles it.

60.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int t; cin >> t;
    while (t--) {
        int n, k; cin >> n >> k;
        vector<long long> a(n);
        for (auto& x : a) cin >> x;
        sort(a.begin(), a.end());
        long long answer = a[k - 1] - a[0];
        for (int left = 1; left + k <= n; ++left)
            answer = min(answer, a[left + k - 1] - a[left]);
        cout << answer << '\n';
    }
}

```

60.8 Full Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))

```



```

index, out = 1, []
for _ in range(data[0]):
    n, k = data[index], data[index + 1]
    index += 2
    a = sorted(data[index:index + n])
    index += n
    out.append(str(min(a[i + k - 1] - a[i] for i in range(n - k + 1))))
print("\n".join(out))

```

60.9 Tests that matter

Test $k=1$, $k=n$, all equal values, negative values, duplicate values, and two values whose difference exceeds signed 32-bit range. The random generator uses combinations as an independent small oracle and includes a 200000-value case to enforce the intended sorting complexity.

60.10 6. 706B – Interesting Drink (1100): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/706/B> Local package: [problems/06_interesting_drink](#)

60.11 Restatement

Find the shortest nonempty contiguous segment that contains every label from 1 through m . If at least one label never occurs, the answer is -1 .

This is the section's blind-practice task. It still uses a moving window, but the window condition is coverage rather than a numeric sum.

60.12 Constraints and traps

The total array length is 200000, so checking every pair of endpoints is too slow. Labels are bounded by m , making a frequency array of size $m+1$ appropriate. A window containing many copies of one label is not complete unless it contains every *different* required label.

60.13 Observation and trace

Keep a window and a counter **missing**: the number of labels that occur zero times in the window. Adding a label whose current count is zero decreases **missing**. Removing a label whose count becomes zero increases it.

For 1 2 1 3 2 with labels 1,2,3:

right value	window after expansion/shrinking	missing	best
1	[1]	2	none
2	[1,2]	1	none
1	[1,2,1]	1	none
3	[2,1,3] after removing the first 1	0	3

right value	window after expansion/shrinking	missing	best
2	[1,3,2] after removing the first 2	0	3

Once **missing** is zero, the current window is valid. Removing from the left is the only way to make that window shorter; keep doing so until one required label disappears.

60.14 Algorithm

```

count[1..m] = 0
missing = m
left = 0
answer = infinity

for right from 0 to n-1:
    add a[right] to count
    if it was absent before: missing -= 1

    while missing == 0:
        answer = min(answer, right-left+1)
        remove a[left] from count
        if it becomes absent: missing += 1
        left += 1

print answer, or -1 when it was never updated

```

60.15 Correctness proof

Lemma 1. At every point, **missing** equals the number of required labels absent from the current window $[left, right]$.

Proof. Initially every required label is absent. The algorithm changes **missing** exactly when adding the first occurrence of a label or removing its last occurrence. These are exactly the events that change whether a label is absent. $\square$

Lemma 2. Every candidate used to update **answer** contains every required label.

Proof. A candidate is recorded only while **missing** == 0. By Lemma 1, no required label is absent from that window. $\square$

Lemma 3. For each right endpoint, the algorithm records the shortest valid window ending at that endpoint.

Proof. While the window is valid, the algorithm removes leftmost labels one at a time and records each valid length. It stops exactly after removing the last occurrence of one required label, at which point no shorter window with the same right endpoint can be valid. Thus the final recorded valid window for that endpoint is shortest. $\square$

Theorem. The algorithm prints the minimum length of a subarray containing all required labels, or -1 when none exists.

Proof. By Lemma 2, every recorded length is feasible. Every feasible subarray has some right endpoint; by Lemma 3 the algorithm considers a valid window no longer than that subarray for the

same endpoint. Taking the minimum therefore gives the global optimum. If no window is recorded, Lemma 1 shows no window ever contained all labels, so -1 is correct. $\square$

60.16 Complexity

Each endpoint moves from left to right at most once. Time is $O(n+m)$ per test case and memory is $O(m)$.

60.17 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        int n, m;
        cin >> n >> m;
        vector<int> values(n);
        for (int &value : values) cin >> value;

        vector<int> count(m + 1, 0);
        int missing = m;
        int left = 0;
        int answer = n + 1;
        for (int right = 0; right < n; ++right) {
            if (count[values[right]]++ == 0) --missing;
            while (missing == 0) {
                answer = min(answer, right - left + 1);
                if (--count[values[left]] == 0) ++missing;
                ++left;
            }
        }
        cout << (answer == n + 1 ? -1 : answer) << '\n';
    }
    return 0;
}
```

60.18 Full Python solution

```
import sys
```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    tests = data[index]
    index += 1
    out: list[str] = []
    for _ in range(tests):
        n = data[index]
        labels = data[index + 1]
        index += 2
        values = data[index:index + n]
        index += n

        count = [0] * (labels + 1)
        missing = labels
        left = 0
        answer = n + 1
        for right, value in enumerate(values):
            if count[value] == 0:
                missing -= 1
            count[value] += 1
            while missing == 0:
                answer = min(answer, right - left + 1)
                count[values[left]] -= 1
                if count[values[left]] == 0:
                    missing += 1
                left += 1
            out.append(str(-1 if answer == n + 1 else answer))
        sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

60.19 Implementation traps and tests

- Count distinct covered labels, not the number of window elements.
- Update the answer before removing the leftmost label.
- Stop shrinking only after a count becomes zero.
- Return -1 when one required label is absent from the whole array.

Test one required label, a missing label, a shortest window at the beginning or end, and windows with many duplicate labels. The random generator uses a quadratic endpoint oracle on small arrays.

61 Stress testing strategy

All three problems have simple slow oracles:

- sorted two-sum: all pairs;
- longest bounded window: all subarrays, keep max valid length;
- count at most k : all subarrays, count valid ones.

The local random tests compare against these slow versions on small arrays. Good tests include duplicates, zeros, limits of zero, all values too large, and arrays where the shrinking loop runs many times.

62 What to carry forward

Two-pointer algorithms are about monotonicity. A pointer move is justified only when it discards candidates that cannot become useful later.

This idea will reappear in:

- binary search predicates;
- greedy algorithms;
- sweep-line active sets;
- offline query processing;
- advanced sliding-window counting problems.

62.1 7. 520B – Two Buttons (1400): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/520/B> Local package: [problems/07_two_buttons](#)

62.2 Restatement and traps

Choose a nonempty-prefix/nonempty-suffix split minimizing the absolute difference of their sums. The smallest split index wins ties. With total n up to 200000, recomputing both sums for every split is quadratic; signed values mean the running difference need not move monotonically.

62.3 Trace and algorithm

For 3 1 4 1 5, total is 14. Prefix sums at split positions 1,2,3,4 are 3,4,8,9, so differences are 8,6,2,4. The answer is 2 3.

```
total = sum(a); left = 0; best = infinity
for i = 1..n-1:
    left += a[i]
    difference = abs(left - (total-left))
    if difference < best: store difference and i
```

62.4 Correctness proof

After processing index i , `left` is exactly the sum of positions 1 through i , so `total-left` is exactly the suffix sum for that split. The loop evaluates every legal split once. Strict replacement preserves the first index among equal minima. Therefore the stored pair is the required minimum difference and smallest position. $\square$

62.5 Complexity and language notes

Time is $O(n)$ and extra memory is $O(1)$ beyond input storage. C++ needs `long long` for totals and differences; Python integers are exact. Do not stop early when the difference rises: negative values destroy that shortcut.

62.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        int n;
        cin >> n;

        vector<long long> a(n);
        long long total = 0;
        for (long long& x : a) {
            cin >> x;
            total += x;
        }

        long long left = 0;
        long long best = LLONG_MAX;
        int position = 1;

        for (int i = 0; i < n - 1; ++i) {
            left += a[i];
            long long difference = llabs(left - (total - left));

            if (difference < best) {
                best = difference;
                position = i + 1;
            }
        }

        cout << best << ' ' << position << '\n';
    }
}
```

62.7 Full Python solution

```
total = sum(a); left = 0; best = None; position = 1
for i in range(len(a) - 1):
    left += a[i]
    difference = abs(left - (total - left))
    if best is None or difference < best:
        best, position = difference, i + 1
print(best, position)
```

62.8 Tests that matter

Test `n=2`, all zeros (tie chooses position one), negative values, a later strict improvement, and sums beyond 32-bit. The random generator uses a separate direct split oracle and includes a 200000-value scale case.

63 Contest testing strategy

For a mixed set, each problem should get a different oracle:

- Problem A: brute force over which distinct colors to keep for very small distinct counts, or compare against sorted frequencies.
- Problem B: direct time simulation for small m .
- Problem C: quadratic enumeration of all subarrays.

The local random tests use these simple references. This is not extra polish; it is how you keep contest code honest.

64 What to carry forward

Phase 0 is now complete. The foundation tools are useful only if you can select them under time pressure. The next phase begins with binary search on the answer, where the same habits continue: read constraints, identify monotonicity, prove the predicate, and test edge cases.

64.1 8. 455A – Boredom (1500): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/455/A> Local package: [problems/08_boredom](#)

64.2 Restatement and predicate

Machines run in parallel; machine i produces $\text{floor}(\text{minutes} / \text{time}[i])$ items by a proposed deadline. The question “can we make at least goal items by x ?” is monotone: if it is true at x , it stays true at every later minute.

64.3 Bounds, trace, and algorithm

Zero minutes is infeasible because the goal is positive. $\text{min}(\text{time}) * \text{goal}$ is feasible because the fastest machine can work alone; the local bounds keep it inside signed 64-bit range. With times 2 and 3, seven items need nine minutes: at eight minutes production is $4+2=6$, while at nine it is $4+3=7$.

```
lo = 0; hi = fastest_time * goal
while lo < hi:
    mid = lo + (hi-lo)/2
    if sum(mid / time[i]) >= goal: hi = mid
    else: lo = mid + 1
print lo
```

64.4 Correctness proof

For any x , each machine’s production is exactly $\text{floor}(x/\text{time}[i])$, so the predicate computes total production correctly. Increasing x cannot decrease any term, hence feasibility is monotone. The binary-search invariant keeps lo infeasible (unless it is the answer boundary) and hi feasible; each iteration retains the first feasible value in the interval. When they meet, that value is the minimum feasible deadline, which is the requested answer.

64.5 Complexity and implementation notes

Each predicate is $O(n)$, so the solution is $O(n \log(\min_time * goal))$ time and $O(n)$ memory for the input. In C++, use `long long` and stop summing once total reaches goal to avoid unnecessary growth. Python integers are safe, but the same early break is useful. Test one machine, goal one, equal machine times, and a case near the 64-bit bound. The random tests use minute-by-minute brute force only on small instances plus a 200000-machine scale case.

64.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int test_count;
    cin >> test_count;

    while (test_count--) {
        int n;
        long long goal;
        cin >> n >> goal;

        vector<long long> time(n);
        for (long long& x : time) {
            cin >> x;
        }

        long long lo = 0;
        long long hi = *min_element(time.begin(), time.end()) * goal;

        while (lo < hi) {
            long long mid = lo + (hi - lo) / 2;
            long long made = 0;

            for (long long machine_time : time) {
                made += mid / machine_time;
                if (made >= goal) break;
            }

            if (made >= goal) {
                hi = mid;
            } else {
                lo = mid + 1;
            }
        }

        cout << lo << '\n';
    }
}
```

```
}  
}
```

64.7 Full Python solution

```
import sys  
  
def minimum_deadline(time: list[int], goal: int) -> int:  
    lo = 0  
    hi = min(time) * goal  
  
    while lo < hi:  
        mid = (lo + hi) // 2  
        made = 0  
  
        for machine_time in time:  
            made += mid // machine_time  
            if made >= goal:  
                break  
  
        if made >= goal:  
            hi = mid  
        else:  
            lo = mid + 1  
  
    return lo  
  
data = list(map(int, sys.stdin.buffer.read().split()))  
i = 1  
out = []  
  
for _ in range(data[0]):  
    n = data[i]  
    goal = data[i + 1]  
    i += 2  
    time = data[i:i + n]  
    i += n  
    out.append(str(minimum_deadline(time, goal)))  
  
sys.stdout.write("\n".join(out))
```

64.8 9. 300C – Beautiful Numbers (1800): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/300/C> Local package: [problems/09_beautiful_numbers](#)

64.9 Restatement

Choose as few integer pickup days as possible so that every attendee interval contains at least one chosen day. The intervals are inclusive: an attendee available from day 2 through day 2 can only be covered on day 2.

This is the section's blind-practice exercise. The output asks for a minimum number of choices, but the safe local choice must still be justified by an exchange argument.

64.10 Observation and trace

Sort intervals by increasing right endpoint. Consider the first interval not already covered, $[l, r]$. Any feasible solution must choose some day d in that interval. Replacing d with r cannot uncover an interval that ends no earlier than r : every remaining interval has right endpoint at least r , and any interval containing d has left endpoint at most $d \leq r$.

So choose day r and continue with intervals it does not cover.

For these intervals:

```
[1,3], [2,5], [4,6], [6,6]
```

the first choice is day 3, covering the first two intervals. The next uncovered interval ends at day 6; choosing day 6 covers both remaining intervals. The answer is 2.

64.11 Algorithm

```
sort intervals by (right endpoint, left endpoint)
last_day = no chosen day
answer = 0

for [left, right] in sorted intervals:
    if no day has been chosen yet or last_day < left:
        choose right
        last_day = right
        answer += 1

print answer
```

An interval with $\text{left} \leq \text{last_day}$ is already covered, because sorted order also ensures last_day is no greater than that interval's right endpoint.

64.12 Correctness proof

Lemma 1. Let $[l, r]$ be the earliest-finishing interval not covered by previous choices. There is an optimal completion that chooses day r for this interval.

Proof. Any feasible completion chooses a day d with $l \leq d \leq r$ to cover $[l, r]$. Replace that choice by r . Every interval still awaiting coverage ends at least at r , because $[l, r]$ has the earliest

right endpoint. If such an interval contained d , its left endpoint is at most d , hence at most r ; together with its right endpoint at least r , it contains r . Thus the replacement keeps all intervals covered and does not increase the number of days. $\square$

Lemma 2. Whenever the algorithm chooses a day, it can be part of an optimal solution for all intervals not yet processed.

Proof. The selected interval is the earliest-finishing uncovered interval, so Lemma 1 applies. $\square$

Theorem. The algorithm prints the minimum number of pickup days.

Proof. Repeatedly applying Lemma 2 fixes one choice that agrees with an optimal completion. The algorithm skips exactly the intervals covered by those fixed choices and continues until all intervals are covered. Therefore it uses the same number of pickup days as some optimum. $\square$

64.13 Complexity

Sorting costs $O(n \log n)$ time. The scan costs $O(n)$ time and uses $O(1)$ extra memory beyond the interval list.

64.14 Implementation traps and tests

- Intervals are inclusive: use `last_day < left` to detect an uncovered one.
- Sort by right endpoint, not by left endpoint or interval length.
- Negative day numbers are valid; do not use `-1` as an unguarded sentinel.
- Several nested intervals need only one pickup day.

Test nested intervals, disjoint single-day intervals, intervals sharing an endpoint, and negative endpoints. The random generator enumerates subsets of right endpoints on small instances and includes a 200000-interval scale case.

65 Stress testing greedy

Greedy solutions are excellent candidates for brute-force stress testing on small inputs.

For activity selection:

```
Try every subset of intervals.  
Check whether it is non-overlapping.  
Take the largest valid subset.
```

For rescue boats:

```
Use recursion or DP over remaining people.  
Try sending the heaviest alone or with each possible partner.
```

For deadline scheduling:

```
Try every subset.  
Sort the subset by deadline.  
Check whether every completion time meets its deadline.
```

The local random generators use exactly these ideas. That is why the reference checker is more valuable than fixed samples alone.

66 What to carry forward

Greedy problems become manageable when you separate three things:

1. the local choice;
2. the proof that the local choice is safe;
3. the data structure that implements repeated local choices efficiently.

For easy greedy problems, the proof may be one exchange. For harder Codeforces, AtCoder, and ICPC tasks, the proof may be an invariant that survives many local edits. The habit is the same: never trust the local rule until you can explain why an optimal solution can be made to follow it.

66.1 Reference implementations

The proof, trace, and hostile cases immediately above establish the greedy choice. The missing part of the older source editorial was the concrete implementation. `last_day` is deliberately represented by a separate boolean or `None` state rather than a numeric sentinel because pickup days may be negative. The comparison is strict: `last_day == left` still covers an inclusive interval.

66.2 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        int n;
        cin >> n;
        vector<pair<long long, long long>> intervals(n);
        for (auto &[left, right] : intervals) cin >> left >> right;
        sort(intervals.begin(), intervals.end(), [](const auto &a, const auto &b) {
            if (a.second != b.second) return a.second < b.second;
            return a.first < b.first;
        });

        long long last_day = LLONG_MIN;
        int answer = 0;
        for (auto [left, right] : intervals) {
            if (last_day < left) {
```



```

        ++answer;
        last_day = right;
    }
}
cout << answer << '\n';
}
return 0;
}

```

66.3 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    tests = data[index]
    index += 1
    out: list[str] = []
    for _ in range(tests):
        n = data[index]
        index += 1
        intervals = [(data[index + 2 * i], data[index + 2 * i + 1]) for i in range(n)]
        index += 2 * n
        intervals.sort(key=lambda interval: (interval[1], interval[0]))

        last_day: int | None = None
        answer = 0
        for left, right in intervals:
            if last_day is None or last_day < left:
                answer += 1
                last_day = right
        out.append(str(answer))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

66.4 10. 166B – Polygons (2100, geometry challenge): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/166/B> Local package: [problems/10_polygons](#)

66.5 Restatement and feasibility

Build a length- n sequence in $[0, k]$ with sum s , choosing the lexicographically smallest valid sequence. It is impossible exactly when $s > n \cdot k$.

66.6 Greedy choice and trace

At position i , leave at most k for each remaining slot. The smallest safe value is $\max(0, \text{remaining_sum} - \text{remaining_slots} \cdot k)$. For $n=4, k=3, s=7$, choose 0, then 1, then 3, then 3.

66.7 Algorithm

```
if s > n*k: print -1
for i=0..n-1:
    x=max(0,s-(n-i-1)*k)
    print x; s-=x
```

66.8 Correctness proof

The chosen x is nonnegative and leaves at most capacity k for every remaining slot, so the suffix remains feasible. Any smaller x leaves more than that total capacity and cannot be completed. Thus each position is the smallest possible value among all feasible prefixes; by induction the full sequence is lexicographically smallest. $\square$

66.9 Complexity and tests

Time is $O(n)$, memory $O(1)$ besides output. Use `long long` for nk . Test $s=0$, $s=nk$, $n=1$, $k=0$, and an impossible total. The random generator includes a 200000-slot scale case.

66.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int t;cin>>t;while(t--){long long n,k,
```

66.11 Full Python solution

```

import sys
data=list(map(int,sys.stdin.buffer.read().split()));i=1;out=[]
for _ in range(data[0]):
    n,k,s=data[i:i+3];i+=3
    if s>n*k:
        out.append('-1');continue
    a=[]
    for p in range(n):
        x=max(0,s-(n-p-1)*k);a.append(x);s-=x
    out.append(' '.join(map(str,a)))
print('\n'.join(out))

```

67 Stress testing constructive greedy

Canonical constructive outputs are stress-testable.

For bracket completion:

```
Enumerate every assignment of '?' for  $n \leq 10$ .  
Keep valid strings.  
Compare the greedy output to the smallest valid string.
```

For bounded sums:

```
Enumerate every sequence in  $[1, m]^n$  for small  $n$  and  $m$ .  
Filter by sum.  
Compare to the lexicographically smallest feasible sequence.
```

For non-decreasing repairs, random tests can compare against the direct forced cost formula. A slower dynamic program is possible for tiny bounded values, but the local lower-bound proof is already very direct.

68 What to carry forward

Constructive greedy is about keeping the future alive.

When solving harder constructive problems, ask:

- What must be true about every prefix?
- What resource or quota remains?
- What are the minimum and maximum possibilities for the suffix?
- If I choose the smallest current value, can I still finish?
- If not, what is the smallest value that makes finishing possible?

Those questions appear repeatedly in Codeforces constructive tasks, AtCoder greedy feasibility tasks, and ICPC-style implementation problems.

68.1 11. 20C – Dijkstra? (1900): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/20/C> Local package: [problems/11_dijkstra_path](#)

68.2 Restatement

Given closed integer intervals with $1 \leq r$, print the largest number covering one integer time and the smallest time with that coverage.

68.3 Constraints and traps

The total interval count is 200000, so scanning every integer coordinate is impossible. Endpoints may be negative. Because intervals are closed, an interval ending at x and one starting at x are both active at x ; remove an interval at $r+1$, not r .

68.4 Closed-interval event model

For a closed integer interval $[l, r]$, add $+1$ at l and -1 at $r+1$. After applying all changes at coordinate x , the running total is exactly the number of intervals containing x . This $r+1$ choice is the essential difference from half-open events: intervals ending and starting at the same coordinate are both active there.

For $[1, 3]$, $[3, 5]$, and $[3, 3]$, events yield active counts one at 1, then three at 3, so the answer is 3.

```

for each [l,r]: delta[l] += 1; delta[r+1] -= 1
active = 0; best = -1
for x in increasing event coordinates:
    active += delta[x]
    if active > best: best = active; answer_time = x

```

68.5 Correctness and complexity

The prefix sum of delta at x equals every interval whose start is at most x minus every interval whose end is before x, hence exactly the closed intervals covering x. Counts only change at event coordinates, so the earliest coordinate with a strict new maximum is the earliest busiest time. Sorted-map processing costs $O(n \log n)$ time and $O(n)$ memory. Use `long long` coordinates in C++; the local coordinate bound makes `r+1` safe. Test singleton intervals, touching closed intervals, equal maximum counts at different times, and negative times.

68.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int T;cin>>T;while(T--){int n;cin>>n;m

```

68.7 Full Python solution

```

import sys
data=list(map(int,sys.stdin.buffer.read().split())); i=1; out=[]
for _ in range(data[0]):
    d={}; n=data[i]; i+=1
    for _ in range(n):
        l,r=data[i],data[i+1]; i+=2; d[l]=d.get(l,0)+1; d[r+1]=d.get(r+1,0)-1
    cur=0; best=-1
    for x in sorted(d):
        cur+=d[x]
        if cur>best: best,when=cur,x
    out.append(f'{best} {when}')
print('\n'.join(out))

```

68.8 12. 161D – Distance in Tree (1800): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/161/D> Local package: [problems/12_distance_in_tree](#)

68.9 Restatement

Count the nonempty contiguous subarrays whose maximum value minus minimum value is at most k . Array values may be negative; only the spread is constrained.

This is the section's blind-practice exercise. It needs two monotonic deques at once: one to expose the current maximum and one to expose the current minimum.

68.10 Observation and trace

For a fixed right endpoint, if $[\text{left}, \text{right}]$ has acceptable spread, every shorter suffix ending at **right** is also acceptable: removing values cannot increase the maximum or decrease the minimum. Therefore after shrinking to a valid window, all $\text{right} - \text{left} + 1$ starts from **left** through **right** contribute.

Keep decreasing values in the maximum deque and increasing values in the minimum deque. Store indices, so expired positions can be removed when **left** moves.

For 1 3 2 5 and $k=2$:

right	value	valid window after shrinking	new subarrays	answer
1		[1]	1	1
3		[1,3]	2	3
2		[1,3,2]	3	6
5		[5]	1	7

At the last step the spread exceeds two until the left pointer has passed the earlier values.

68.11 Algorithm

```
left = 0
max_deque = empty decreasing deque of indices
min_deque = empty increasing deque of indices
answer = 0

for right from 0 to n-1:
    remove dominated back indices from max_deque
    append right to max_deque
    remove dominated back indices from min_deque
    append right to min_deque

    while a[max_deque.front] - a[min_deque.front] > k:
        remove left from either deque when it is at the front
        left += 1

    answer += right-left+1
```

68.12 Correctness proof

Lemma 1. After each update, the front of `max_deque` is an index of the maximum value in the current window, and the front of `min_deque` is an index of its minimum value.

Proof. When a new value arrives, every older maximum-deque back value no larger than it is dominated: the new index lasts longer and is at least as large. Removing those backs preserves all possible future maxima. The minimum argument is symmetric for values no smaller than the new value. Expired front indices are removed when the left endpoint passes them. Thus the surviving fronts are exactly the needed extrema. $\square$

Lemma 2. After the shrinking loop, `[left, right]` has spread at most `k`, and every subarray ending at `right` that starts before `left` is invalid.

Proof. Lemma 1 makes the loop condition exactly the current window's spread. The loop stops only when it is at most `k`. Each discarded left index belongs to a window whose spread was greater than `k`; extending that discarded window farther left cannot reduce its spread, so any earlier start is invalid. $\square$

Theorem. The algorithm returns the required number of subarrays.

Proof. By Lemma 2, exactly the `right-left+1` suffixes ending at each right endpoint are valid. The algorithm adds this number once per endpoint. Every nonempty subarray has one right endpoint, so all and only valid subarrays are counted once. $\square$

68.13 Complexity

Each index enters and leaves each deque at most once. Time is $O(n)$ per test case and memory is $O(n)$ in the worst case. The answer can be $n(n+1)/2$, so use `long long` in C++.

68.14 Implementation traps and tests

- Store indices, not just values, so you can remove expired endpoints.
- Use `<=` for the decreasing maximum deque and `>=` for the increasing minimum deque; retaining only the newest equal value is safe.
- Shrink before adding the count for the current right endpoint.
- Negative values are valid, so do not use a sum-based sliding window.

Test `k=0`, all equal values, alternating extreme values, a one-element array, and the all-equal 200000-element case that forces a 64-bit answer. The random generator uses direct quadratic enumeration on small arrays.

68.15 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int t;cin>>t;while(t--){int n;long lon
```


68.16 Full Python solution

```
from collections import deque
import sys
data = list(map(int, sys.stdin.buffer.read().split()))
index = 1
out = []
for _ in range(data[0]):
    n, k = data[index], data[index + 1]
    index += 2
    values = data[index:index + n]
    index += n
    maximum, minimum = deque(), deque()
    left = 0
    answer = 0
    for right, value in enumerate(values):
        while maximum and values[maximum[-1]] <= value: maximum.pop()
        maximum.append(right)
        while minimum and values[minimum[-1]] >= value: minimum.pop()
        minimum.append(right)
        while values[maximum[0]] - values[minimum[0]] > k:
            if maximum[0] == left: maximum.popleft()
            if minimum[0] == left: minimum.popleft()
            left += 1
        answer += right - left + 1
    out.append(str(answer))
print("\n".join(out))
```

69 Stress testing

For next greater:

```
For each i, scan j=i+1..n-1 and take the first greater value.
```

For sliding maximum:

```
For each window, compute max directly.
```

For shortest subarray:

```
Try every left endpoint and extend right while summing.
```

The local generators use these brute-force checks on small arrays.

70 What to carry forward

The important habit is to identify dominance.

Ask:

- What makes an older candidate useless?
- Should equality remove the older candidate?
- Does the structure need values, indices, or both?
- When does a candidate expire?
- Can each candidate be pushed and popped only once?

These questions are the bridge from simple stacks and queues to later graph BFS, DP optimization, and advanced monotonic data structures.

70.1 13. 580D – Kefa and Dishes (1800): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/580/D> Local package: [problems/13_kefa_and_dishes](#)

70.2 Restatement

Every item has an activation level and a possibly negative weight. For each query limit x , output the sum of weights of all items whose level is at most x , preserving the input query order.

70.3 Constraints and traps

Both n and q can be 200000, so scanning all items for every query is $O(nq)$ and impossible. Query limits arrive in arbitrary order, and equal item levels must all be included. Totals require `long long` in C++.

70.4 Observation and walk-through

Sort items by level and queries as $(\text{limit}, \text{original_id})$. Query limits then only increase, so an item that has become active never needs removal.

For items $(5,10), (1,-3), (3,7), (5,2)$, the sorted sweep answers limits 0,1,4,5 with 0,-3,4,16. At limit 4, exactly levels 1 and 3 have entered the running sum. Store each answer by its original id before printing.

70.5 Algorithm

```
sort items by level
sort (query_limit, original_id) pairs by query_limit
sum = 0, item = 0
for (limit, id) in sorted queries:
    while item exists and items[item].level <= limit:
        sum += items[item].weight
        item += 1
    answer[id] = sum
print answers in id order
```

70.6 Correctness proof

Before each sorted query limit x , the invariant is that `sum` contains exactly the weights of items with level at most the previous processed limit. The while-loop adds precisely the remaining items whose level is at most x ; because items are sorted, it adds no level above x . Thus `sum` is exactly the required total for x . Saving it at the original id restores the answer to its input position. Induction over sorted queries proves every printed total is correct.

70.7 Complexity

Sorting costs $O((n+q) \log(n+q))$; the two pointers move forward at most $n+q$ times. Memory is $O(n+q)$.

70.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<pair<long long, long long>> items(n);
    for (auto &item : items) cin >> item.first >> item.second;
    vector<pair<long long, int>> queries(q);
    for (int i = 0; i < q; ++i) cin >> queries[i].first, queries[i].second = i;
    sort(items.begin(), items.end());
    sort(queries.begin(), queries.end());
    vector<long long> answer(q);
    long long sum = 0;
    int item = 0;
    for (auto [limit, id] : queries) {
        while (item < n && items[item].first <= limit) sum += items[item++].second;
        answer[id] = sum;
    }
}
```

```

    for (int i = 0; i < q; ++i) cout << answer[i] << " \n"[i + 1 == q];
}

```

70.9 Full Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
n, q = data[0], data[1]
index = 2
items = sorted((data[i], data[i + 1]) for i in range(index, index + 2 * n, 2))
index += 2 * n
queries = sorted((data[index + i], i) for i in range(q))
answer = [0] * q
total = item = 0
for limit, query_id in queries:
    while item < n and items[item][0] <= limit:
        total += items[item][1]
        item += 1
    answer[query_id] = total
print(*answer)

```

70.10 Tests that matter

Test queries below every level, duplicate levels, negative weights, and limits that arrive in descending input order. The random oracle directly checks each query against every small item; the scale case validates the offline sweep.

70.11 14. 1245D – Shichikuji and Power Grid (1900): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1245/D> Local package: [problems/14_power_grid](#)

70.12 Restatement

Given closed busy intervals within $0..limit$ and a required integer duration d , find the smallest start $s \geq 0$ whose entire interval $[s, s+d-1]$ is not busy. Return -1 if no such start exists.

70.13 Constraints and traps

The total number of intervals is at most 200000, so testing every possible start is impossible. Intervals are closed: if one ends at r , the earliest time after it is $r+1$, not r . Input intervals may overlap, nest, or touch, and $1 \leq r$ is guaranteed. Use `long long` for $s+d-1$.

70.14 Observation and walk-through

Sort closed busy intervals and maintain `cur`, the earliest time not already blocked by processed intervals. If `cur+d-1 < l`, the gap before the next interval fits and `cur` is the answer. Otherwise merge the obstacle by setting `cur=max(cur,r+1)`. After all intervals, accept only when `cur+d-1<=limit`.

For busy intervals `[1,2]`, `[5,7]`, `[7,9]`, duration three, and limit twelve, start `cur=0`. The initial candidate `[0,2]` overlaps `[1,2]`, so move to 3. `[3,5]` overlaps `[5,7]`, so move to 8; `[8,10]` overlaps `[7,9]`, so move to 10. Finally `[10,12]` is free, so the answer is 10.

70.15 Algorithm

```
sort intervals by left endpoint
cur = 0
for [l, r] in sorted intervals:
    if cur + d - 1 < l: return cur
    cur = max(cur, r + 1)
return cur if cur + d - 1 <= limit else -1
```

70.16 Correctness proof

The invariant is that no start below `cur` can still begin a legal slot after the processed intervals. Sorting makes every future interval start no earlier than the current one, so a fitting gap can be returned immediately. Each interval either blocks the current candidate and advances `cur` past its right endpoint, or leaves a gap large enough to return the smallest possible start. After all intervals, the final limit test accepts exactly the remaining legal candidate. Thus the algorithm returns the earliest valid start or `-1`.

70.17 Complexity

Sorting costs $O(n \log n)$, the scan is linear, and memory is $O(n)$.

70.18 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests; cin >> tests;
    while (tests--) {
        int n; long long duration, limit;
        cin >> n >> duration >> limit;
        vector<pair<long long, long long>> busy(n);
        for (auto &interval : busy) cin >> interval.first >> interval.second;
        sort(busy.begin(), busy.end());
```

```

    long long cur = 0;
    for (auto [left, right] : busy) {
        if (cur + duration - 1 < left) break;
        cur = max(cur, right + 1);
    }
    cout << (cur + duration - 1 <= limit ? cur : -1) << '\n';
}
}

```

70.19 Full Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
index = 1
out = []
for _ in range(data[0]):
    n, duration, limit = data[index:index + 3]
    index += 3
    busy = sorted((data[i], data[i + 1]) for i in range(index, index + 2 * n, 2))
    index += 2 * n
    cur = 0
    for left, right in busy:
        if cur + duration - 1 < left:
            break
        cur = max(cur, right + 1)
    out.append(str(cur if cur + duration - 1 <= limit else -1))
print("\n".join(out))

```

70.20 Tests that matter

Test no intervals, touching intervals such as [2,2] and [3,3], nested intervals, and a valid slot ending exactly at limit. The small oracle scans integer starts; the scale case checks the sorted scan is necessary.

70.21 15. 271D – Good Substrings (1800): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/271/D> Local package: [problems/15_good_substrings](#)

70.22 Restatement

An undirected, unweighted graph contains several station vertices. For every vertex, print the minimum number of edges needed to reach any station. Print -1 when its connected component contains no station.

The graph can have up to 200000 vertices and edges, so running an ordinary BFS separately from every vertex, or separately from every station, can be too slow.

70.23 Key idea: start BFS from every station

Imagine adding one virtual source connected to every station by a zero-length edge. The desired answer is the shortest distance from that virtual source. We do not need to build the virtual source: put every station in the BFS queue initially and assign all of them distance 0.

Then run the usual BFS. When a vertex v is first discovered from u , write

```
dist[v] = dist[u] + 1
```

Vertices in components without stations are never put in the queue, so their initial value -1 remains the correct answer.

70.24 Walk-through

For the chain below, stations are vertices 1 and 5:

```
1 -- 2 -- 3 -- 4 -- 5
```

Initial queue and distances:

```
queue: [1, 5]
dist:  [0, -1, -1, -1, 0]
```

Processing 1 discovers 2; processing 5 discovers 4. Both receive distance 1. Next, either 2 or 4 discovers 3, which receives distance 2. The answer is therefore:

```
0 1 2 1 0
```

It does not matter which same-distance vertex is popped first. BFS always processes all vertices at distance d before any vertex at distance $d + 1$.

70.25 Algorithm

```
build an adjacency list for the undirected graph
set every dist value to -1

for every station s:
    if dist[s] is -1:          # harmless if a station is listed twice
        dist[s] = 0
        push s into the queue

while the queue is not empty:
    u = pop from the front
    for every neighbor v of u:
```



```

        if dist[v] is -1:
            dist[v] = dist[u] + 1
            push v into the queue

print dist[1], dist[2], ..., dist[n]

```

70.26 Correctness proof

We prove that the algorithm prints the required distance for every vertex.

Every station is inserted with distance 0, so its printed value is correct. Consider a non-station vertex v that is first discovered from u . BFS processes vertices in nondecreasing distance order. Thus u has the minimum possible distance to any station among vertices that could first reach v . Appending edge (u, v) gives a path of length $\text{dist}[u] + 1$ from a station to v , so the algorithm assigns a reachable distance to v .

Suppose a shorter station-to- v path of length L existed. Let w be the vertex directly before v on that path. Because $L < \text{dist}[v] = \text{dist}[u] + 1$, we have $L - 1 < \text{dist}[u]$. BFS reaches and processes w before u , then discovers v with a value at most L . This contradicts the assignment of the larger value $\text{dist}[u] + 1$. Therefore $\text{dist}[v]$ is the minimum distance to any station.

Finally, an unvisited vertex cannot be connected to a station: a path from a station would make BFS traverse its edges and discover it. Its value remains -1 , which is correct. Hence all printed values are correct.

70.27 Complexity

Each vertex enters the queue at most once and each undirected edge is examined twice. The time complexity is $O(n + m)$ and the adjacency list, distances, and queue use $O(n + m)$ memory.

70.28 Reference implementation in C++

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    cin >> n >> m >> k;

    vector<vector<int>>> graph(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
}

```

```

vector<int> dist(n + 1, -1);
queue<int> q;
for (int i = 0; i < k; ++i) {
    int station;
    cin >> station;
    if (dist[station] == -1) {
        dist[station] = 0;
        q.push(station);
    }
}

while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : graph[u]) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}

for (int v = 1; v <= n; ++v) {
    cout << dist[v] << (v == n ? '\\n' : ' ');
}
}

```

70.29 Reference implementation in Python

```

import sys
from collections import deque

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, k = data[:3]
    index = 3

    graph = [[] for _ in range(n + 1)]
    for _ in range(m):
        u, v = data[index], data[index + 1]
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    dist = [-1] * (n + 1)
    q = deque()
    for station in data[index:index + k]:
        if dist[station] == -1:
            dist[station] = 0

```

```

        q.append(station)

    while q:
        u = q.popleft()
        for v in graph[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)

    print(*dist[1:])

if __name__ == "__main__":
    main()

```

70.30 Tests worth checking

- One station in a disconnected component: vertices in other components must remain -1.
- Stations at both ends of a path: the middle vertex uses the closer end.
- Adjacent stations and duplicated station input: every station still has distance 0.
- A long chain with one station at an endpoint: checks both linear time and queue behavior.

70.31 16. 61E – Enemy Is Weak (1900): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/61/E> Local package: [problems/16_enemy_is_weak](#)

70.32 Restatement

The input is an undirected simple graph that can be disconnected. Classify every connected component as either a tree component (it has no cycle) or a cyclic component (it has at least one cycle), then print the two counts.

70.33 Constraints and traps

Both the number of vertices and roads can be 200000, so repeatedly scanning the graph for every component, or doing a separate cycle search after every traversal, risks unnecessary work. Isolated vertices count: a one-vertex, zero-edge component is a tree. Every undirected road appears in two adjacency lists, so summing degrees counts each road twice.

The graph is simple: there are no self-loops or repeated roads. That makes the standard connected-component characterization especially clean:

```
connected component is a tree <=> number of edges = number of vertices - 1
```

70.34 Observation and walk-through

Run one iterative DFS from each unvisited vertex. During that traversal, increment `vertices` whenever a vertex is popped, and add the length of its adjacency list to `degree_sum`. At the end, the component has `edges = degree_sum / 2` roads.

For the sample, DFS from 1 sees vertices 1, 2, 3. Its degree sum is $1 + 2 + 1 = 4$, so it has two edges and is a tree because $2 = 3 - 1$. DFS from 4 sees a triangle: three vertices and degree sum 6, hence three edges. Since $3 \neq 3 - 1$, that component is cyclic. Finally, 7--8 is a second tree component.

70.35 Algorithm

```
build an undirected adjacency list
seen[v] = false for every vertex
tree_components = cyclic_components = 0

for every vertex start:
    if start is seen:
        continue

    vertices = degree_sum = 0
    mark start and push it onto a stack
    while stack is not empty:
        u = pop stack
        vertices += 1
        degree_sum += number of neighbors of u
        for every neighbor v of u:
            if v is unseen:
                mark v
                push v

    edges = degree_sum / 2
    if edges == vertices - 1:
        tree_components += 1
    else:
        cyclic_components += 1

print tree_components and cyclic_components
```

Mark a vertex when it is pushed, not when it is popped. Otherwise two neighbors can push the same vertex before either copy is processed.

70.36 Correctness proof

Lemma 1. One DFS started at an unvisited vertex visits exactly its connected component, and no other component.

Proof. Every visited vertex is reached through a chain of graph edges from the start, so it belongs to the start's component. Conversely, DFS follows every edge incident to every vertex it visits. Any

vertex connected to the start has a path from it; proceeding along that path shows DFS eventually marks every vertex on the path, including the endpoint. $\square$

Lemma 2. After one DFS, `vertices` and `degree_sum / 2` are respectively the number of vertices and edges in that connected component.

Proof. By Lemma 1, each component vertex is popped exactly once and adds one to `vertices`. Every component road appears once in the adjacency list of each endpoint, so it contributes exactly two to `degree_sum`. $\square$

Lemma 3. A connected component is a tree exactly when its edge count is one less than its vertex count.

Proof. A tree with V vertices has $V-1$ edges: start with one vertex and each newly attached vertex contributes one edge. Conversely, a connected graph with $V-1$ edges cannot contain a cycle. Removing an edge from a cycle keeps the graph connected, which would leave a connected graph with only $V-2$ edges; a connected graph needs at least $V-1$ edges to reach all V vertices. Thus it is acyclic and therefore a tree. $\square$

Theorem. The algorithm prints the correct number of tree and cyclic components.

Proof. The outer loop starts exactly one DFS for each connected component. By Lemma 2 it computes that component's exact vertex and edge counts, and by Lemma 3 it classifies the component correctly. Adding one to the matching counter for every component gives exactly both requested totals. $\square$

70.37 Complexity

Every vertex is pushed once and every undirected road is scanned from each of its two endpoints.

```
Time:    O(n + m)
Memory:  O(n + m)
```

70.38 C++ implementation notes

Store `degree_sum` in `long long`. It fits in `int` under these bounds, but the type makes the relationship to an edge count explicit and remains safe if the exercise limits later grow. A `vector<char>` is enough for the `seen` state.

70.39 Python implementation notes

Use a normal list as a stack with `append` and `pop`; both are amortized constant time. Read all integers from `sys.stdin.buffer` to avoid repeated input overhead on the scale case.

70.40 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<int>> graph(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<char> seen(n, false);
    int tree_components = 0, cyclic_components = 0;
    for (int start = 0; start < n; ++start) {
        if (seen[start]) continue;

        long long vertices = 0, degree_sum = 0;
        vector<int> stack = {start};
        seen[start] = true;
        while (!stack.empty()) {
            int u = stack.back();
            stack.pop_back();
            ++vertices;
            degree_sum += graph[u].size();
            for (int v : graph[u]) {
                if (!seen[v]) {
                    seen[v] = true;
                    stack.push_back(v);
                }
            }
        }

        if (degree_sum / 2 == vertices - 1) ++tree_components;
        else ++cyclic_components;
    }

    cout << tree_components << ' ' << cyclic_components << '\n';
}

```

70.41 Full Python solution

```
import sys
```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[0], data[1]
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u = data[index] - 1
        v = data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    seen = [False] * n
    tree_components = cyclic_components = 0
    for start in range(n):
        if seen[start]:
            continue

        vertices = degree_sum = 0
        stack = [start]
        seen[start] = True
        while stack:
            u = stack.pop()
            vertices += 1
            degree_sum += len(graph[u])
            for v in graph[u]:
                if not seen[v]:
                    seen[v] = True
                    stack.append(v)

        if degree_sum // 2 == vertices - 1:
            tree_components += 1
        else:
            cyclic_components += 1

    print(tree_components, cyclic_components)

if __name__ == "__main__":
    main()

```

70.42 Common wrong answers

- Starting DFS only from vertex 1, which ignores other components.
- Calling every component with at least one edge cyclic; a path has edges but is still a tree.
- Comparing `degree_sum` directly with `vertices - 1` instead of dividing by two.
- Marking vertices only on pop, allowing duplicate stack entries and corrupting the collected statistics.

70.43 Tests that matter

- no roads: every isolated vertex is a tree component;
- one triangle plus a path: checks both categories in one graph;
- a cycle with a long tail: it remains one cyclic component;
- nested-looking traversal order with many cross edges: edge counting must not depend on DFS parent choices;
- a 200000-vertex cycle: rejects recursive implementations that exhaust the call stack and quadratic repeated traversals.

71 Stress testing

Good stress tests for this section compare independent ideas:

Problem A:

```
generate a random tree
compute subtree sizes with the iterative method
compare with a slow recursive method on small n
```

Problem B:

```
for each connected component, count vertices and edges
in a simple undirected graph, a connected component is cyclic iff edges >= vertices
```

Problem C:

```
compare DFS colors against Floyd-Warshall reachability on tiny directed graphs:
there is a cycle iff some edge u->v has v reachable back to u
```

Problem D:

```
brute-force every connected component with a tiny BFS
count its vertices and unique undirected edges
classify it using edges == vertices - 1
```

The local generators use direct reference implementations and include both fixed edge cases and randomized cases for C++ and Python.

72 What to carry forward

DFS is the tool to reach for when:

```
the graph is unweighted but shortest distance is not the point
the problem asks about descendants or subtrees
you need to know when a branch is finished
you need to detect cycles
```

The next section builds on DFS finishing order and introduces topological sorting and dynamic programming on DAGs.

72.1 17. 86D – Powerful Array (2200): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/86/D> Local package: [problems/17_powerful_array](#)

72.2 Restatement

Each directed edge $a \rightarrow b$ says that build step a must occur before step b . There are three possible outcomes:

- a directed cycle makes every complete build order impossible;
- an acyclic graph can have two or more valid orders, so the answer is **AMBIGUOUS**;
- otherwise there is exactly one valid order, which must be printed.

The graph has up to 200000 vertices and edges. Trying permutations is useful only for a tiny brute-force tester, not for the submitted solution.

72.3 Observation: available choices determine uniqueness

Run Kahn's algorithm and consider the set of currently available vertices: those with indegree 0 after already chosen vertices have been removed.

- If it is empty before all vertices are processed, a cycle remains.
- If it ever contains two or more vertices, either one may be chosen next. Those two choices begin two distinct valid topological orders.
- If it contains exactly one vertex at every step and all vertices are processed, every position is forced. The order is unique.

The queue does not need to be a min-heap. When the order is unique there is only one possible next vertex; when it is ambiguous we print no order. A plain FIFO queue is therefore sufficient.

Do not immediately print **AMBIGUOUS** after seeing two available vertices. A separate component may still contain a cycle. Continue Kahn's algorithm and let **IMPOSSIBLE** take priority if fewer than n vertices are processed.

72.4 Walk-through

First consider a forced chain:

```
1 -> 2 -> 3 -> 4
```

The available set has one member at every stage:

```
available: {1} -> {2} -> {3} -> {4}
order:      1      2      3      4
```

So the result is:

```
UNIQUE
1 2 3 4
```

Now add two independent starting steps:

```
1 -> 3 <- 2
```

Initially both 1 and 2 have indegree 0:

```
available: {1, 2}
```

Choosing 1 first gives 1, 2, 3; choosing 2 first gives 2, 1, 3. Thus the graph is **AMBIGUOUS**.

Finally, a graph can look ambiguous locally and still be impossible overall:

```
1 -> 3 <- 2      4 <-> 5
```

Steps 1 and 2 are initially available, but 4 and 5 can never be removed. The correct answer is **IMPOSSIBLE**, not **AMBIGUOUS**.

72.5 Algorithm

```
build adjacency lists and indegree values
put every vertex with indegree 0 into a queue
ambiguous = false
order = empty list

while queue is not empty:
    if queue has at least two vertices:
        ambiguous = true

    remove one vertex u from the queue
    append u to order
    for every edge u -> v:
        decrease indegree[v]
        if indegree[v] becomes 0:
            add v to the queue
```

```

if order has fewer than n vertices:
    print IMPOSSIBLE
else if ambiguous:
    print AMBIGUOUS
else:
    print UNIQUE and order

```

72.6 Correctness proof

We prove that the algorithm prints the required classification and, when applicable, the required order.

Lemma 1. If the algorithm processes fewer than n vertices, the graph contains a directed cycle.

Proof. Kahn's algorithm stops only when no unprocessed vertex has indegree 0 within the remaining graph. Starting from any remaining vertex and repeatedly following an incoming remaining edge must eventually repeat a vertex because the remaining graph is finite. The repeated portion is a directed cycle. $\square$

Lemma 2. If at some iteration the queue contains two different vertices, the graph has at least two valid topological orders.

Proof. Both vertices have no unprocessed prerequisites, so either can be placed next after the already constructed prefix. Removing either available vertex from a DAG leaves a DAG, which Kahn's algorithm can complete. One completion starts with the first chosen vertex and another starts with the second, so the two valid orders differ at this position. $\square$

Lemma 3. If the algorithm processes all vertices and the queue has exactly one vertex at every iteration, the recorded order is the only valid topological order.

Proof. Induct on the position in the order. Before the first position, the only indegree-zero vertex is the only vertex that can legally be first. After that forced vertex is removed, the same argument applies to the unique available vertex for the next position. Every position is forced, so no different valid order exists. $\square$

Theorem. The algorithm prints the correct output.

Proof. By Lemma 1, fewer than n processed vertices means **IMPOSSIBLE** is correct. Otherwise the graph is acyclic. If **ambiguous** is true, Lemma 2 gives two valid orders, so **AMBIGUOUS** is correct. If it is false, Lemma 3 proves that the recorded topological order is unique, so printing **UNIQUE** and that order is correct. $\square$

72.7 Complexity

Every vertex enters the queue once, and every edge is examined once:

```

Time:     $O(n + m)$ 
Memory:  $O(n + m)$ 

```

72.8 Implementation notes

- Check `queue.size()` or `len(queue)` before removing the next vertex.
- Keep processing after finding multiple available vertices; a cycle has higher-priority output.
- Record the order only so it can be printed in the unique case.
- With $n, m \leq 200000$, use adjacency lists and an iterative queue.

72.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> graph(n + 1);
    vector<int> indegree(n + 1, 0);
    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        graph[a].push_back(b);
        ++indegree[b];
    }

    queue<int> available;
    for (int step = 1; step <= n; ++step) {
        if (indegree[step] == 0) {
            available.push(step);
        }
    }

    bool ambiguous = false;
    vector<int> order;
    while (!available.empty()) {
        if (available.size() > 1) {
            ambiguous = true;
        }

        int u = available.front();
        available.pop();
        order.push_back(u);
        for (int v : graph[u]) {
            --indegree[v];
            if (indegree[v] == 0) {
                available.push(v);
            }
        }
    }
}
```

```

    }

    if (static_cast<int>(order.size()) != n) {
        cout << "IMPOSSIBLE\\n";
    } else if (ambiguous) {
        cout << "AMBIGUOUS\\n";
    } else {
        cout << "UNIQUE\\n";
        for (int i = 0; i < n; ++i) {
            cout << order[i] << (i + 1 == n ? '\\n' : ' ');
        }
    }
}
}

```

72.10 Full Python solution

```

import sys
from collections import deque

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n + 1)]
    indegree = [0] * (n + 1)
    index = 2
    for _ in range(m):
        a, b = data[index], data[index + 1]
        index += 2
        graph[a].append(b)
        indegree[b] += 1

    available = deque(step for step in range(1, n + 1) if indegree[step] == 0)
    ambiguous = False
    order = []
    while available:
        if len(available) > 1:
            ambiguous = True

        u = available.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                available.append(v)

    if len(order) != n:
        print("IMPOSSIBLE")
    elif ambiguous:
        print("AMBIGUOUS")
    else:

```

```
print("UNIQUE")
print(*order)

if __name__ == "__main__":
    main()
```

72.11 Common wrong answers

- Printing **AMBIGUOUS** as soon as two vertices are available, then missing a cycle in another component.
- Checking only the initial indegree-zero set. Choices can appear later.
- Using a min-heap because a previous task requested lexicographic order. This problem asks about uniqueness, not the smallest order.
- Treating a vertex with outgoing edges as unavailable. Only unmet incoming dependencies affect availability.

72.12 Tests that matter

- a one-vertex graph, which is uniquely ordered;
- a long chain, which is uniquely ordered;
- no edges on several vertices, which is ambiguous immediately;
- two independent starting vertices that later merge;
- a cycle with no initial available vertex;
- two available vertices plus a separate directed cycle, which must print **IMPOSSIBLE**.

73 Stress testing

For Problem D, keep random graphs very small and enumerate permutations of all vertices. Count valid orders only until the second one:

```
zero valid permutations -> IMPOSSIBLE  
one valid permutation  -> UNIQUE and that permutation  
two valid permutations -> AMBIGUOUS
```

This oracle is deliberately different from the submitted Kahn-based solution. It catches mistakes such as declaring ambiguity before ruling out a cycle. A separate large acyclic graph with two initial sources checks that the actual solution remains linear at 200000 vertices and edges.

74 What to carry forward

Topological sorting is more than a way to produce an order. The set of currently available vertices records what dependency choices remain:

```
no available vertex before completion -> cycle
one available vertex at every step      -> forced order
two available vertices at some step     -> at least two orders
```

Whenever a directed dependency graph is acyclic, process it in a topological order and ask what state can be propagated along each forward edge.

74.1 18. 498C – Array and Operations (2100): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/498/C> Local package: [problems/18_array_and_operations](#)

74.2 Restatement

We may deactivate cables in an undirected weighted graph. The remaining network must preserve the original connectivity relation: if two computers were connected by some path before deactivation, they must still have a path afterwards. Different original connected components do not need to become connected.

We want the greatest total cost of deactivated cables. Cable costs are nonnegative, and n , m can both reach 200000, so checking connectivity after each possible removal is far too slow.

74.3 Observation: keep a minimum spanning forest

Let **total** be the sum of all cable costs. Maximizing removed cost is the same as minimizing the cost of the cables we keep:

```
saved cost = total - kept cost
```

Consider one original connected component. The kept cables only need to keep that component connected. A minimum-cost connected subgraph can be reduced to a tree: if it contains a cycle, removing any edge on that cycle keeps it connected and never increases cost because all costs are nonnegative. Thus we need a minimum spanning tree for each original component.

Together, those trees form a **minimum spanning forest**. Kruskal's algorithm already builds exactly that forest if we simply process every edge and never declare a disconnected graph impossible.

74.4 Walk-through

Suppose the graph has two original components:

```
component 1:  1 --4-- 2 --1-- 3
               \      /
               ---3---

component 2:  4 --7-- 5
```

Process edges in increasing cost:

```
2-3, cost 1: keep
1-3, cost 3: keep
1-2, cost 4: skip; it would make a cycle
4-5, cost 7: keep
```

The kept forest costs $1 + 3 + 7 = 11$. All cable costs total 15, so the maximum saving is:

```
15 - 11 = 4
```

An isolated vertex needs no kept cable and contributes neither kept cost nor saved cost.

74.5 Algorithm

```
read all edges and add their weights to total
sort edges by increasing weight
make a DSU with one component per vertex
kept = 0

for every edge (u, v, w) in sorted order:
    if union(u, v) succeeds:
        kept += w

print total - kept
```

Unlike a connected-MST problem, do not check whether exactly $n - 1$ edges were kept. A disconnected input is valid: Kruskal returns one tree for each component.

74.6 Correctness proof

We prove that the algorithm outputs the maximum possible saved cost.

Lemma 1. In every original connected component, there is an optimal set of kept cables that is a spanning tree.

Proof. Any valid kept subgraph is connected within that component. If it contains a cycle, remove an edge from the cycle. Connectivity remains because the other cycle edges give an alternate route. Since

cable costs are nonnegative, removing the edge does not increase kept cost. Repeating this removes all cycles and leaves a connected acyclic subgraph, which is a spanning tree. $\square$

Lemma 2. Kruskal's selected edges have minimum possible total cost among all spanning forests that preserve the original components.

Proof. Restrict attention to one original component. Kruskal processes its edges in the same increasing-cost order as ordinary Kruskal on that component, and only joins different current DSU components. By the cut property, every such selected lightest connecting edge is safe for a minimum spanning tree of that component. Therefore its selected edges form a minimum spanning tree there. Applying this independently to every original component proves that the whole selected forest has minimum total cost. $\square$

Theorem. The algorithm prints the maximum total cost that can be saved.

Proof. By Lemma 1, an optimal valid retained network can be chosen as a spanning forest. By Lemma 2, the algorithm's forest has no greater kept cost than any such valid network. The input's total cost is fixed, so subtracting this minimum kept cost from it yields the greatest possible removed cost. Thus the printed value is optimal. $\square$

74.7 Complexity

Sorting dominates the work:

```
Time:    O(m log m)
Memory:  O(n + m)
```

DSU operations are effectively constant amortized time because of path compression and union by size.

74.8 Implementation notes

- Accumulate `total` and `kept` in `long long` in C++; their values can exceed a 32-bit integer.
- Repeated cables are harmless. At most one can be useful for a particular DSU merge; the rest are skipped once their endpoints are connected.
- Zero-cost cables can be selected normally. Keeping or removing an extra zero-cost cycle edge does not change the answer.
- Do not print `IMPOSSIBLE` for a disconnected graph. The requirement is to preserve each original component, not to connect all vertices together.

74.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;

    explicit DSU(int n) : parent(n + 1), size(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
};
```

```

}

int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}

bool unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (size[a] < size[b]) swap(a, b);
    parent[b] = a;
    size[a] += size[b];
    return true;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<tuple<long long, int, int>> edges;
    long long total = 0;
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        edges.emplace_back(w, u, v);
        total += w;
    }

    sort(edges.begin(), edges.end());
    DSU dsu(n);
    long long kept = 0;
    for (auto [w, u, v] : edges) {
        if (dsu.unite(u, v)) {
            kept += w;
        }
    }

    cout << total - kept << '\\n';
}

```

74.10 Full Python solution

```
import sys
```

```

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n + 1))
        self.size = [1] * (n + 1)

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> bool:
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = []
    total = 0
    index = 2
    for _ in range(m):
        u, v, w = data[index:index + 3]
        index += 3
        edges.append((w, u, v))
        total += w

    edges.sort()
    dsu = DSU(n)
    kept = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            kept += w

    print(total - kept)

if __name__ == "__main__":
    main()

```

74.11 Common wrong answers

- Running a connected-MST solution and printing IMPOSSIBLE for an input with multiple original components.

- Summing only skipped edges without accounting for every cable in `total`.
- Using `int` for the sum of up to 200000 costs of 10^9 .
- Sorting in decreasing order, which keeps an expensive forest and minimizes the saving instead.
- Removing a cable whenever its endpoints are currently connected without considering edge costs; the kept forest must be minimum cost.

74.12 Tests that matter

- a graph with no cables and several isolated vertices, which saves 0;
- one tree, where every cable is necessary and saves 0;
- repeated cables with different costs;
- zero-cost edges and cycles;
- multiple disconnected original components;
- a large graph whose total cost requires 64-bit arithmetic.

75 Stress testing

For tiny graphs, enumerate every subset of cables to keep. For each subset, run ordinary BFS from every vertex before and after removals and keep only subsets with exactly the same reachability relation. The best value of

```
total cost - retained cost
```

is an independent oracle for Kruskal's answer. The local generator uses at most ten edges for this exhaustive check and separately creates a 200000-edge case to exercise sorting and DSU at the intended scale.

76 What to carry forward

Kruskal is useful even when the goal is not phrased as “find an MST.” When a problem asks to preserve connectivity while discarding cost, translate it to:

```
minimum cost needed to keep = minimum spanning forest
maximum removable cost      = total cost - forest cost
```

For disconnected inputs, think component by component rather than forcing a single spanning tree.

76.1 19. 319C – Kalila and Dimna in the Logging Industry (2100): local companion

External Codeforces task: <https://codeforces.com/problemset/problem/319/C> Local package: [problems/19_kalila_and_dimna](#)

76.2 Restatement

Find the least-cost directed route from vertex 1 to vertex n , subject to one extra condition: the route must use an even number of roads. Roads have nonnegative costs, vertices and roads may be revisited, and there can be no valid even-hop route even when an ordinary route exists.

76.3 Constraints and traps

n and m can both be 200000, so enumerating routes or storing a distance for every possible number of hops is impossible. The parity of the hop count, not the exact count, is all that matters. A one-road route to the destination is invalid, while the zero-road route from 1 to itself is valid and costs zero.

Do not keep just one best distance per vertex. A cheaper arrival with odd parity might be unusable, while a more expensive even-parity arrival is the one that can answer the problem.

76.4 Observation and walk-through

For every original vertex u , make two states:

```
(u, 0): u has been reached using an even number of roads
(u, 1): u has been reached using an odd number of roads
```

Every road flips the parity. Thus a directed road $u \rightarrow v$ of cost w becomes two state transitions:


```
(u, 0) -> (v, 1), cost w
(u, 1) -> (v, 0), cost w
```

Start in $(1, 0)$ and seek $(n, 0)$. This is an ordinary nonnegative shortest-path problem on $2n$ states.

In the sample, the tempting route $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ costs 12, but has three roads and therefore ends in state $(5, 1)$. The route $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ costs $3 + 1 + 1 + 1 = 6$ and uses four roads, so it reaches $(5, 0)$ and is valid.

76.5 Algorithm

```
dist[u][parity] = infinity for all vertices and both parities
dist[1][0] = 0
push (0, 1, 0) into a min-heap

while heap is not empty:
    pop (cost, u, parity)
    if cost is stale: continue
    for every road u -> v with weight w:
        next_parity = parity xor 1
        relax dist[v][next_parity] with cost + w

print dist[n][0], or -1 if it stayed infinite
```

76.6 Correctness proof

Lemma 1. Every route from 1 to u using parity p corresponds to a path from state $(1, 0)$ to state (u, p) of the same cost, and vice versa.

Proof. Starting at parity zero, each original road toggles parity exactly once. The expanded graph has precisely the matching parity-toggling transition with the same weight for every original road. Applying this correspondence road by road maps an original route to a state path of equal cost. Reversing the mapping reconstructs an original route from any state path. $\square$

Lemma 2. Dijkstra computes the minimum cost for every reachable parity state.

Proof. By construction every expanded transition has the original road's nonnegative cost. Dijkstra's usual smallest-non-stale-pop invariant therefore applies to the expanded graph, so each finalized state distance is shortest. $\square$

Theorem. The algorithm prints the minimum cost of a valid even-hop route, or -1 exactly when none exists.

Proof. By Lemma 1, valid routes are exactly paths from $(1, 0)$ to $(n, 0)$, with equal costs. Lemma 2 finds the cheapest such state path. If $(n, 0)$ is unreachable, Lemma 1 says no valid route exists, so -1 is correct. $\square$

76.7 Complexity

The expanded graph has $2n$ states and $2m$ transitions.

Time: $O((n + m) \log n)$

Memory: $O(n + m)$

76.8 C++ implementation notes

Use `array<long long, 2>` for the two state distances per vertex. The bitwise expression `parity ^ 1` toggles 0 to 1 and 1 to 0. Use `long long` for all path costs because a shortest route can exceed 32-bit range.

76.9 Python implementation notes

Store distances as `dist[vertex][parity]`, and push `(cost, vertex, parity)` tuples into `heapq`. Python integers do not overflow, but the same stale-entry check is still essential for both correctness and running time.

76.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, long long>>> graph(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        graph[u - 1].push_back({v - 1, w});
    }

    const long long INF = (1LL << 62);
    vector<array<long long, 2>> dist(n, {INF, INF});
    using State = tuple<long long, int, int>;
    priority_queue<State, vector<State>, greater<State>> heap;
    dist[0][0] = 0;
    heap.push({0, 0, 0});

    while (!heap.empty()) {
        auto [cost, u, parity] = heap.top();
        heap.pop();
```

```

        if (cost != dist[u][parity]) continue;
        for (auto [v, weight] : graph[u]) {
            int next_parity = parity ^ 1;
            long long next_cost = cost + weight;
            if (next_cost < dist[v][next_parity]) {
                dist[v][next_parity] = next_cost;
                heap.push({next_cost, v, next_parity});
            }
        }
    }

    cout << (dist[n - 1][0] == INF ? -1 : dist[n - 1][0]) << '\n';
}

```

76.11 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u, v, weight = data[index:index + 3]
        index += 3
        graph[u - 1].append((v - 1, weight))

    inf = 10**30
    dist = [[inf, inf] for _ in range(n)]
    dist[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        cost, u, parity = heapq.heappop(heap)
        if cost != dist[u][parity]:
            continue
        for v, weight in graph[u]:
            next_parity = parity ^ 1
            next_cost = cost + weight
            if next_cost < dist[v][next_parity]:
                dist[v][next_parity] = next_cost
                heapq.heappush(heap, (next_cost, v, next_parity))

    print(-1 if dist[n - 1][0] == inf else dist[n - 1][0])

if __name__ == "__main__":
    main()

```

76.12 Common wrong answers

- Running ordinary Dijkstra with one distance per vertex, which loses parity.
- Printing the cheaper of `dist[n][0]` and `dist[n][1]`; only even parity is valid.
- Starting at (1, 1) instead of (1, 0).
- Counting total edge weight parity instead of the number of roads.
- Marking a state final when it is pushed into the heap rather than skipping stale popped entries.

76.13 Tests that matter

- $n = 1$ with no roads, whose answer is zero;
- a one-road path to the target, which is unreachable with even parity;
- an odd-hop cheap route and an even-hop more expensive route;
- zero-cost cycles that change parity without changing the cost;
- parallel roads of different costs;
- a long graph with 200000 roads, to test state count and heap behavior.

77 Common wrong answers

- Using BFS because the graph is “just shortest path”.
- Marking a vertex visited when it is pushed into the heap.
- Forgetting to skip stale heap entries.
- Using 32-bit `int` for distances.
- Applying Dijkstra to negative edges.
- In Discount Route, allowing the discount more than once.

78 Stress testing

For small graphs, compare Dijkstra to Floyd-Warshall. For grids, compare to Floyd-Warshall on the explicit small cell graph or to a slow repeated relaxation. For Discount Route, build the two-layer graph explicitly and run a generic Dijkstra to compare against the optimized state implementation.

For Even-Hop Route, build the $2n$ parity-state graph explicitly on tiny inputs and run Floyd-Warshall over those states. This independent oracle catches the two common modeling errors: starting in odd parity and accepting the odd-parity destination state. A separate long path with one parity-changing shortcut checks the 200000-edge scale.

79 What to carry forward

Dijkstra is the default shortest-path algorithm for nonnegative weighted graphs. The modeling skill is deciding what a state must remember. Once the state graph has nonnegative edge weights, the same invariant and heap template apply.

79.1 20. 786B – Legacy: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/786/B> Local package: [problems/20_legacy](#)

79.2 Restatement

The grid has ordinary open cells, walls, a start, a goal, and lowercase-letter portal cells. Walking to an open four-neighbor cell costs 1. From a letter cell, jumping to any cell with that same letter costs 0. We need the minimum cost from S to G, or -1 when no route exists.

The grid can contain 10^6 cells. It is not enough to use ordinary BFS, because a zero-cost teleport must be processed before a one-cost walk. It is also not enough to scan every matching-letter list every time a portal cell is visited: a grid full of the same letter would become quadratic.

79.3 Model the implicit graph

Use one graph vertex per non-wall grid cell.

```
walk to a four-neighbor open cell    edge cost 1
same-letter teleport                  edge cost 0
```

This is exactly a 0-1 shortest-path problem. Store the positions of each of the 26 letters while reading the grid, then use a deque:

```
cost 0 -> push_front
cost 1 -> push_back
```

79.4 Expanding one portal group once

Suppose the deque first removes a cell labelled **a** at distance **d**. Every other **a** cell is reachable from it for cost 0, so all of them can receive distance **d** immediately. There is no reason to scan the complete **a** list again when another **a** cell is reached later. Mark the letter as expanded after this first scan.

The word *first* matters. In 0-1 BFS, a cell removed from the front has the smallest currently possible distance. Therefore the first removed **a** cell has the minimum distance at which any **a** portal can be reached.

79.5 Walk-through

Consider:

```
S # a # G
. a . a .
. . . . .
```

Walking from S to the left a costs 2:

```
S -> (1,0) -> left a
```

When that a is removed from the deque, expand the a group:

```
left a -- cost 0 --> right a
```

The right a also has distance 2. Two ordinary moves reach G, so the answer is 4. The longer all-walking detour is never preferable.

79.6 Algorithm

```
record positions for each letter while reading the grid
dist[cell] = infinity for every cell
dist[S] = 0
push S into a deque
expanded[letter] = false

while deque is not empty:
    remove cell u from the front

    if u has letter x and expanded[x] is false:
        expanded[x] = true
        for every cell v with letter x:
            relax v using cost 0

    for every open four-neighbor v of u:
        relax v using cost 1

print dist[G], or -1 if it is still infinity
```

To relax an edge of cost w , update only if:

```
dist[u] + w < dist[v]
```

Then push v to the front for $w = 0$, otherwise to the back.

79.7 Correctness proof

We prove that the algorithm prints the minimum possible cost.

Lemma 1. When a portal letter x is expanded, the current distance of the removed x cell is the minimum distance of any route from S to an x cell.

Proof. 0-1 BFS removes cells from the deque in nondecreasing finalized distance order. The expanded cell is the first removed cell labelled x . If a cheaper route to any x cell existed, that cell would have a smaller distance and would be removed first, a contradiction. $\square$

Lemma 2. Expanding each portal letter at most once preserves every shortest-path distance.

Proof. When x is expanded at distance d , every x cell is relaxed to distance at most d by a zero-cost teleport. By Lemma 1, no x cell can later be reached with a value smaller than d . Thus scanning the same zero-cost teleport edges later cannot improve any distance. $\square$

Lemma 3. Apart from skipped redundant portal scans, the algorithm performs the same successful relaxations as 0-1 BFS on the implicit graph.

Proof. Every walking edge is examined whenever its source cell is removed. For a portal edge, its whole same-letter group is examined at the first and minimum-distance source by Lemma 1; Lemma 2 proves later scans are redundant. Each examined edge is relaxed with its stated cost and sent to the correct end of the deque. $\square$

Theorem. The algorithm prints the minimum travel cost from S to G , or -1 exactly when the goal is unreachable.

Proof. By Lemma 3, the algorithm is 0-1 BFS on all useful implicit graph edges. The 0-1 BFS invariant therefore gives the shortest distance from S to every reachable cell, including G . If G remains at infinity, no sequence of valid walking or teleport edges reached it, so it is unreachable. The printed result is consequently correct. $\square$

79.8 Complexity

Each cell has at most four walking edges. Each portal position appears in one letter list, and that list is scanned once. Therefore:

```
Time:    O(n * m)
Memory:  O(n * m)
```

79.9 Implementation notes

- Store letters in 26 lists indexed by `letter - 'a'` or `ord(letter) - ord('a')`.
- Mark a letter expanded before scanning its list.
- A teleport is cost 0, so its improved endpoints go to the deque front.
- A normal grid step costs 1, so it goes to the deque back.
- Do not treat S and G as portal letters; only lowercase characters are portal labels.

79.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<string> grid(n);
    vector<vector<int>> portals(26);
    int start = -1, goal = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            int id = r * m + c;
            char cell = grid[r][c];
            if (cell == 'S') start = id;
            if (cell == 'G') goal = id;
            if ('a' <= cell && cell <= 'z') {
                portals[cell - 'a'].push_back(id);
            }
        }
    }

    const int INF = 1e9;
    vector<int> dist(n * m, INF);
    vector<char> expanded(26, false);
    deque<int> dq;
    dist[start] = 0;
    dq.push_back(start);
    int dr[4] = {1, -1, 0, 0};
    int dc[4] = {0, 0, 1, -1};

    while (!dq.empty()) {
        int id = dq.front();
        dq.pop_front();
        int r = id / m, c = id % m;
        int current = dist[id];

        char cell = grid[r][c];
        if ('a' <= cell && cell <= 'z' && !expanded[cell - 'a']) {
            expanded[cell - 'a'] = true;
            for (int other : portals[cell - 'a']) {
                if (current < dist[other]) {
                    dist[other] = current;
                    dq.push_front(other);
                }
            }
        }
    }
}

```

```

    for (int k = 0; k < 4; ++k) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m || grid[nr][nc] == '#') {
            continue;
        }
        int other = nr * m + nc;
        if (current + 1 < dist[other]) {
            dist[other] = current + 1;
            dq.push_back(other);
        }
    }
}

cout << (dist[goal] == INF ? -1 : dist[goal]) << '\\n';
}

```

79.11 Full Python solution

```

import sys
from collections import deque

def main() -> None:
    data = sys.stdin.buffer.read().split()
    n, m = map(int, data[:2])
    grid = [row.decode() for row in data[2:2 + n]]
    portals = [[] for _ in range(26)]
    start = goal = -1

    for r, row in enumerate(grid):
        for c, cell in enumerate(row):
            cell_id = r * m + c
            if cell == "S":
                start = cell_id
            elif cell == "G":
                goal = cell_id
            elif "a" <= cell <= "z":
                portals[ord(cell) - ord("a")].append(cell_id)

    inf = 10**9
    dist = [inf] * (n * m)
    expanded = [False] * 26
    dist[start] = 0
    dq = deque([start])

    while dq:
        cell_id = dq.popleft()
        r, c = divmod(cell_id, m)
        current = dist[cell_id]
        cell = grid[r][c]

```

```

    if "a" <= cell <= "z":
        letter = ord(cell) - ord("a")
        if not expanded[letter]:
            expanded[letter] = True
            for other in portals[letter]:
                if current < dist[other]:
                    dist[other] = current
                    dq.appendleft(other)

    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != "#":
            other = nr * m + nc
            if current + 1 < dist[other]:
                dist[other] = current + 1
                dq.append(other)

    print(-1 if dist[goal] == inf else dist[goal])

if __name__ == "__main__":
    main()

```

79.12 Common wrong answers

- Using ordinary BFS, which counts teleports as if they cost one move.
- Putting a zero-cost teleport at the deque back.
- Scanning all cells with letter **a** every time any **a** cell is visited.
- Marking a portal letter used when it is first discovered instead of when it is removed at its minimum distance.
- Allowing moves through walls or treating **S** and **G** as lowercase labels.

79.13 Tests that matter

- no portals, which reduces to an ordinary shortest grid path;
- a portal shortcut that is cheaper than the fewest-step walk;
- repeated cells of one letter, to catch repeated full-list scans;
- an unreachable goal;
- a portal first reached by a costly route but later reachable more cheaply;
- a large open grid, which checks linear memory and time.

80 Stress testing

On a tiny grid, build the implicit graph literally: from every letter cell, add a zero-cost edge to every equal-letter cell. Then run heap Dijkstra and compare its answer with 0-1 BFS. This is deliberately slower but independent of the portal-group optimization. The local generator uses that oracle on small random grids and also includes a 1000 $\times$ 1000 open grid with known answer 1998.

81 What to carry forward

0-1 BFS remains linear only when the transition model is also efficient. A large group of equivalent zero-cost moves should usually be activated once at the first finalized distance:

```
implicit zero-cost group + one-time expansion = linear total work
```

First identify the state and the costs, then account for how often every generated transition is scanned.

81.1 21. 191C – Fools and Roads: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/191/C> Local package: [problems/21_fools_and_roads](#)

81.2 Restatement

Every undirected road has a difficulty. A route is judged by its hardest road, not by the sum of its road difficulties. We need the smallest possible value of that maximum on a route from city 1 to city n , or -1 if no route exists. When $n = 1$, the empty route has difficulty 0.

This wording is a trap: it is not an ordinary shortest path. Adding a road of difficulty 4 and one of difficulty 5 gives route difficulty 5, not 9.

81.3 Threshold viewpoint

Fix a candidate limit X . Keep only roads with difficulty at most X .

```
There is a route of difficulty at most X  
if and only if  
1 and n are connected using only roads of difficulty at most X.
```

As X increases, roads only get added, so connectivity can only change from false to true. Sort all roads by difficulty and add them in that order with a DSU. The first difficulty at which cities 1 and n share a component is the answer.

This is related to Kruskal, but we do not need to finish an MST. Stop as soon as the two required cities become connected.

81.4 Walk-through

For these roads:

1-3: 3	1-2: 4	3-4: 5	4-5: 5	2-5: 8
--------	--------	--------	--------	--------

Process increasing difficulties:

```
add 1-3 (3): components of 1 and 5 differ
add 1-2 (4): components of 1 and 5 differ
add 3-4 (5): components of 1 and 5 differ
add 4-5 (5): 1 and 5 are now connected
```

The answer is 5. The connecting route is 1 -> 3 -> 4 -> 5, whose maximum edge difficulty is 5.

81.5 Algorithm

```
if n is 1:
    print 0
    stop

sort all roads by difficulty
create DSU with one component per city

for each road (u, v, w) in sorted order:
    union(u, v)
    if find(1) equals find(n):
        print w
        stop

print -1
```

81.6 Correctness proof

Lemma 1. After all roads of difficulty at most X have been processed, the DSU puts two cities in the same component exactly when they are connected using only roads of difficulty at most X .

Proof. DSU starts with isolated cities and merges exactly the endpoints of every processed road. Thus every DSU merge represents an allowed road path, and every allowed road is included in the merges. Its components are exactly the connected components of the threshold graph. $\square$

Lemma 2. When the algorithm first finds cities 1 and n connected at road difficulty w , there exists a route from 1 to n whose maximum road difficulty is at most w .

Proof. All roads processed so far have difficulty at most w . By Lemma 1, DSU connectivity gives a path using only those roads. $\square$

Lemma 3. No route from 1 to n has maximum road difficulty less than the first reported value w .

Proof. Before processing roads of difficulty w , the two cities were in different DSU components. By Lemma 1, they were disconnected in the graph of all roads with difficulty below w . Such a route therefore cannot exist. $\square$

Theorem. The algorithm prints the required minimum route difficulty, or -1 exactly when city n is unreachable from city 1 .

Proof. Lemmas 2 and 3 show that a reported w is feasible and minimal. If the scan ends without connecting the cities, Lemma 1 says no input road path connects them, so -1 is correct. The $n = 1$ case is correct by the empty route convention. $\square$

81.7 Complexity

```
Time:    O(m log m)
Memory:  O(n + m)
```

Sorting roads dominates; DSU operations are effectively constant amortized time.

81.8 Implementation notes

- Use `long long` for road difficulties in C++ even though DSU stores city ids.
- Repeated roads are harmless; DSU ignores a road within an existing component.
- Do not add difficulties along a path.
- Do not return the first road incident to city n ; check DSU connectivity.

81.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;
    explicit DSU(int n) : parent(n + 1), size(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
```



```

cin >> n >> m;
vector<tuple<long long, int, int>> roads;
for (int i = 0; i < m; ++i) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    roads.emplace_back(w, u, v);
}
if (n == 1) {
    cout << 0 << '\\n';
    return 0;
}
sort(roads.begin(), roads.end());
DSU dsu(n);
for (auto [w, u, v] : roads) {
    dsu.unite(u, v);
    if (dsu.find(1) == dsu.find(n)) {
        cout << w << '\\n';
        return 0;
    }
}
cout << -1 << '\\n';
}

```

81.10 Full Python solution

```

import sys

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n + 1))
        self.size = [1] * (n + 1)

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    roads = sorted(
        (data[i + 2], data[i], data[i + 1])
        for i in range(2, 2 + 3 * m, 3)
    )
    if n == 1:
        print(0)
        return

    dsu = DSU(n)
    for w, u, v in roads:
        dsu.union(u, v)
        if dsu.find(1) == dsu.find(n):
            print(w)
            return
    print(-1)

if __name__ == "__main__":
    main()

```

81.11 Common wrong answers

- Summing road difficulties and using ordinary shortest-path Dijkstra.
- Building a full MST even after cities 1 and n are connected; correct but unnecessary.
- Declaring -1 merely because a low-difficulty prefix is disconnected.
- Forgetting the one-city empty-route case.

81.12 Tests that matter

- city 1 and city n disconnected;
- one city and no roads;
- a low-sum route with one very difficult road versus a longer route with a lower maximum;
- repeated roads with different difficulties;
- many equal-difficulty roads that connect the endpoints together.

82 Stress testing

For small graphs, use a different formulation: minimax Dijkstra. Let `dist[v]` be the smallest possible maximum edge on a route to `v`, and relax an edge of difficulty `w` with:

```
candidate = max(dist[u], w)
```

Use a heap on that candidate and compare its answer with the DSU threshold sweep. The local generator uses this independent oracle for small random graphs and a direct-answer 200000-edge scale case.

83 What to carry forward

When a route is scored by its worst edge, convert a minimization question into a connectivity threshold:

```
"Can I travel using only edges <= X?"
```

Sorting thresholds and DSU are often simpler than forcing an additive shortest path interpretation.

83.1 22. 208E – Blood Cousins: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/208/E> Local package: [problems/22_blood_cousins](#)

83.2 Restatement

You have unlimited copies of each positive coin denomination. Make exactly the target amount with the fewest coins, or print -1 if no selection of coins has that exact total. The target can be zero, in which case the empty selection is valid and uses zero coins.

83.3 Constraints and traps

The target can reach 200000, so enumerating every sequence of coin choices is exponential. Coins can be reused, and their listed order must not affect the answer. A greedy choice of the largest coin is not reliable: with coins 4 and 3, target 6 is possible using two 3 coins even though choosing a 4 first leaves an impossible remainder.

Some amounts may be unreachable. They must not be treated as if they required zero coins, because that would let later transitions build fake payments from them.

83.4 Observation and walk-through

Let $dp[x]$ mean the minimum number of coins needed to make exactly amount x ; use infinity when x is impossible. Consider the final coin of a nonempty payment for x . If that coin has value c , the previous coins must make exactly $x-c$. Therefore:

```
dp[x] = min(dp[x-c] + 1 for every denomination c <= x)
```

For target 15 and coins 4, 7, 9:

```

dp[0] = 0
dp[4] = dp[0] + 1 = 1
dp[7] = dp[0] + 1 = 1
dp[8] = dp[4] + 1 = 2
dp[11] = dp[7] + 1 = 2
dp[15] = min(dp[11] + 1, dp[8] + 1, dp[6] + 1)
        = min(3, 3, infinity) = 3

```

The two optimal final-coin choices give $4 + 4 + 7$ and $7 + 4 + 4$ as the same multiset; the DP only needs the optimal count, not the chosen order.

83.5 Algorithm

```

dp[0] = 0
dp[x] = infinity for every x from 1 through target

for x from 1 through target:
    for every coin c:
        if c <= x and dp[x-c] is finite:
            dp[x] = min(dp[x], dp[x-c] + 1)

print dp[target] if finite, otherwise -1

```

Amounts are computed from small to large, so $\text{dp}[x-c]$ is already complete when it is needed. This is also why coins may be used repeatedly: a value such as $\text{dp}[8]$ can use the already-computed $\text{dp}[4]$ and another coin of value 4.

83.6 Correctness proof

Lemma 1. Every nonempty exact payment for amount x has a final coin c such that its remaining coins form an exact payment for $x-c$.

Proof. Remove one coin from the payment and call its denomination c . The values of all remaining coins sum to $x-c$, so they form the claimed payment. $\square$

Lemma 2. After the algorithm computes $\text{dp}[x]$, it equals the minimum number of coins in an exact payment for x , or infinity if no such payment exists.

Proof. Induct on x . The base $x=0$ is correct because the empty payment uses zero coins. For $x>0$, Lemma 1 partitions every nonempty payment by its final coin c . By the induction hypothesis, $\text{dp}[x-c]$ is the minimum number of coins for the remaining amount, so $\text{dp}[x-c]+1$ is the best payment ending with c . The algorithm takes the minimum over every legal final denomination. If none has a finite predecessor, no payment exists; otherwise the minimum is exactly optimal. $\square$

Theorem. The algorithm prints the minimum possible number of coins for the target, or -1 exactly when exact payment is impossible.

Proof. Lemma 2 applies to **target**. The final conversion from infinity to -1 distinguishes the impossible case; otherwise the stored value is the minimum coin count. $\square$

83.7 Complexity

There are `target + 1` states, each considering at most `k` denominations.

```
Time:    O(target * k)
Memory:  O(target)
```

With `target <= 200000` and `k <= 20`, this fits comfortably.

83.8 C++ implementation notes

Set `INF = target + 1`. No valid exact payment needs more than `target` coins: every coin is positive, so this sentinel is safely larger than every answer. Only add one to a finite predecessor, avoiding accidental use of the sentinel.

83.9 Python implementation notes

The nested loop performs at most four million iterations at the largest input. Keep `coins` as a small list and use a simple integer list for `dp`; no recursion or special library is needed.

83.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int target, k;
    cin >> target >> k;
    vector<int> coins(k);
    for (int &coin : coins) cin >> coin;

    const int INF = target + 1;
    vector<int> dp(target + 1, INF);
    dp[0] = 0;
    for (int amount = 1; amount <= target; ++amount) {
        for (int coin : coins) {
            if (coin <= amount && dp[amount - coin] != INF) {
                dp[amount] = min(dp[amount], dp[amount - coin] + 1);
            }
        }
    }

    cout << (dp[target] == INF ? -1 : dp[target]) << '\n';
}
```

83.11 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    target, k = data[:2]
    coins = data[2:2 + k]

    inf = target + 1
    dp = [inf] * (target + 1)
    dp[0] = 0
    for amount in range(1, target + 1):
        for coin in coins:
            if coin <= amount and dp[amount - coin] != inf:
                dp[amount] = min(dp[amount], dp[amount - coin] + 1)

    print(-1 if dp[target] == inf else dp[target])

if __name__ == "__main__":
    main()
```

83.12 Common wrong answers

- Greedily taking the largest denomination first.
- Setting every `dp` entry to zero, which invents payments for unreachable amounts.
- Solving at most one copy of each denomination instead of allowing reuse.
- Returning the best amount below the target rather than requiring exact payment.
- Forgetting that target zero has answer zero.

83.13 Tests that matter

- target zero;
- a target smaller than every denomination;
- a target whose gcd condition makes it impossible, such as target 5 with coins 4 and 6;
- a case where greedy fails, such as target 6 with coins 4 and 3;
- repeated denominations;
- target 200000 with a denomination of 1, to check the largest DP table.

84 Testing and stress strategy

The section checker runs:

```
CP_TARGET=solution python3 sections/22_one_dimensional_dp/check.py
```

The random tests cover:

Broken Stairs:

random broken sets, no broken steps, and blocked targets.

Training Score:

positive, negative, and mixed scores.

Energy Route:

varied k, single-platform cases, and random heights.

For your own stress tests:

A: compare DP against brute-force recursive enumeration for $n \leq 20$.

B: enumerate all subsets for $n \leq 20$ and reject adjacent selections.

C: run Dijkstra on the DAG for $n \leq 50$ and compare with the DP.

D: run BFS over amounts on targets at most 80. Every use of one coin is one unweighted edge, so BFS independently finds the smallest coin count.

The brute force should be used only as a testing oracle on tiny inputs, not as the submitted solution.

85 What to carry forward

One-dimensional DP is the first reusable DP pattern:

```
last move -> transition
last choice -> take/skip
small prefix -> bigger prefix
```

The next sections will add more state dimensions and more complex computation orders. The discipline from this section remains the same: define the state before writing the recurrence.

85.1 23. 375D – Tree and Queries: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/375/D> Local package: [problems/23_tree_and_queries](#)

85.2 Restatement and reduction

Let the total of all values be T . If one group receives a subset with sum s , the other receives $T - s$, so the split difference is:

$$|s - (T - s)| = |T - 2s|$$

Thus we need a reachable subset sum as close as possible to $T / 2$. The total is at most 100000, so use the section's 0/1 subset-sum DP rather than testing all 2^n partitions.

For 1, 6, 11, 5, $T = 23$. Choosing subset {11} gives sums 11 and 12, so the answer is 1.

85.3 Constraints and traps

All values are positive integers and their total T is at most 100000. That total, rather than the number of input values alone, is the capacity dimension: an $O(nT)$ boolean table is practical, while enumerating 2^n assignments is not.

```
Every occurrence is a separate item, including equal values.
Each occurrence may be assigned to one group exactly once.
There is no requirement that the two sums are equal.
Only a sum  $s \leq T/2$  needs final inspection: its complementary sum is  $T-s$ .
```

Scanning above $T/2$ is redundant. If $s > T/2$, the other group has sum $T-s < T/2$ and represents the same unordered split with the same difference.

85.4 Reachability trace

For values 1, 6, 11, 5, begin with only sum zero reachable:

```
after no value: {0}
after 1:        {0, 1}
after 6:        {0, 1, 6, 7}
after 11:       {0, 1, 6, 7, 11, 12, 17, 18}
after 5:        ... including 11
```

The target half is $23/2 = 11.5$, so the best reachable sum at or below it is 11. The resulting difference is $23 - 2*11 = 1$.

When processing 6, update the boolean array from high sums down to low sums. For instance, `possible[7]` may use the old `possible[1]`, but that old value must not already include the current 6. An upward loop would reach `possible[12]` through the newly set `possible[6]`, incorrectly using 6 twice.

85.5 Algorithm

```
possible[0] = true
for each value x:
    for s from T down to x:
        possible[s] |= possible[s-x]

answer = minimum T - 2*s over reachable s with s <= T/2
```

The loop descends so `possible[s-x]` is still the state before the current value; each value is used at most once.

85.6 Correctness proof

Lemma 1. After any prefix of values, `possible[s]` is true exactly when a subset of that prefix sums to `s`.

Proof. Initially only the empty subset makes 0. For new value `x`, a subset making `s` either omits `x`, or uses it with an earlier subset making `s-x`. Descending updates implement exactly those two cases without reusing `x`. $\square$

Lemma 2. Every split is represented by a reachable sum `s` $\leq T/2$, and its difference is $T - 2s$.

Proof. Choose the smaller of the two group sums as `s`. The other is $T-s$, so its difference is $(T-s)-s$. $\square$

Theorem. The algorithm prints the minimum possible difference.

Proof. Lemma 1 gives exactly every available group sum. Lemma 2 shows the final scan evaluates every possible split's difference, so its minimum is optimal. $\square$

85.7 Complexity

Time: $O(n * T)$

Memory: $O(T)$

where T is the total input sum.

85.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    int total = 0;
    for (int& x : a) {
        cin >> x;
        total += x;
    }
    vector<char> possible(total + 1);
    possible[0] = true;
    for (int x : a) {
        for (int s = total; s >= x; --s) {
            possible[s] = possible[s] || possible[s - x];
        }
    }
    int answer = total;
    for (int s = 0; s <= total / 2; ++s) {
        if (possible[s]) answer = min(answer, total - 2 * s);
    }
    cout << answer << '\\\\n';
}
```

85.9 C++ implementation notes

`vector<char>` is a compact boolean table and avoids the unusual proxy behavior of `vector<bool>`. The table has indices 0 through `total`, inclusive. The descending loop condition is `s >= x`, so `s-x` is always a valid index.

85.10 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    values = data[1:]
    total = sum(values)
    possible = bytearray(total + 1)
    possible[0] = 1
    for value in values:
        for current in range(total, value - 1, -1):
            if possible[current - value]:
                possible[current] = 1
    print(min(total - 2 * s for s in range(total // 2 + 1) if possible[s]))

if __name__ == "__main__":
    main()

```

85.11 Python implementation notes

Use `bytearray(total + 1)` for a compact mutable reachability table; `possible[0]` must start true. In Python, `range(total, value - 1, -1)` is the descending 0/1 update. Only sums through $T/2$ need inspection after the DP, although updating through T keeps the transition simple.

85.12 Common wrong answers

- An upward loop accidentally permits a value multiple times.
- Requiring an exactly half-total subset instead of accepting the closest one.
- Deduplicating equal input values even though they are separate items.

85.13 Tests that matter

- one value;
- an odd total;
- an exactly balanced input;
- repeated values;
- total sum 100000.

86 Stress testing

For at most 18 values, enumerate every bitmask and minimize $\text{abs}(T - 2*s)$. This independent brute-force oracle checks the DP on random small cases. The local generator separately uses one hundred values of 1000 for the 100000-sum scale case.

87 What to carry forward

Partition questions often become subset-sum DP after expressing both group sums in terms of one selected subset. Derive the algebra first, then choose the capacity state and its loop direction.

87.1 24. 600E – Lomsat gelral: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/600/E> Local package: [problems/24_lomsat_gelral](#)

87.2 Restatement

Choose two right/down routes from the top-left to the bottom-right of a value grid. Their score is the sum of the values in the union of their visited cells: when both routes stand on one cell, its value is counted once. Maximize the score. Values may be negative, so a route cannot simply skip an inconvenient cell.

87.3 Constraints and traps

$n, m \leq 70$ makes an exponential search over two routes impossible. A route has $n+m-2$ moves, so even one route has a binomial number of possibilities.

Use 64-bit integers: as many as $2 * (n+m-1)$ cells can contribute.
Both couriers must make exactly $n+m-2$ moves.
The start and finish cells are shared and each contributes once.
Do not independently optimize the two routes: their overlap changes the score.

87.4 Key observation and trace

After `step` moves, every right/down route is on a cell with:

```
row + column = step
```

Therefore, if we know courier one's row `r1` and courier two's row `r2`, their columns are forced:

```
c1 = step - r1  
c2 = step - r2
```

For example, after three moves, rows 1 and 2 mean positions (1,2) and (2,1). The state needs no separate columns. On the next move each courier chooses **right** or **down**, giving four transition pairs. Add both destination values, except add only one when the two destinations are identical.

87.5 State and algorithm

Let:

```
dp[r1][r2] = best score after `step` moves,  
             with courier one at row r1 and courier two at row r2.
```

Initialize `dp[0][0]` with the start-cell value. For every step, create a new table initialized to negative infinity. For each reachable pair of rows, try the four pairs (right/right, right/down, down/right, down/down). Reject a move that leaves the grid. At `step = n+m-2`, both rows must be `n-1`.

```
dp[0][0] = value[0][0]  
  
for step = 0 .. n+m-3:  
    next = all negative infinity  
    for every reachable r1, r2:  
        derive c1 = step-r1 and c2 = step-r2  
        for each of the four next-move pairs:  
            derive nr1,nc1,nr2,nc2  
            if both destinations are in the grid:  
                gain = value[nr1][nc1]  
                if destinations differ: gain += value[nr2][nc2]  
                relax next[nr1][nr2]  
  
    dp = next  
  
answer = dp[n-1][n-1]
```

The two layers are sufficient because every transition advances exactly one step. This is also why negative infinity is necessary: zero is not a valid default when all available partial scores can be negative.

87.6 Correctness proof

Lemma 1. At a fixed `step`, rows `r1,r2` uniquely determine both courier positions whenever the derived columns are in range.

Proof. Each move increases `row + column` by one, starting from zero. Thus after `step` moves every position satisfies `row + column = step`; rearranging gives `column = step - row`. $\square$

Lemma 2. For every reachable state after `step` moves, `dp[r1][r2]` equals the maximum union score among all pairs of routes with those two positions.

Proof. Induct on `step`. At step zero there is only the pair of routes at the start, and the initialization counts that cell once. For the induction step, every pair of routes ending in a given next state has one of the four pairs of final moves. Removing those moves yields exactly a predecessor state considered by the algorithm. By induction its stored score is optimal; the transition adds precisely the new destination cells, once if equal. Taking the maximum over all four predecessor choices therefore stores the optimal score. $\square$

Theorem. The algorithm prints the maximum score obtainable by two legal courier routes.

Proof. After `n+m-2` moves, any legal courier is at the bottom-right cell, so all legal route pairs are represented by state `(n-1,n-1)`. Lemma 2 says that state stores the best score among exactly those pairs. $\square$

87.7 Complexity

There are $n+m-1$ layers and at most n^2 row pairs per layer. Each uses four constant-time transitions.

Time: $O((n+m) n^2)$

Memory: $O(n^2)$

87.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<long long>> a(n, vector<long long>(m));
    for (auto &row : a) for (long long &x : row) cin >> x;

    const long long NEG = -(1LL << 62);
    vector<vector<long long>> dp(n, vector<long long>(n, NEG));
    dp[0][0] = a[0][0];
    for (int step = 0; step < n + m - 2; ++step) {
        vector<vector<long long>> nxt(n, vector<long long>(n, NEG));
        for (int r1 = 0; r1 < n; ++r1) for (int r2 = 0; r2 < n; ++r2) {
            int c1 = step - r1, c2 = step - r2;
            if (c1 < 0 || c1 >= m || c2 < 0 || c2 >= m || dp[r1][r2] == NEG) continue;
            for (int d1 = 0; d1 <= 1; ++d1) for (int d2 = 0; d2 <= 1; ++d2) {
                int nr1 = r1 + d1, nc1 = c1 + 1 - d1;
                int nr2 = r2 + d2, nc2 = c2 + 1 - d2;
                if (nr1 >= n || nr2 >= n || nc1 >= m || nc2 >= m) continue;
                long long gain = a[nr1][nc1];
                if (nr1 != nr2 || nc1 != nc2) gain += a[nr2][nc2];
                nxt[nr1][nr2] = max(nxt[nr1][nr2], dp[r1][r2] + gain);
            }
        }
        dp.swap(nxt);
    }
    cout << dp[n - 1][n - 1] << '\n';
}
```

87.9 Full Python solution

```
import sys

def main() -> None:
```



```

data = list(map(int, sys.stdin.buffer.read().split()))
if not data:
    return
n, m = data[:2]
a = [data[2 + r * m:2 + (r + 1) * m] for r in range(n)]
neg = -(1 << 62)
dp = [[neg] * n for _ in range(n)]
dp[0][0] = a[0][0]
for step in range(n + m - 2):
    nxt = [[neg] * n for _ in range(n)]
    for r1 in range(n):
        c1 = step - r1
        if not 0 <= c1 < m:
            continue
        for r2 in range(n):
            c2 = step - r2
            if not 0 <= c2 < m or dp[r1][r2] == neg:
                continue
            for d1 in (0, 1):
                for d2 in (0, 1):
                    nr1, nc1 = r1 + d1, c1 + 1 - d1
                    nr2, nc2 = r2 + d2, c2 + 1 - d2
                    if nr1 >= n or nr2 >= n or nc1 >= m or nc2 >= m:
                        continue
                    gain = a[nr1][nc1]
                    if (nr1, nc1) != (nr2, nc2):
                        gain += a[nr2][nc2]
                    nxt[nr1][nr2] = max(nxt[nr1][nr2], dp[r1][r2] + gain)
    dp = nxt
print(dp[n - 1][n - 1])

if __name__ == "__main__":
    main()

```

87.10 Common wrong answers and tests

Adding a meeting cell twice.

Using 0 for an unreachable state on a negative grid.

Forgetting that the 1x1 grid has only one counted cell.

Trying to store both columns even though they are determined by the step.

Test a 1x1 negative grid, a single row or column where both routes must coincide, a grid where separating routes is valuable, and an all-negative grid. The random checker uses exhaustive enumeration of every route pair on grids up to 5x5, then adds a 70x70 scale case.

87.11 Stress testing

For tiny $n, m \leq 5$, generate every right/down route by choosing the positions of its down moves. Try every pair of generated routes, form the union of their visited cells, and take the best value. This

is deliberately a different algorithm from the synchronized DP, so agreement is meaningful. Include negative values in the tiny grids; they expose implementations that initialize unreachable states to zero. The separate 70x70 all-zero case checks that the intended polynomial state space is used without requiring an expensive oracle.

88 What to carry forward

Grid DP is not restricted to one path and one cell state. Find a quantity that advances in lockstep, then store only the independent coordinates. Here the step fixes both columns, reducing a four-coordinate description to two rows. The same synchronization idea reappears in problems with two walkers, aligned strings, and simultaneous operations.

88.1 25. 877E – Danil and a Part-time Job: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/877/E> Local package: [problems/25_danil_part_time_job](#)

88.2 Restatement

Two players alternately take either the leftmost or rightmost remaining value from an array. Both play to maximize their own final total. Print the first player's final total minus the second player's final total, which can be negative.

88.3 Constraints and traps

The array can contain negative values and is long enough that trying every sequence of endpoint choices is infeasible. A game of n turns has up to 2^n possible move sequences, while interval DP considers only $O(n^2)$ subarrays.

The requested quantity is a score difference, not the first player's sum.
Both players are optimal; taking the larger endpoint greedily is not reliable.
The player to move changes after every pick.
Use signed 64-bit totals because values and their sum may be large.

88.4 State and transition

Store the best score difference for the player whose turn it is:

```
dp[l][r] = current player's final sum - opponent's final sum on a[l..r]
```

Taking left gives $a[l] - dp[l+1][r]$: after that move, the opponent becomes the player to move on the remaining interval. Taking right similarly gives $a[r] - dp[l][r-1]$.

```
dp[i][i] = a[i]  
dp[l][r] = max(a[l] - dp[l+1][r], a[r] - dp[l][r-1])
```

For 1, 5, 233, 7, optimal play gives difference 222. Negative values need no special case: a one-element interval with -9 has value -9 .

88.5 Small trace

For $a = [4, 1, 7]$, the length-one states are their values:

```
dp[0][0] = 4, dp[1][1] = 1, dp[2][2] = 7
```

For interval $[4,1]$, taking 4 leaves the opponent an advantage of 1, so the resulting advantage is $4-1 = 3$. Taking 1 leaves advantage 4, so it gives $1-4 = -3$. Thus $dp[0][1] = 3$. For $[1,7]$, the analogous value is 6. On $[4,1,7]$, the two choices give:

```
take 4: 4 - dp[1][2] = 4 - 6 = -2
take 7: 7 - dp[0][1] = 7 - 3 = 4
```

The current player takes 7 and can finish four points ahead.

88.6 Algorithm

Use one array. Iterate l from right to left and r from left to right:

```
dp[l] = a[l]
dp[r] = max(a[l] - old dp[r], a[r] - dp[r-1])
```

Before updating, $dp[r]$ is $dp[l+1][r]$; $dp[r-1]$ was already updated to $dp[l][r-1]$.

88.7 Correctness proof

Lemma. $dp[l][r]$ is the largest advantage the player to move can force.

Proof. A one-value interval is forced. On a longer interval the only legal first moves take left or right. The opponent's best advantage in the resulting smaller interval is subtracted, producing exactly the two transition values. Choosing their maximum is optimal. Induction on interval length proves the lemma. $\square$

Theorem. The algorithm prints the optimal first-player-minus-second-player score.

Proof. The first player is the player to move on $[0, n-1]$, so the lemma applies to $dp[0][n-1]$. $\square$

88.8 Complexity

```
Time:  $O(n^2)$ 
Memory:  $O(n)$ 
```

88.9 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> a(n), dp(n);
    for (long long &x : a) cin >> x;
    for (int l = n - 1; l >= 0; --l) {
        dp[l] = a[l];
        for (int r = l + 1; r < n; ++r)
            dp[r] = max(a[l] - dp[r], a[r] - dp[r - 1]);
    }
    cout << dp[n - 1] << '\\n';
}

```

88.10 C++ implementation notes

`dp[r]` initially represents the interval whose left endpoint was one greater than the current `l`; save nothing separately because the first expression reads it before overwriting it. Processing `r` increasingly makes `dp[r-1]` already represent the current interval's left endpoint.

88.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    values = data[1:]
    n = len(values)
    dp = [0] * n
    for left in range(n - 1, -1, -1):
        dp[left] = values[left]
        for right in range(left + 1, n):
            dp[right] = max(values[left] - dp[right], values[right] - dp[right - 1])
    print(dp[-1])

if __name__ == "__main__":
    main()

```

88.12 Python implementation notes

The same loop order is essential after compression: `left` decreases, then `right` increases. Python integers already handle large score differences, but the empty-input guard is unnecessary because the problem guarantees `n >= 1`.

88.13 Traps and tests

- Do not greedily take the larger endpoint; it may expose a better reply.
- Use `long long` for accumulated differences.
- Test one negative value, all equal values, and alternating signs.

89 Stress testing

For at most 16 values, recursively enumerate every left/right game choice without DP memoization. This independent game tree checks the recurrence on small random cases. The generator also has a direct-answer array of 3000 ones to exercise the quadratic bound.

90 What to carry forward

For two-player interval games, define the state from the current player's viewpoint. When turns swap, the opponent's future advantage is subtracted.

90.1 26. 242E – XOR on Segment: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/242/E> Local package: [problems/26_xor_on_segment](#)

90.2 Restatement

Every tree vertex has a positive activation cost. Activate some vertices so that every tree edge touches at least one activated endpoint, while minimizing the sum of activated costs. A one-vertex tree has no roads to cover, so its optimal cost is zero.

90.3 Constraints and traps

n can be 200000 and costs can reach 10^9 . Enumerating activated subsets is exponential, and a recursive traversal can fail on a path-shaped tree.

An activated vertex covers every incident edge, not only edges to children.
If a parent is inactive, every child must be active to cover the parent-child road.
If a parent is active, each child may independently be active or inactive.
The answer may leave the root inactive.

Use `long long` in C++ for the total cost.

90.4 State and walk-through

Root the tree at vertex 1. Define two subtree costs:

```
active[u]    = minimum cost in u's subtree when u is activated
inactive[u]  = minimum cost in u's subtree when u is not activated
```

If u is active, edge (u, v) is already covered for every child v , so that child can use its cheaper state.
If u is inactive, each child must be active to cover edge (u, v) :

```
active[u]    = cost[u] + sum(min(active[v], inactive[v]))
inactive[u]  =          sum(active[v])
```


For the sample, process leaves first:

```
vertex 2: active=2, inactive=0
vertex 4: active=1, inactive=0
vertex 5: active=3, inactive=0

vertex 3: active    = 4 + min(1,0) + min(3,0) = 4
          inactive  = 1 + 3 = 4

vertex 1: active    = 5 + min(2,0) + min(4,4) = 9
          inactive  = 2 + 4 = 6
```

The root has no parent edge that needs coverage, so either state is legal. The answer is $\min(9, 6) = 6$, obtained by activating vertices 2 and 3.

90.5 Algorithm

```
root the tree at vertex 1 and record an iterative traversal order
active[u] = cost[u], inactive[u] = 0 for every vertex

for u in reverse traversal order:
    for every child v of u:
        active[u] += min(active[v], inactive[v])
        inactive[u] += active[v]

print min(active[root], inactive[root])
```

Reverse traversal order is postorder: all child state pairs are ready before their parent uses them.

90.6 Correctness proof

Lemma 1. For every vertex u , $\text{active}[u]$ is the minimum cost of a valid activation choice inside u 's subtree under the condition that u is active.

Proof. Induct on subtree height. For a leaf, activating it costs exactly $\text{cost}[u]$. For an internal u , every edge from u to a child is covered by the active endpoint u . Thus each child subtree may choose either valid child state independently. By the induction hypothesis, the least contribution of child v is $\min(\text{active}[v], \text{inactive}[v])$; adding the mandatory $\text{cost}[u]$ gives exactly the transition. $\square$

Lemma 2. For every vertex u , $\text{inactive}[u]$ is the minimum cost of a valid activation choice inside u 's subtree under the condition that u is inactive.

Proof. A leaf needs no activated vertex in its subtree, so its cost is zero. For an internal inactive u , each edge (u,v) must be covered by its child endpoint, which forces every child v active. Child subtrees are independent, and by Lemma 1 their least valid contributions are $\text{active}[v]$. Their sum is precisely the transition for $\text{inactive}[u]$. $\square$

Theorem. The algorithm prints the minimum activation cost covering every road in the tree.

Proof. Lemmas 1 and 2 prove the meanings of both root states. The root has no parent edge, so it may be active or inactive. Taking the smaller of the two valid root costs is therefore the global optimum. $\square$

90.7 Complexity

Each vertex and edge is processed a constant number of times.

Time: $O(n)$

Memory: $O(n)$

90.8 C++ implementation notes

Build `parent` and `order` iteratively, then scan `order` backward. Initialize `active` with vertex costs and `inactive` with zero; this already gives the correct leaf base case. Use `long long` for both arrays.

90.9 Python implementation notes

The pattern for `u in order:` is safe even while appending newly discovered children to the list. Then `reversed(order)` provides postorder without recursion. Python integers safely hold every possible cost total.

90.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> cost(n);
    for (long long &value : cost) cin >> value;
    vector<vector<int>> graph(n);
    for (int i = 0; i + 1 < n; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> parent(n, -1), order = {0};
    parent[0] = 0;
    for (int index = 0; index < static_cast<int>(order.size()); ++index) {
        int u = order[index];
        for (int v : graph[u]) {
            if (parent[v] == -1) {
                parent[v] = u;
            }
        }
        order.push_back(v);
    }
}
```

```

        order.push_back(v);
    }
}

vector<long long> active = cost, inactive(n, 0);
for (int index = n - 1; index >= 0; --index) {
    int u = order[index];
    for (int v : graph[u]) {
        if (parent[v] == u) {
            active[u] += min(active[v], inactive[v]);
            inactive[u] += active[v];
        }
    }
}

cout << min(active[0], inactive[0]) << '\n';
}

```

90.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    cost = data[1:1 + n]
    graph = [[] for _ in range(n)]
    index = 1 + n
    for _ in range(n - 1):
        u = data[index] - 1
        v = data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    parent = [-1] * n
    parent[0] = 0
    order = [0]
    for u in order:
        for v in graph[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)

    active = cost[:]
    inactive = [0] * n
    for u in reversed(order):
        for v in graph[u]:
            if parent[v] == u:

```

```

        active[u] += min(active[v], inactive[v])
        inactive[u] += active[v]

    print(min(active[0], inactive[0]))

if __name__ == "__main__":
    main()

```

90.12 Common wrong answers

- Applying the independent-set recurrence, which reverses the required child state when the parent is active/inactive.
- Forcing the root active even though it has no parent road to cover.
- Adding `min(active[v], inactive[v])` to `inactive[u]`; an inactive parent forces an active child.
- Recursing into a long path and overflowing the call stack.
- Using a 32-bit integer for costs.

90.13 Tests that matter

- a one-vertex tree, whose answer is zero;
- a long path, which checks iterative traversal and alternating choices;
- a star with an expensive center, which can favor activating all leaves;
- a star with cheap center, which favors the center;
- equal costs, where several optimal guard sets exist;
- costs near 10^9 on a large tree, which require 64-bit totals.

91 Testing and carry-forward

The section checker runs:

```
CP_TARGET=solution python3 sections/26_tree_dp_basics/check.py
```

Tests include single-node trees, paths, stars, and random trees. For Road Guards, the local generator enumerates every activated subset on trees up to 18 vertices, then uses a 200000-vertex path as the scale case.

Carry forward:

```
Root the tree.
Define states around the node's relationship to its parent/children.
Process children before parents.
Combine child subtrees independently.
```

91.1 27. 103D – Time to Raid Cowavans: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/103/D> Local package: [problems/27_time_to RAID_cowavans](#)

91.2 Restatement

Choose a subset of at most twenty performers. A listed incompatible pair may not be chosen together. Scores can be negative and choosing nobody is allowed, so the goal is the greatest total score among all compatible subsets, possibly zero.

This is the local blind exercise for the section. Its statement does not tell you what information to store for a subset; this editorial develops that state after the attempt.

91.3 Constraints and traps

The central constraint is $n \leq 20$. There are $2^{20} = 1,048,576$ possible subsets: too many to inspect each pair from scratch for every subset, but small enough to keep one or two arrays indexed by a mask.

Scores have magnitude up to 10^9 . A valid lineup can contain twenty positive performers, so its total can reach $2 \cdot 10^{10}$. Use `long long` in C++; Python integers already grow as needed.

An incompatible pair is undirected. If (u, v) is read, then choosing u forbids v and choosing v forbids u .

Finally, negative scores matter. They do not make a compatible subset invalid, but the empty lineup is legal and has score zero. Initializing the answer to a negative infinity would incorrectly force a performer into an all-negative instance.

91.4 Observations

Let `mask` represent the exact set of selected performers. A direct test of a mask could scan every input edge and reject it if both endpoints have their bits set. Repeating that scan for all masks costs $O(m 2^n)$. It fits only narrowly and does unnecessary repeated work.

Instead, obtain each non-empty mask from one smaller mask. Pick one set bit, call its performer `p`, and remove it:

```
bit      = mask & -mask
p        = index of bit
previous = mask ^ bit
```

Now the lineup represented by `mask` is compatible exactly when both facts are true:

1. `previous` was compatible;
2. none of `p`'s incompatible neighbours is in `previous`.

The total score has the same one-item recurrence:

```
score[mask] = score[previous] + w[p]
```

For fast neighbour tests, make `forbidden[p]` a mask containing every performer incompatible with `p`. Then condition 2 is simply:

```
(forbidden[p] & previous) == 0
```

This is a useful bitmask-DP pattern: the mask itself describes the whole choice, while one removed bit supplies a unique smaller state from which to maintain a property incrementally.

91.5 Sample walk-through

Number performers from zero internally. For the sample, the scores are `[8, 6, -3, 7]`, and performer 1 conflicts with both 0 and 3.

Consider the mask 1001, which selects performers 0 and 3:

```
mask      = 1001
bit       = 0001          (performer 0)
previous  = 1000          (performer 3)
score     = score[1000] + 8 = 7 + 8 = 15
```

`previous` is compatible. Performer 0 is incompatible only with performer 1, and bit 1 is not set in 1000, so the new subset is compatible.

In contrast, for 0011 we remove performer 0 and get `previous = 0010`. That previous mask selects performer 1; the intersection of performer 0's forbidden mask with 0010 is non-zero. Therefore 0011 is rejected. The best accepted total is 15.

91.6 Algorithm

```

read scores and build forbidden[p] for every performer p

compatible[0] = true
total[0] = 0
answer = 0

for mask from 1 to (1 << n) - 1:
    bit = lowest set bit of mask
    p = index of bit
    previous = mask without bit

    total[mask] = total[previous] + score[p]
    compatible[mask] = compatible[previous] and
                      (forbidden[p] & previous) == 0
    if compatible[mask]:
        answer = max(answer, total[mask])

print answer

```

The recurrence always points to a numerically smaller mask, because removing a set bit lowers the integer. Therefore increasing mask order has already computed `previous`.

91.7 Correctness proof

We prove that the algorithm prints the maximum score of any compatible lineup.

Lemma 1. For every mask, `total[mask]` equals the sum of the scores of exactly the performers whose bits are set in `mask`.

Proof. The claim is true for mask 0, whose sum is zero. For a non-empty mask, the algorithm removes one selected performer `p` to form `previous` and adds `score[p]` to `total[previous]`. By induction on the number of set bits, `total[previous]` is the sum of every other selected performer, so the result is precisely the sum for `mask`. $\square$

Lemma 2. For every mask, `compatible[mask]` is true exactly when no incompatible pair has both endpoints selected by `mask`.

Proof. The empty mask is compatible, establishing the base case. Consider a non-empty `mask`, with removed performer `p` and `previous` as in the algorithm. By induction, `compatible[previous]` is true exactly when all pairs wholly inside `previous` are allowed. The only pairs newly relevant after adding `p` are pairs between `p` and a selected performer in `previous`. The bit intersection `forbidden[p] & previous` is zero exactly when no such pair exists. Thus the algorithm marks `mask` compatible exactly in the required case. $\square$

Theorem. The printed `answer` is the maximum total score of a compatible lineup.

Proof. Every possible lineup corresponds to exactly one mask, and every mask is processed once. By Lemma 2, the algorithm considers `total[mask]` for exactly the compatible lineups. By Lemma 1, that value is the lineup's actual total score. `answer` starts with the empty compatible lineup's score and takes the maximum over all other compatible masks, so it equals the required optimum. $\square$

91.8 Complexity

Time: $O(n + m + 2^n)$
Memory: $O(2^n + n)$

Building the incompatibility masks costs $O(m)$. Each subset then does a constant number of bit operations and array accesses.

91.9 C++ implementation notes

`__builtin_ctz(mask)` returns the number of trailing zero bits, which is the index of the least significant set bit when `mask` is non-zero. Call it only inside the non-empty-mask loop. `1 << n` is safe here because `n <= 20`, but the accumulated scores still need `long long`.

`vector<char>` is a compact, ordinary boolean-like array. It avoids the special proxy behavior of `vector<bool>` without changing the recurrence.

91.10 Python implementation notes

Python has no built-in trailing-zero operation, but `lowest_bit.bit_length() - 1` gives the index after computing `lowest_bit = mask & -mask`. A `bytearray` stores compatibility flags compactly; values 0 and 1 work directly in an `if` condition.

Use `not (forbidden[performer] & previous)` for the zero-intersection test. Parentheses keep the intended precedence clear.

91.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> score(n);
    for (long long& value : score) cin >> value;

    vector<int> forbidden(n, 0);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        forbidden[u] |= 1 << v;
    }
}
```



```

        forbidden[v] |= 1 << u;
    }

    int total_masks = 1 << n;
    vector<long long> total(total_masks, 0);
    vector<char> compatible(total_masks, false);
    compatible[0] = true;
    long long answer = 0;

    for (int mask = 1; mask < total_masks; ++mask) {
        int performer = __builtin_ctz(mask);
        int previous = mask ^ (1 << performer);
        total[mask] = total[previous] + score[performer];
        compatible[mask] = compatible[previous] &&
            ((forbidden[performer] & previous) == 0);
        if (compatible[mask]) answer = max(answer, total[mask]);
    }

    cout << answer << '\n';
    return 0;
}

```

91.12 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    it = iter(data)
    n = next(it)
    m = next(it)
    score = [next(it) for _ in range(n)]
    forbidden = [0] * n
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        forbidden[u] |= 1 << v
        forbidden[v] |= 1 << u

    total_masks = 1 << n
    total = [0] * total_masks
    compatible = bytearray(total_masks)
    compatible[0] = 1
    answer = 0

    for mask in range(1, total_masks):
        lowest_bit = mask & -mask

```

```

performer = lowest_bit.bit_length() - 1
previous = mask ^ lowest_bit
total[mask] = total[previous] + score[performer]
if compatible[previous] and not (forbidden[performer] & previous):
    compatible[mask] = 1
    if total[mask] > answer:
        answer = total[mask]

print(answer)

if __name__ == "__main__":
    main()

```

91.13 Common wrong answers

Checking only whether the last performer has a conflict and forgetting whether the previous subset was already invalid.
 Treating an incompatible pair as directed and setting only one forbidden bit.
 Using int for totals, which overflows on twenty scores of 10^9 .
 Starting answer at the smallest score instead of allowing the empty lineup.
 Calling `__builtin_ctz(0)`, which is undefined in C++.

91.14 Tests that matter

All scores negative and no conflicts: answer must be 0.
 A complete incompatibility graph: at most one performer can be chosen.
 No conflicts and twenty scores of 10^9 : checks 64-bit accumulation and scale.
 A conflict graph with disconnected components: compatible choices may combine performers from different components.

92 Testing discussion

The random generators intentionally use small n so that the expected outputs can be checked by brute force for assignment, route, and lineup problems. This matters: bitmask DP bugs often pass samples while failing on a single missing bit test.

For your own stress tests:

1. Generate small random cases.
2. Solve them with brute force over subsets or permutations.
3. Compare against the DP.
4. Only then trust the optimized constraints.

93 What to carry forward

Bitmask DP is a compact way to remember choices over a small set. The three most reusable states are:

```
dp[mask]           coverage or used-set optimization
dp[mask] with popcount(mask) deciding the next row
dp[mask][last]     route/order problems where current position matters
```

Future advanced sections will add subset iteration tricks, meet-in-the-middle, and transforms. The foundation remains the same: define the mask meaning before writing the transition.

93.1 28. 1000F – One Occurrence: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1000/F> Local package: [problems/28_one_occurrence](#)

93.2 Restatement and reduction

Choose non-adjacent booths on a circle for maximum reward. On a line, let `best[i]` be the best reward through position `i`; then:

```
best[i] = max(best[i-1], best[i-2] + a[i])
```

The circle only adds one conflict: the first and last booths cannot both be chosen. Every valid choice either excludes the last booth or excludes the first booth. Solve those two lines and take the larger answer. For one booth, return `max(0, a[0])` directly.

For sample 2 7 9 3 1, excluding the last permits $2 + 9 = 11$; no valid circular selection is better.

93.3 Constraints and traps

`n` can be 200000, so subset enumeration and a two-dimensional DP are out of scope. Rewards may be negative and can be as large as 10^9 in magnitude.

```
Selecting no booth is allowed, so the answer is never negative.
Booth 1 conflicts with booth n as well as with booth 2.
n = 1 is not two ordinary line cases; handle it directly.
Use 64-bit totals in C++.
```

93.4 Worked trace

For the sample [2, 7, 9, 3, 1], first exclude the last booth and solve [2, 7, 9, 3]:

```
value:      2  7  9  3
best so far: 2  7 11 11
```

Now exclude the first booth and solve [7, 9, 3, 1]:

```
value:      7  9  3  1
best so far: 7  9 10 10
```

The larger case result is 11. For $n = 1$, say [-4], neither line case is used: the legal choices are take -4 or take nothing, giving 0.

93.5 Algorithm

```
line(l, r):
    previous_two = previous_one = 0
    for i from l to r:
        current = max(previous_one, previous_two + a[i])
        shift values forward
    return previous_one

if n == 1: answer = max(0, a[0])
else:     answer = max(line(0, n-2), line(1, n-1))
```

93.6 Correctness proof

Lemma 1. `line(l,r)` returns the maximum reward from non-adjacent booths in that line interval.

Proof. For each position, an optimal choice either skips it, giving the previous optimum, or takes it and must combine it with the optimum ending two positions earlier. The recurrence considers both cases. Induction over the line positions proves the result. $\square$

Lemma 2. Every valid circular selection belongs to at least one of the two line cases.

Proof. A valid selection cannot contain both first and last booths. Thus it omits the last (first line case), or omits the first (second line case), or both. $\square$

Theorem. The algorithm prints the maximum valid circular reward.

Proof. Lemma 2 places every valid selection in a solved line case. Lemma 1 optimizes each case, so their maximum is optimal. The single-booth branch has exactly the allowed choices: take it or take nothing. $\square$

93.7 Complexity

```
Time: O(n)
Memory: O(1)
```

93.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long line(const vector<long long>& a, int left, int right) {
    long long two_back = 0, one_back = 0;
    for (int i = left; i <= right; ++i) {
        long long current = max(one_back, two_back + a[i]);
        two_back = one_back;
        one_back = current;
    }
    return one_back;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> a(n);
    for (long long& x : a) cin >> x;
    if (n == 1) cout << max(0LL, a[0]) << "\\n";
    else cout << max(line(a, 0, n - 2), line(a, 1, n - 1)) << "\\n";
}
```

93.9 C++ implementation notes

Pass the original vector plus inclusive `left`, `right` indices to `line`; this does not copy either case. Initialize both rolling values to zero so skipping all negative booths stays legal.

93.10 Full Python solution

```
import sys

def line(values: list[int], left: int, right: int) -> int:
    two_back = one_back = 0
    for index in range(left, right + 1):
        two_back, one_back = one_back, max(one_back, two_back + values[index])
    return one_back

def main() -> None:
```

```

data = list(map(int, sys.stdin.buffer.read().split()))
values = data[1:]
if len(values) == 1:
    print(max(0, values[0]))
else:
    print(max(line(values, 0, len(values) - 2),
              line(values, 1, len(values) - 1)))

if __name__ == "__main__":
    main()

```

93.11 Python implementation notes

Avoid `values[:-1]` and `values[1:]`: each slice creates a new list. Passing the same list with endpoint indices preserves the advertised $O(1)$ auxiliary DP memory (besides the unavoidable input list).

93.12 Traps and tests

- Treating the circle as a line allows selecting both endpoints.
- Forgetting the one-booth special case makes its two line intervals empty.
- Negative rewards mean taking no booth can be best.
- Test one booth, all equal rewards, endpoint-heavy values, and all negatives.

94 Stress testing

For at most 18 booths, enumerate every subset and reject any subset containing adjacent circular positions. This independent oracle checks the two-case DP. The local generator also includes 200000 booths of reward one, whose direct answer is 100000.

95 What to carry forward

When a cycle blocks a simple path DP only at its endpoints, split on one endpoint's inclusion/exclusion and solve the remaining path cases.

95.1 29. 240F – TorCoder: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/240/F> Local package: [problems/29_torcoder](#)

95.2 Restatement and observation

We need $a^{(b^c) \bmod P} \bmod P$, where $P = 1000000007$ is prime and a is nonzero modulo P . Fermat's theorem gives:

$$a^{(P-1)} = 1 \bmod P$$

so only the exponent modulo $P-1$ matters. Compute the inner exponent with a second binary exponentiation:

```
e = b^c mod (P-1)
answer = a^e mod P
```

For example, $2^{(3^2)} = 2^9 = 512$.

95.3 Constraints and traps

There can be up to 200000 queries, while b and c may be enormous. The outer base is guaranteed not to be divisible by P ; that contract is vital.

```
Never form b^c as an ordinary integer.
Reducing the exponent modulo P-1 is valid here only because gcd(a, P) = 1.
The exponent residue may be zero; pow(a, 0, P) correctly returns 1.
Use 64-bit multiplication in C++: products stay below about 10^18.
```

If the base could be zero modulo P , the exponent reduction would need a separate zero/nonzero analysis. This problem deliberately avoids that case.

95.4 Worked trace

Consider $a=2$, $b=3$, $c=5$. First compute the inner exponent modulo $P-1 = 1000000006$ by binary exponentiation:

$$e = 3^5 \bmod (P-1) = 243$$

Fermat permits replacing the true exponent 3^5 with this residue, so the second binary power computes $2^{243} \bmod P$. The two powers use the same tool; only their moduli differ. For a huge c , the first call still examines only the bits of c .

95.5 Algorithm

```
for each query a,b,c:
    exponent = power(b, c, P-1)
    print power(a, exponent, P)
```

Both calls are standard binary exponentiation.

95.6 Correctness proof

Lemma. `power(x,y,m)` returns $x^y \bmod m$.

Proof. Binary exponentiation maintains `result * baseremaining` equal to the original power modulo m ; each bit either multiplies `result` by the current power of two or skips it. When `remaining` becomes zero, the invariant gives the result. $\square$

Theorem. The algorithm prints $a^{(b^c)} \bmod P$.

Proof. The first power call returns $e = b^c \bmod (P-1)$ by the lemma. Because a is nonzero modulo prime P , Fermat implies exponents differing by $P-1$ give the same residue, so $a^{(b^c)} = a^e \bmod P$. The second power call returns that value by the lemma. $\square$

95.7 Complexity

Each query uses two logarithmic powers:

```
Time:  $O(q (\log b + \log c))$ 
Memory:  $O(1)$  besides output
```

95.8 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long power(long long base, long long exponent, long long mod) {
    long long result = 1 % mod;
    for (base %= mod; exponent; exponent >>= 1, base = base * base % mod)
        if (exponent & 1) result = result * base % mod;
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q; cin >> q;
    while (q--) {
        long long a, b, c; cin >> a >> b >> c;
        cout << power(a, power(b, c, MOD - 1), MOD) << "\\n";
    }
}

```

95.9 C++ implementation notes

Use `long long` for the products in binary exponentiation: both operands are less than P , and their product fits in signed 64-bit range. Normalize an input base with `% MOD` if the statement later allows values outside the usual range.

95.10 Full Python solution

```

import sys
MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(data[0]):
        a, b, c = data[1 + 3 * i:4 + 3 * i]
        out.append(str(pow(a, pow(b, c, MOD - 1), MOD)))
    print("\\n".join(out))

if __name__ == "__main__":
    main()

```

95.11 Python implementation notes

Python's three-argument `pow(base, exponent, modulus)` is binary exponentiation implemented in C. It avoids constructing the enormous integer `b ** c`; use it for both nested powers.

95.12 Traps and tests

- Reducing the outer exponent by $P-1$ requires a nonzero modulo P .
- Do not compute b^c as an ordinary integer.
- Test $b=1$, huge c , and base $P-1$ with odd/even exponent.

96 Stress testing

For tiny bases and exponents, compute b^c as an ordinary integer and then multiply a that many times modulo P . This independent slow oracle validates the two modular power calls. The generator also emits 200000 queries with known answer 2 to exercise input and logarithmic query processing.

97 What to carry forward

Under a prime modulus, a nonzero base has exponent period dividing $P-1$. Nested powers are handled from inside out, reducing each exponent at the modulus required by the next power.

```
## 30. 343D -- Water Tree: local companion
```

```
External Codeforces task: <https://codeforces.com/problemset/problem/343/D>
```

```
Local package: [problems/30_water_tree](problems/30_water_tree/README.md)
```

```
## Restatement
```

For every query, find the least non-negative x satisfying:

```
```text
a*x == b (mod m).
```

The output is  $-1$  when no such residue exists. Unlike the inverse problem, the right-hand side is not necessarily one and  $a$  need not be coprime with  $m$ .

This is the blind exercise for the section. The statement gives only the congruence; it does not reveal the reduction needed to solve it.

### 97.1 Constraints and traps

With up to  $2 * 10^5$  queries and values up to  $10^9$ , trying candidates  $x = 0, 1, 2, \dots$  can take a billion checks for one query. The intended work is a single gcd computation per query.

The case  $\gcd(a, m) > 1$  is not automatically impossible. For example,  $6x == 8 \pmod{14}$  has the answer 6. The correct divisibility test involves the right-hand side too.

When a solution exists there are generally several integer solutions, but they are all equal modulo a smaller modulus. The problem asks for the representative in  $[0, m/g - 1]$ , where  $g = \gcd(a, m)$ , not merely any signed Bezout coefficient.

If  $m/g = 1$ , every integer is equivalent modulo one, and the requested least non-negative answer is 0. Both `% 1` in C++ and Python's `% 1` produce zero, so no separate special case is required.

### 97.2 Observations

The congruence means that  $m$  divides  $a*x-b$ . Equivalently, there is some integer  $y$  such that:

```
a*x + m*y = b.
```

This is a linear Diophantine equation. Let  $g = \gcd(a, m)$ . It has a solution exactly when  $g$  divides  $b$ .

When it does, divide every term by  $g$ :

$$(a/g)*x + (m/g)*y = b/g.$$

Now  $a/g$  and  $m/g$  are coprime. Extended Euclid on the original pair gives coefficients  $p, q$  with  $a*p + m*q = g$ . Multiplying by  $b/g$  gives one solution:

$$x_0 = p * (b/g).$$

Only the residue of  $x$  modulo  $m/g$  matters, so normalizing  $x_0$  by that modulus returns the smallest non-negative answer.

## 97.3 Sample walk-through

Consider  $6x \equiv 8 \pmod{14}$ .

```
gcd(6,14) = 2, and 2 divides 8, so a solution may exist.
6*5 + 14*(-2) = 2.
```

Scale this Bezout identity by  $8/2 = 4$ :

$$6*20 + 14*(-8) = 8.$$

Thus  $x = 20$  works. The reduced modulus is  $14/2 = 7$ , and  $20 \bmod 7 = 6$ . Indeed,  $6*6 = 36$ , which is  $8$  modulo  $14$ . The complete solution family is  $x = 6 + 7t$ , so  $6$  is the requested smallest non-negative value.

For  $6x \equiv 7 \pmod{14}$ , the gcd is still two, but two does not divide seven. The associated equation  $6x + 14y = 7$  therefore has no integer solution.

## 97.4 Algorithm

```
for each query (a, b, m):
 (g, p, q) = extended_gcd(a, m)
 if b mod g != 0:
 print -1
 continue

 reduced_modulus = m / g
 x0 = p * (b / g)
 answer = x0 mod reduced_modulus, normalized into [0, reduced_modulus)
 print answer
```

For the positive inputs in this task, the standard extended-gcd helper returns the positive  $g$ . In C++, normalize a potentially negative remainder with:

```
answer %= reduced_modulus;
if (answer < 0) answer += reduced_modulus;
```

Python's % already returns a non-negative remainder for a positive modulus.

## 97.5 Correctness proof

We prove that the algorithm prints the required answer for each query.

**Lemma 1.** The congruence  $a \cdot x \equiv b \pmod{m}$  has an integer solution if and only if  $\gcd(a, m)$  divides  $b$ .

**Proof.** The congruence is equivalent to the existence of  $y$  with  $a \cdot x + m \cdot y = b$ . Every left-hand side is divisible by  $\gcd(a, m)$ , proving necessity. Conversely, extended Euclid produces  $p, q$  such that  $a \cdot p + m \cdot q = g$ , where  $g = \gcd(a, m)$ . If  $g$  divides  $b$ , multiplying this identity by  $b/g$  produces an equation with right-hand side  $b$ , proving sufficiency.  $\square$

**Lemma 2.** If a solution exists,  $x_0 = p \cdot (b/g)$  produced by the algorithm is a solution, and every solution has the same residue modulo  $m/g$ .

**Proof.** The scaled Bezout identity in Lemma 1 gives  $a \cdot x_0 + m \cdot (q \cdot b/g) = b$ , so  $x_0$  satisfies the congruence. For any two solutions  $x_1, x_2$ , subtract their congruences to get  $m$  divides  $a \cdot (x_1 - x_2)$ . Dividing  $a$  and  $m$  by their gcd leaves coprime integers, so  $m/g$  divides  $x_1 - x_2$ . Thus all solutions share one residue modulo  $m/g$ .  $\square$

**Theorem.** For each query, the algorithm prints  $-1$  exactly when no solution exists; otherwise it prints the smallest non-negative solution.

**Proof.** By Lemma 1, the algorithm prints  $-1$  exactly in the impossible case. In the solvable case, Lemma 2 says  $x_0$  is a solution and all solutions are congruent modulo  $m/g$ . Normalizing  $x_0$  produces the unique representative of that residue in  $[0, m/g)$ , which is necessarily the smallest non-negative solution.  $\square$

## 97.6 Complexity

Each query runs extended Euclid once:

```
Time: O(log min(a, m)) per query
Memory: O(log min(a, m)) recursion stack, or O(1) with iterative Euclid
```

The aggregate time is  $O(q \log 10^9)$ , which is appropriate for the query limit.

## 97.7 C++ implementation notes

The scaled coefficient has magnitude below the signed 64-bit limit under the local bounds: both the Bezout coefficient and  $b/g$  are at most on the order of  $10^9$ . Keep all arithmetic in `long long`, never `int`.

Do not compute `a * answer` merely to verify the result in a 32-bit type. For a debug assertion with larger bounds, use `__int128` for that product.



## 97.8 Python implementation notes

Use an output list and one final `"\n".join(...)` for  $2 * 10^5$  answers. Python integers make the scale multiplication exact. Its positive-modulus remainder rule means `(coefficient * scale) % reduced_modulus` is already the required representative.

## 97.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long extended_gcd(long long a, long long b, long long& x, long long& y) {
 if (b == 0) {
 x = 1;
 y = 0;
 return a;
 }
 long long x1, y1;
 long long g = extended_gcd(b, a % b, x1, y1);
 x = y1;
 y = x1 - (a / b) * y1;
 return g;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 while (q--) {
 long long a, b, m;
 cin >> a >> b >> m;

 long long coefficient, unused;
 long long g = extended_gcd(a, m, coefficient, unused);
 if (b % g != 0) {
 cout << -1 << '\n';
 continue;
 }

 long long reduced_modulus = m / g;
 long long scale = b / g;
 long long answer = (coefficient * scale) % reduced_modulus;
 if (answer < 0) answer += reduced_modulus;
 cout << answer << '\n';
 }
 return 0;
}
```

## 97.10 Full Python solution

```
import sys

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
 if b == 0:
 return a, 1, 0
 g, x1, y1 = extended_gcd(b, a % b)
 return g, y1, x1 - (a // b) * y1

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 values = iter(data[1:])
 answer: list[str] = []
 for _ in range(q):
 a = next(values)
 b = next(values)
 m = next(values)
 g, coefficient, _ = extended_gcd(a, m)
 if b % g:
 answer.append("-1")
 continue
 reduced_modulus = m // g
 answer.append(str((coefficient * (b // g)) % reduced_modulus))
 sys.stdout.write("\n".join(answer) + "\n")

if __name__ == "__main__":
 main()
```

## 97.11 Common wrong answers

Requiring  $\text{gcd}(a, m) = 1$ , which rejects valid cases such as  $6x \equiv 8 \pmod{14}$ .  
Checking  $\text{gcd}(a, m)$  divides  $a$  rather than the right-hand side  $b$ .  
Using an inverse of  $a$  modulo  $m$  before dividing out the  $\text{gcd}$ .  
Printing a negative Bezout-derived  $x$  instead of normalizing by  $m/g$ .  
Normalizing by  $m$  instead of  $m/g$ , which can return a valid but non-minimal  $x$ .

## 97.12 Tests that matter

$6x \equiv 8 \pmod{14}$ : solvable even though  $\text{gcd}$  is 2.  
 $6x \equiv 7 \pmod{14}$ : impossible because 2 does not divide 7.

```
Any solvable query with $m = \gcd(a, m)$: the answer must be 0.
A coprime query with a negative Bezout coefficient: checks normalization.
200,000 queries with $a = 1$: checks input/output throughput.
```

## 97.13 D-specific stress testing

For tiny moduli, an independent oracle needs no number theory: scan every candidate  $x$  from 0 through  $m-1$  and keep the first for which  $a \cdot x \bmod m == b \bmod m$ . If none works, the expected answer is  $-1$ . This direct enumeration cross-checks both the divisibility rejection and the minimal-residue normalization without reusing extended Euclid.

The separate scale case uses  $a = 1$ . Then  $x == b \pmod m$ , so the known smallest answer is simply  $b \bmod m$  for each of 200000 queries. It exercises input/output and the logarithmic extended-gcd loop while retaining a direct expected output.

## 98 Formative upsolve exercises

1. For A, explain why computing  $(a/g)*b$  is safer than  $a*b/g$ , then evaluate the lcm of 1,000,000,000 and 1,000,000,000.

Hint: cancel the gcd before multiplying.

Answer: the gcd is  $10^9$ , so the lcm is  $10^9$ ; multiplying first is an unnecessary overflow risk in smaller integer types.

2. For B, use extended Euclid to find the inverse of 11 modulo 26.

Hint:  $26 = 2*11 + 4$ , then continue Euclid.

Answer:  $11*19 = 209 = 1 \pmod{26}$ , so the normalized inverse is 19.

3. For C, decide whether  $35x + 15y = 12$  is solvable before constructing any coefficients.

Hint: compute only the gcd first.

Answer: it is **not** solvable because  $\gcd(35,15)=5$  does not divide 12. This is the point of performing the divisibility test first.

## 99 Local workflow

Run the complete local suite with the reference target:

```
CP_TARGET=solution python3 sections/30_gcd_lcm_diophantine/check.py
```

Then run focused cases while developing:

```
A: (12,18), (7,5), (10^9,10^9)
B: (3,11), (10,17), (6,9), (25,1)
C: (6,10,14), (6,10,15), (7,5,1)
D: (6,8,14), (6,7,14), (10,1,17)
```

For C, a valid judge must check the equation itself rather than require one particular Bezout parameterization. Different valid pairs are expected.

## 100 Carry-forward checklist

1. Preserve a gcd or Bezout invariant; do not search candidates blindly.
2. Divide before multiplying for lcm, and still audit the final product range.
3. A modular inverse is a gcd-one statement, not automatically a Fermat statement.
4. For  $ax+by=c$ , gcd divisibility is both the impossibility test and the bridge to construction.
5. Treat negative coefficients as normal integer solutions, then verify by substitution.
6. For  $a \cdot x \equiv b \pmod{m}$ , divide by  $\gcd(a,m)$  only after checking that it divides  $b$ .

# 101 Full reference implementations

## 101.1 A. GCD LCM Queries: C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 while (q--) {
 long long a, b;
 cin >> a >> b;
 long long g = gcd(a, b);
 cout << g << ' ' << a / g * b << '\n';
 }
 return 0;
}
```

## 101.2 A. GCD LCM Queries: Python

```
import math
import sys

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 idx = 1
 out = []
 for _ in range(q):
 a, b = data[idx], data[idx + 1]
 idx += 2
 g = math.gcd(a, b)
 out.append(f"{g} {a // g * b}")
 print("\n".join(out))
```

```
if __name__ == "__main__":
 main()
```

## 101.3 B. General Inverse: C++

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
 if (b == 0) {
 x = 1;
 y = 0;
 return a;
 }
 long long x1, y1;
 long long g = egcd(b, a % b, x1, y1);
 x = y1;
 y = x1 - (a / b) * y1;
 return g;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 while (q--) {
 long long a, m;
 cin >> a >> m;
 if (m == 1) {
 cout << 0 << '\n';
 continue;
 }
 long long x, y;
 long long g = egcd(a, m, x, y);
 if (g != 1) {
 cout << -1 << '\n';
 } else {
 x %= m;
 if (x < 0) x += m;
 cout << x << '\n';
 }
 }
 return 0;
}
```



## 101.4 B. General Inverse: Python

```
import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
 if b == 0:
 return a, 1, 0
 g, x1, y1 = egcd(b, a % b)
 return g, y1, x1 - (a // b) * y1

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 idx = 1
 out = []
 for _ in range(q):
 a, m = data[idx], data[idx + 1]
 idx += 2
 if m == 1:
 out.append("0")
 continue
 g, x, _ = egcd(a, m)
 out.append(str(x % m) if g == 1 else "-1")
 print("\n".join(out))

if __name__ == "__main__":
 main()
```

## 101.5 C. Linear Diophantine: C++

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
 if (b == 0) {
 x = 1;
 y = 0;
 return a;
 }
 long long x1, y1;
 long long g = egcd(b, a % b, x1, y1);
 x = y1;
 y = x1 - (a / b) * y1;
 return g;
}
```

```

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 while (q--) {
 long long a, b, c;
 cin >> a >> b >> c;
 long long x, y;
 long long g = egcd(a, b, x, y);
 if (c % g != 0) {
 cout << "IMPOSSIBLE\n";
 } else {
 long long scale = c / g;
 cout << x * scale << ' ' << y * scale << '\n';
 }
 }
 return 0;
}

```

## 101.6 C. Linear Diophantine: Python

```

import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
 if b == 0:
 return a, 1, 0
 g, x1, y1 = egcd(b, a % b)
 return g, y1, x1 - (a // b) * y1

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 idx = 1
 out = []
 for _ in range(q):
 a, b, c = data[idx], data[idx + 1], data[idx + 2]
 idx += 3
 g, x, y = egcd(a, b)
 if c % g != 0:
 out.append("IMPOSSIBLE")
 else:
 scale = c // g
 out.append(f"{x * scale} {y * scale}")
 print("\n".join(out))

```

```
if __name__ == "__main__":
 main()
```

## 101.7 D. Clock Offset: C++

```
#include <bits/stdc++.h>
using namespace std;

long long extended_gcd(long long a, long long b, long long& x, long long& y) {
 if (b == 0) {
 x = 1;
 y = 0;
 return a;
 }
 long long x1, y1;
 long long g = extended_gcd(b, a % b, x1, y1);
 x = y1;
 y = x1 - (a / b) * y1;
 return g;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 while (q--) {
 long long a, b, m;
 cin >> a >> b >> m;

 long long coefficient, unused;
 long long g = extended_gcd(a, m, coefficient, unused);
 if (b % g != 0) {
 cout << -1 << '\n';
 continue;
 }

 long long reduced_modulus = m / g;
 long long scale = b / g;
 long long answer = (coefficient * scale) % reduced_modulus;
 if (answer < 0) answer += reduced_modulus;
 cout << answer << '\n';
 }
 return 0;
}
```

## 101.8 D. Clock Offset: Python

```
import sys

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
 if b == 0:
 return a, 1, 0
 g, x1, y1 = extended_gcd(b, a % b)
 return g, y1, x1 - (a // b) * y1

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 values = iter(data[1:])
 answer: list[str] = []
 for _ in range(q):
 a = next(values)
 b = next(values)
 m = next(values)
 g, coefficient, _ = extended_gcd(a, m)
 if b % g:
 answer.append("-1")
 continue
 reduced_modulus = m // g
 answer.append(str((coefficient * (b // g)) % reduced_modulus))
 sys.stdout.write("\n".join(answer) + "\n")

if __name__ == "__main__":
 main()
```

## 101.9 31. 1209E2 – Rotate Columns: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1209/E2> Local package: [problems/31\\_rotate\\_columns](#)

### 101.10 Restatement

For each positive integer  $x$ , print the smallest positive multiplier  $k$  such that  $x * k$  is a perfect square. The question is about the smallest multiplier, not merely whether  $x$  is already a square.

### 101.11 Constraints and traps

There can be 200000 values, each at most  $10^6$ . Trial-dividing every value individually repeats work; build one smallest-prime-factor table through the largest query value.

```
x = 1 already is a square, so its answer is 1.
Each prime with an odd exponent needs one more copy in k.
Each prime with an even exponent needs no copy in k.
k is a product of primes, so use long long in C++ even though this limit is safe.
```

Do not multiply every distinct prime factor. For  $12 = 2^2 * 3$ , the exponent of 2 is already even, so the answer is 3, not 6.

## 101.12 Observation and walk-through

Write the factorization as:

```
x = p1^e1 * p2^e2 * ... * pt^et
```

A perfect square has an even exponent for every prime. Therefore the smallest completion is:

```
k = product of pi over exactly those i for which ei is odd
```

For 72:

```
72 = 2^3 * 3^2
 ^^^ odd exponent: add one factor 2
 even exponent: add nothing

k = 2
72 * 2 = 2^4 * 3^2 = 144
```

For  $36 = 2^2 * 3^2$ , every exponent is already even, so the empty product gives  $k = 1$ .

## 101.13 Algorithm

```
read every query and build spf through the largest x

for each x:
 remaining = x
 k = 1
 while remaining > 1:
 p = spf[remaining]
 count how many times p divides remaining
 if that count is odd:
 k *= p
 print k
```

Only the parity of the exponent matters. The code can toggle a Boolean while dividing copies of  $p$ , avoiding a separate exponent integer.

## 101.14 Correctness proof

**Lemma 1.** The factor loop visits each distinct prime divisor of  $x$  once and determines its exact exponent in  $x$ .

**Proof.** `spf[remaining]` is a prime divisor of the current remaining value. The inner loop divides all copies of that prime, so the next outer iteration cannot choose it again. Repeating until the remaining value is one produces exactly the unique prime-factor blocks of  $x$ .  $\square$

**Lemma 2.** The multiplier produced by the algorithm makes  $x * k$  a perfect square.

**Proof.** By Lemma 1, for each prime  $p$  the algorithm knows its exponent  $e$  in  $x$ . If  $e$  is even it contributes no  $p$  to  $k$ , leaving an even exponent. If  $e$  is odd it contributes exactly one  $p$ , changing  $e$  to  $e+1$ , which is even. Thus every prime exponent in  $x*k$  is even, so the product is a square.  $\square$

**Lemma 3.** No smaller positive multiplier than the algorithm's  $k$  can make  $x$  a perfect square.

**Proof.** Every odd exponent of a prime  $p$  in  $x$  must be changed by an odd number of added copies of  $p$ ; at least one copy is necessary. The algorithm adds exactly one such copy and adds no unnecessary prime factor. Any valid multiplier must therefore be divisible by every prime in the algorithm's product, so it cannot be smaller.  $\square$

**Theorem.** The algorithm prints the smallest positive multiplier that completes every query value to a perfect square.

**Proof.** Lemma 2 proves the printed multiplier is valid, and Lemma 3 proves it is minimal.  $\square$

## 101.15 Complexity

Let  $M$  be the largest query value. The SPF table costs  $O(M \log \log M)$  time and  $O(M)$  memory. Each query divides out its prime factors, taking  $O(\text{number of prime factors with multiplicity})$  time.

## 101.16 C++ implementation notes

Read all values before constructing `spf`; then the precomputation stops at the needed maximum. `bool odd_exponent` toggles for each division. Store the output multiplier in `long long` for a safe general template.

## 101.17 Python implementation notes

The same SPF construction uses a list of integers. Collect output strings in a list and join once; this avoids one buffered write per query at the largest input size.

## 101.18 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

vector<int> build_spf(int limit) {
 vector<int> spf(limit + 1);
 iota(spf.begin(), spf.end(), 0);
 if (limit >= 1) spf[1] = 1;
 for (long long p = 2; p * p <= limit; ++p) {
 if (spf[p] == p) {
 for (long long multiple = p * p; multiple <= limit; multiple += p) {
 if (spf[multiple] == multiple) spf[multiple] = p;
 }
 }
 }
 return spf;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 cin >> q;
 vector<int> values(q);
 for (int &value : values) cin >> value;
 vector<int> spf = build_spf(*max_element(values.begin(), values.end()));

 for (int value : values) {
 int remaining = value;
 long long multiplier = 1;
 while (remaining > 1) {
 int prime = spf[remaining];
 bool odd_exponent = false;
 while (remaining % prime == 0) {
 odd_exponent = !odd_exponent;
 remaining /= prime;
 }
 if (odd_exponent) multiplier *= prime;
 }
 cout << multiplier << '\n';
 }
}

```

## 101.19 Full Python solution

```

import sys

def build_spf(limit: int) -> list[int]:
 spf = list(range(limit + 1))
 if limit >= 1:

```

```

 spf[1] = 1
 for prime in range(2, int(limit**0.5) + 1):
 if spf[prime] == prime:
 for multiple in range(prime * prime, limit + 1, prime):
 if spf[multiple] == multiple:
 spf[multiple] = prime
 return spf

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 values = data[1:1 + data[0]]
 spf = build_spf(max(values))
 out = []
 for value in values:
 remaining = value
 multiplier = 1
 while remaining > 1:
 prime = spf[remaining]
 odd_exponent = False
 while remaining % prime == 0:
 odd_exponent = not odd_exponent
 remaining //= prime
 if odd_exponent:
 multiplier *= prime
 out.append(str(multiplier))
 print("\n".join(out))

if __name__ == "__main__":
 main()

```

## 101.20 Common wrong answers

- Multiplying every distinct prime factor instead of only odd-exponent ones.
- Counting a prime factor only once without removing its repeated copies.
- Returning zero for  $x = 1$ ; the required positive multiplier is one.
- Trial-dividing every one of 200000 values instead of reusing an SPF table.
- Forgetting that a prime itself needs to be multiplied by itself.

## 101.21 Tests that matter

- $x = 1$  and an already perfect square;
- a prime such as 999983, whose answer is itself;
- a prime power with odd exponent, such as  $2^{19}$ ;
- a product with one odd and one even exponent, such as 72;
- many repeated queries, which exercise output handling and SPF reuse;
- values near  $10^6$ .



## 102 Testing and stress discussion

For A, compare against a slow loop that calls trial division on each number in a short interval. For B and C, compare factor-derived statistics with trial division on random values at most a few thousand. Essential fixed cases are:

```
A: [1,1], a singleton prime, and an interval with no primes
B: 1, a large prime, 64, and 360
C: 1, a prime, a prime power, and 210
D: 1, a square, a prime, and mixed odd/even prime exponents
```

For D's tiny random values, the generator tries multipliers one by one and checks integer square roots. This deliberately avoids using factorization in the oracle. A separate case contains 200000 copies of the large prime 999983, checking preprocessing, repeated queries, and output volume.

Run the supplied generator with:

```
CP_TARGET=solution python3 sections/31_sieve_primes_factorization/check.py
```

## 103 What to carry forward

Use a sieve for properties of every value in a bounded range. Use SPF when the question needs the factorization itself. Once prime exponents are available, many apparently different number-theory statistics become one short loop.

### 103.1 32. 1416C – XOR Inverse: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1416/C> Local package: [problems/32\\_xor\\_inverse](#)

### 103.2 Restatement

There are  $r$  distinct red performers and  $b$  distinct blue performers. Count the full lineups in which every performer appears exactly once and no adjacent pair is red-red. The answer is required modulo 1,000,000,007 for up to 200,000 queries.

This is the section's blind exercise. The statement describes the restriction but deliberately does not state how to turn it into a binomial coefficient.

### 103.3 Constraints and traps

$r$  and  $b$  are each at most one million. Any one query can have far more than  $10^6$  valid lineups, so enumeration is impossible and ordinary division is not valid after taking a modulus.

The performers are **distinct** even when they share a color. Swapping  $R_1$  and  $R_2$  produces a different lineup. It is therefore not enough to count color patterns.

When  $r > b+1$ , no valid lineup exists: even if every blue performer separates two red performers, there are only  $b+1$  places in which to put one red. That case must produce zero before indexing a binomial coefficient with an invalid lower argument.

The largest factorial index used is  $b+1$ , at most 1,000,001, safely below the prime modulus. Precomputing to  $r+b$  is unnecessary and can double the memory used by the factorial arrays.

### 103.4 Observations

First place only the blue performers. They can be ordered in  $b!$  ways. Once a blue order is fixed, it creates exactly  $b+1$  visible slots:

```
_ B _ B _ B _
```

At most one red performer may occupy a slot, otherwise two reds would become adjacent. Choose which  $r$  of the  $b+1$  slots receive a red in:

```
C(b+1, r)
```

ways. Finally, arrange the  $r$  distinct red performers in their selected slots in  $r!$  ways. The independent choices multiply, giving:

```
answer = b! * C(b+1, r) * r!
```

when  $r \leq b+1$ , and zero otherwise.

The useful discipline is the order of construction: reserve separators first, then choose legal gaps, then assign labels. It prevents both accidental red-red adjacency and accidental treatment of same-colored performers as identical.

## 103.5 Sample walk-through

For  $r=2$ ,  $b=2$ , first arrange the two blue performers. There are  $2! = 2$  orders. Consider one of them:

```
_ B1 _ B2 _
```

There are three slots and we choose two, so  $C(3,2)=3$  slot choices. For each choice, the two red performers can be assigned in  $2!=2$  orders. The total is:

```
2! * C(3,2) * 2! = 2 * 3 * 2 = 12.
```

For  $r=3, b=1$ , the blue performer gives only two slots:

```
_ B1 _
```

Three reds cannot occupy distinct slots, so the answer is zero.

## 103.6 Algorithm

```
read all (r,b) queries
limit = maximum over queries of max(r, b+1)
precompute fact[0..limit] and invfact[0..limit]

for each (r,b):
 if r > b+1:
 print 0
 else:
 slots = C(b+1, r)
 print fact[b] * slots * fact[r] modulo MOD
```

The  $\max(r, b+1)$  limit also covers malformed-but-guarded impossible queries: the arrays remain valid even before the  $r > b+1$  test.

## 103.7 Correctness proof

We prove that the algorithm returns the number of valid lineups for one query.

**Lemma 1.** After a blue-performer order is fixed, every valid lineup places at most one red performer in each of the  $b+1$  slots around that order.

**Proof.** The slots are exactly the positions before the first blue, between successive blues, and after the last blue. Two reds placed in one slot would have no blue performer between them and would be adjacent, contradicting validity. Conversely, reds in distinct slots always have at least one blue performer between consecutive selected slots, so they are not adjacent.  $\square$

**Lemma 2.** For a fixed blue order, the number of valid completions is  $C(b+1, r) * r!$  when  $r \leq b+1$ , and zero otherwise.

**Proof.** By Lemma 1, a valid completion chooses exactly  $r$  distinct slots from  $b+1$ , which can be done in  $C(b+1, r)$  ways. The  $r$  red performers are distinct, so each selected slot set has  $r!$  label assignments. The mapping from a slot set and assignment to a lineup is reversible. If  $r > b+1$ , no such slot set exists.  $\square$

**Theorem.** The algorithm prints exactly the number of valid lineups.

**Proof.** There are  $b!$  possible orders of the distinct blue performers. For each such order, Lemma 2 counts all and only its valid completions. The blue order is uniquely recoverable from any completed lineup, so completions from different blue orders do not overlap. Multiplying  $b!$  by the count from Lemma 2 produces exactly every valid lineup once. The algorithm computes this product modulo MOD, as requested.  $\square$

## 103.8 Complexity

Let  $L = \max \max(r, b+1)$  over all queries.

```
Preprocessing: O(L) time, O(L) memory
Each query: O(1) time
```

The one modular exponentiation used to make the inverse-factorial table costs  $O(\log \text{MOD})$ .

## 103.9 C++ implementation notes

Use `long long` for every modular product. Each factor is below MOD, and the product of two such values is below about  $10^{18}$ , which fits in signed 64-bit. The C++ reference reads every query before allocating, then computes the inverse factorial table by one Fermat inverse and a backward pass.

The formula has three factors. Reduce after each multiplication:

```
answer = fact[blue] * slots % MOD * fact[red] % MOD;
```

## 103.10 Python implementation notes

Use `pow(fact[limit], MOD - 2, MOD)` for the one modular inverse. Keep output in a list of strings and write it once; repeated printing is unnecessarily slow for the largest query count.

The value `blue + 1` is an ordinary Python integer, but it still must be included in the preprocessing limit before indexing the factorial tables.

## 103.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007LL;

long long mod_pow(long long base, long long exponent) {
 long long result = 1;
 while (exponent > 0) {
 if (exponent & 1LL) result = result * base % MOD;
 base = base * base % MOD;
 exponent >>= 1LL;
 }
 return result;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 vector<pair<int, int>> queries(q);
 int limit = 0;
 for (auto& [red, blue] : queries) {
 cin >> red >> blue;
 limit = max(limit, max(red, blue + 1));
 }

 vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
 for (int i = 1; i <= limit; ++i) fact[i] = fact[i - 1] * i % MOD;
 invfact[limit] = mod_pow(fact[limit], MOD - 2);
 for (int i = limit; i >= 1; --i) invfact[i - 1] = invfact[i] * i % MOD;

 for (auto [red, blue] : queries) {
 if (red > blue + 1) {
 cout << 0 << '\n';
 continue;
 }
 long long choose_slots = fact[blue + 1] * invfact[red] % MOD
 * invfact[blue + 1 - red] % MOD;
 long long answer = choose_slots * fact[red] % MOD * fact[blue] % MOD;
 }
}
```

```

 cout << answer << '\n';
 }
 return 0;
}

```

## 103.12 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
 fact = [1] * (limit + 1)
 for value in range(1, limit + 1):
 fact[value] = fact[value - 1] * value % MOD
 invfact = [1] * (limit + 1)
 invfact[limit] = pow(fact[limit], MOD - 2, MOD)
 for value in range(limit, 0, -1):
 invfact[value - 1] = invfact[value] * value % MOD
 return fact, invfact

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 queries = [(data[index], data[index + 1]) for index in range(1, 2 * q + 1, 2)]
 limit = max(max(red, blue + 1) for red, blue in queries)
 fact, invfact = prepare(limit)

 output: list[str] = []
 for red, blue in queries:
 if red > blue + 1:
 output.append("0")
 continue
 slots = fact[blue + 1] * invfact[red] % MOD * invfact[blue + 1 - red] % MOD
 output.append(str(slots * fact[red] % MOD * fact[blue] % MOD))
 sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__":
 main()

```

## 103.13 Common wrong answers

Counting only color patterns and forgetting the  $r!$  and  $b!$  label permutations.  
Using  $C(r+b, r)$ , which counts all lineups and ignores the adjacency restriction.  
Returning zero only when  $r > b$  instead of the correct  $r > b+1$  condition.  
Precomputing only through  $b$ , then indexing  $\text{fact}[b+1]$ .  
Dividing factorials with  $/$  after taking a modulo instead of multiplying by inverse factorials.

## 103.14 Tests that matter

$r=0, b=0$ : the empty lineup is one valid lineup.  
 $r=1, b=0$ : exactly one lineup;  $r=2, b=0$ : none.  
 $r=b+1$ : every gap must contain one red.  
 $r > b+1$ : impossible without any factorial formula.  
 $r=1, b=1,000,000$ : checks the  $b+1$  preprocessing boundary.  
200,000 impossible queries: checks input/output throughput.

## 104 Testing and stress discussion

For A, compare small values with `math.comb`. For B, enumerate permutations of short strings and compare the number of unique strings. For C, enumerate small nonnegative vectors recursively, then compare with the formula. For D, enumerate all labeled orders for at most eight people and reject adjacent-red orders directly. Important fixed tests are:

```
A: k=0, k=n, and the largest n
B: all equal letters and all distinct letters
C: s=0, n=1, and a query that determines the global maximum n+s-1
D: r=0, r=b+1, r>b+1, and b=1,000,000
```

Run the supplied checks with:

```
CP_TARGET=solution python3 sections/32_combinatorics_under_modulo/check.py
```



## 105 What to carry forward

First translate a story into a mathematical object. Then determine the largest factorial index *before* coding. Prime-modulus factorial preprocessing is a powerful constant-time query engine, but only while every relevant index remains strictly below the modulus.

### 105.1 33. 13E – Holes: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/13/E> Local package: [problems/33\\_holes](#)

### 105.2 Restatement

Count the ways to seat  $n$  labeled people in  $n$  labeled seats so that nobody sits in their own seat. This is the derangement count, requested modulo the prime 1\_000\_000\_007.

### 105.3 Constraints and traps

$n$  can reach 1\_000\_000, so enumerating  $n!$  seatings is impossible. The factorial arrays must therefore be linear, and all division in the formula is modular division.

```
n = 1 has answer 0: its only seating fixes that person.
The inclusion-exclusion sign is positive for k = 0.
Subtracting a term in C++ needs normalization back into [0, MOD).
The factorial bound is below MOD, so every fact[k] for k <= n is invertible.
```

### 105.4 Bad events and reduction

Let event  $E_i$  mean person  $i$  remains in seat  $i$ . We want permutations in none of these events. For any chosen set of  $k$  fixed people, the remaining  $n-k$  people can be seated in  $(n-k)!$  ways. There are  $C(n,k)$  chosen sets, so inclusion-exclusion gives:

```
answer = sum(k=0..n) (-1)^k C(n,k) (n-k)!
 = sum(k=0..n) (-1)^k n! / k!
```

All arithmetic is modulo prime MOD, so compute factorials and inverse factorials with Fermat inverses.

For  $n=4$ , the alternating terms are  $24 - 24 + 12 - 4 + 1 = 9$ .

## 105.5 Worked intersection trace

For three people, define  $E_1, E_2, E_3$ . There are  $3! = 6$  unrestricted seatings. Each individual event, such as  $E_1$ , fixes one person and leaves two people free, hence has  $2! = 2$  seatings. The three single-event counts subtract  $3 * 2 = 6$ .

Any two fixed-seat events force the third person as well, so each pair intersection has  $1! = 1$  seating and the three pair intersections add 3. The triple intersection also has one seating and is subtracted:

$$6 - 3*2 + 3*1 - 1 = 2$$

The two remaining seatings are the two 3-cycles. More generally, choosing a specific set of  $k$  events forces exactly those  $k$  people, leaving  $(n-k)!$  arrangements; there are  $C(n,k)$  ways to choose the set.

## 105.6 Algorithm

```
precompute fact[0..n] and inv_fact[0..n]
answer = 0
for k from 0 to n:
 term = fact[n] * inv_fact[k]
 add term for even k; subtract it for odd k
print normalized answer
```

## 105.7 Correctness proof

**Lemma.** The intersection of any selected  $k$  fixed-seat events contains  $(n-k)!$  permutations.

**Proof.** Those  $k$  people and seats are forced. The remaining people may be permuted freely among remaining seats.  $\square$

**Theorem.** The algorithm returns the number of seatings with no original seat match.

**Proof.** Inclusion-exclusion over events  $E_i$  adds and subtracts the intersection counts from the lemma. A permutation with one or more fixed seats has total coefficient zero; a permutation with none has coefficient one. The factorial formula computes each grouped term modulo MOD.  $\square$

## 105.8 Complexity

```
Time: $O(n)$
Memory: $O(n)$
```

## 105.9 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
long long power(long long a, long long b) {
 long long r = 1;
 for (; b; b >>= 1, a = a * a % MOD) if (b & 1) r = r * a % MOD;
 return r;
}
int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);
 int n; cin >> n;
 vector<long long> fact(n + 1), inv_fact(n + 1);
 fact[0] = 1;
 for (int i = 1; i <= n; ++i) fact[i] = fact[i - 1] * i % MOD;
 inv_fact[n] = power(fact[n], MOD - 2);
 for (int i = n; i; --i) inv_fact[i - 1] = inv_fact[i] * i % MOD;
 long long answer = 0;
 for (int k = 0; k <= n; ++k) {
 long long term = fact[n] * inv_fact[k] % MOD;
 answer = (answer + (k & 1 ? -term : term) + MOD) % MOD;
 }
 cout << answer << '\\n';
}

```

## 105.10 C++ implementation notes

Compute  $\text{inv\_fact}[n] = \text{fact}[n]^{(\text{MOD}-2)}$  once, then fill inverse factorials downward with  $\text{inv\_fact}[i-1] = \text{inv\_fact}[i] * i$ . Normalize each alternating update immediately with  $+ \text{MOD}$  before  $\% \text{MOD}$ , rather than allowing a negative C++ remainder to escape.

## 105.11 Full Python solution

```

import sys
MOD = 1_000_000_007

def main() -> None:
 n = int(sys.stdin.buffer.read())
 fact = [1] * (n + 1)
 for i in range(1, n + 1):
 fact[i] = fact[i - 1] * i % MOD
 inv_fact = [1] * (n + 1)
 inv_fact[n] = pow(fact[n], MOD - 2, MOD)
 for i in range(n, 0, -1):
 inv_fact[i - 1] = inv_fact[i] * i % MOD
 print(sum(((-1 if k & 1 else 1) * fact[n] * inv_fact[k]) for k in range(n + 1)) % MOD)

```

```
if __name__ == "__main__":
 main()
```

## 105.12 Python implementation notes

Python's final % MOD normalizes a negative alternating sum automatically. The list comprehensions are not needed here: fill the two factorial arrays in loops so their  $O(n)$  storage and indexing remain explicit.

## 105.13 Traps and tests

- The sign starts positive at  $k=0$ .
- $n! / k!$  is modular multiplication by an inverse factorial, not integer division after reduction.
- Test  $n=1$ ,  $n=2$ , and a large  $n$  for factorial preprocessing.

## 106 Stress testing

For  $n \leq 8$ , enumerate all permutations and directly reject those with a fixed position. This is independent of inclusion-exclusion. The scale oracle uses the separate derangement recurrence  $D[n] = (n-1)(D[n-1] + D[n-2])$  for  $n=1000000$ .

## 107 What to carry forward

For a “none of these conditions” count, define one bad event per condition, count a selected intersection, and group equal-size subsets only after that intersection is understood.

### 107.1 34. 342E – Xenia and Tree: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/342/E> Local package: [problems/34\\_xenia\\_and\\_tree](#)

### 107.2 Restatement

Independent game  $i$  is won with probability  $p[i] / q[i]$ . Count every unordered pair of games that are both won, then print the expectation of that count modulo  $MOD$ .

### 107.3 Constraints and traps

$n$  can be 200000, so checking all  $O(n^2)$  pairs is too slow. The result is an exact rational number, not a decimal. Each  $q[i]$  is nonzero modulo the prime  $MOD$ , so represent each probability with its Fermat inverse.

Independence matters here: for two distinct games, the probability that both win is the product of their probabilities. A game with probability zero simply contributes no winning pair.

### 107.4 Observation and trace

For each pair  $i < j$ , define an indicator that is one precisely when both games win. Linearity gives the expected answer as the sum of all pair probabilities:

```
sum over $i < j$ of $\text{probability}[i] * \text{probability}[j]$
```

Process games left to right. If `sum_previous` is the sum of earlier win probabilities, a new chance  $p$  contributes  $p * \text{sum\_previous}$ , then joins the sum.

For probabilities  $1/2$ ,  $1/3$ ,  $1$ :

```
new 1/2: contribution 0
new 1/3: contribution (1/3)*(1/2) = 1/6
new 1: contribution 1*(1/2 + 1/3) = 5/6
total = 1
```

## 107.5 Algorithm

```
sum_previous = answer = 0
for each (p, q):
 chance = p * inverse(q) mod MOD
 answer += chance * sum_previous
 sum_previous += chance
 reduce both modulo MOD
print answer
```

## 107.6 Correctness proof

**Lemma.** Before processing game  $j$ , `sum_previous` equals the sum of the win probabilities of exactly games  $1..j-1$ , and `answer` equals the expected number of winning pairs wholly among those games.

**Proof.** Both values are zero before game one. When adding game  $j$ , each new pair is exactly  $(i, j)$  for one earlier game  $i$ . Independence gives its probability as `chance * probability[i]`, whose sum is `chance * sum_previous`. Adding that preserves the answer invariant, and then adding `chance` preserves the probability-sum invariant.  $\square$

**Theorem.** The algorithm prints the expected number of unordered pairs of games that are both won.

**Proof.** After all games, the lemma says `answer` sums the probability of every unordered winning-pair indicator exactly once. By linearity of expectation, that sum is the requested expectation.  $\square$

## 107.7 Complexity

Each modular inverse uses binary exponentiation.

```
Time: O(n log MOD)
Memory: O(1) besides input/output buffering
```

## 107.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD=1000000007LL;
long long mod_pow(long long a,long long e){long long r=1;for(;e>=1;a=a*a%MOD)if(e&1)r=r*a%M
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>>n))return 0;long long
```

## 107.9 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 total = answer = 0
 for index in range(data[0]):
 p, q = data[1 + 2 * index:3 + 2 * index]
 chance = p * pow(q, MOD - 2, MOD) % MOD
 answer = (answer + total * chance) % MOD
 total = (total + chance) % MOD
 print(answer)

if __name__ == "__main__":
 main()

```

## 107.10 Implementation notes and tests

- Use `long long` for modular products in C++; use `pow(q, MOD-2, MOD)` in Python.
- Do not add `chance * chance`: pairs require two distinct games.
- Test one game, probability zero, all certain wins, and mixed fractions.

## 107.11 Stress testing

For at most ten games, enumerate every win/loss outcome with exact rational `Fraction` arithmetic, count its winning pairs, and weight it by its outcome probability. This does not reuse the running-sum formula. The scale case has 200000 games of probability  $1/2$ , whose expected answer is  $C(200000, 2) / 4$  modulo MOD.



## 108 Local workflow

Run:

```
CP_TARGET=solution python3 sections/34_probability_expected_value/check.py
```

For small verification, use exact rational arithmetic. Enumerate a few reward outcomes, solve the first-step equation symbolically, and compare coupon stage sums using Python Fraction before reducing numerator times inverse(denominator) modulo MOD. # Full reference implementations

### 108.1 A. Expected Prize: C++

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
 long long result = 1;
 while (b > 0) {
 if (b & 1LL) result = result * a % MOD;
 a = a * a % MOD;
 b >>= 1LL;
 }
 return result;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 if (!(cin >> n)) return 0;
 long long ans = 0;
 for (int i = 0; i < n; i++) {
 long long p, q, v;
 cin >> p >> q >> v;
 ans = (ans + (v % MOD) * (p % MOD) % MOD * mod_pow(q, MOD - 2)) % MOD;
 }
 cout << ans << '\n';
 return 0;
}
```

## 108.2 A. Expected Prize: Python

```
import sys

MOD = 1_000_000_007

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n = data[0]
 idx = 1
 ans = 0
 for _ in range(n):
 p, q, v = data[idx], data[idx + 1], data[idx + 2]
 idx += 3
 ans = (ans + v % MOD * p % MOD * pow(q, MOD - 2, MOD)) % MOD
 print(ans)

if __name__ == "__main__":
 main()
```

## 108.3 B. First Success: C++

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
 long long result = 1;
 while (b > 0) {
 if (b & 1LL) result = result * a % MOD;
 a = a * a % MOD;
 b >>= 1LL;
 }
 return result;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int t;
 if (!(cin >> t)) return 0;
 while (t--) {
 long long p, q;
```

```

 cin >> p >> q;
 cout << q % MOD * mod_pow(p, MOD - 2) % MOD << '\n';
 }
 return 0;
}

```

## 108.4 B. First Success: Python

```

import sys

MOD = 1_000_000_007

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 t = data[0]
 idx = 1
 out = []
 for _ in range(t):
 p, q = data[idx], data[idx + 1]
 idx += 2
 out.append(str(q % MOD * pow(p, MOD - 2, MOD) % MOD))
 print("\n".join(out))

if __name__ == "__main__":
 main()

```

## 108.5 C. Coupon Collector: C++

```

#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
 long long result = 1;
 while (b > 0) {
 if (b & 1LL) result = result * a % MOD;
 a = a * a % MOD;
 b >>= 1LL;
 }
 return result;
}

```

```

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int q;
 if (!(cin >> q)) return 0;
 vector<int> values(q);
 int limit = 0;
 for (int &n : values) {
 cin >> n;
 limit = max(limit, n);
 }
 vector<long long> harmonic(limit + 1, 0);
 for (int i = 1; i <= limit; i++) {
 harmonic[i] = (harmonic[i - 1] + mod_pow(i, MOD - 2)) % MOD;
 }
 for (int n : values) cout << n * harmonic[n] % MOD << '\n';
 return 0;
}

```

## 108.6 C. Coupon Collector: Python

```

import sys

MOD = 1_000_000_007

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 q = data[0]
 values = data[1 : 1 + q]
 limit = max(values)
 inv = [0] * (limit + 1)
 harmonic = [0] * (limit + 1)
 for i in range(1, limit + 1):
 inv[i] = pow(i, MOD - 2, MOD)
 harmonic[i] = (harmonic[i - 1] + inv[i]) % MOD
 print("\n".join(str(n * harmonic[n] % MOD) for n in values))

if __name__ == "__main__":
 main()

```

## 109 What to carry forward

Define the random variable or expected remaining state first. Linearity avoids distribution enumeration, and a first-step equation exposes every self-loop.

### 109.1 35. 895C – Square Subsets: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/895/C> Local package: [problems/35\\_square\\_subsets](#)

### 109.2 Restatement and constraints

There are  $k$  symbols, numbered from 1 to  $k$ . Count the strings of length  $n$  in which every symbol appears at least once, modulo 1,000,000,007. Here  $1 \leq n \leq 10^{18}$  and  $1 \leq k \leq 20$ . The huge length rules out building strings or dynamic programming over positions; the small alphabet is the important bound.

This is a *surjection* count: each of the  $k$  symbols must receive at least one of the  $n$  positions. It is not simply  $k^n$ , because that includes strings that miss one or more symbols.

### 109.3 Observation and worked trace

For each symbol  $s$ , let  $E_s$  be the set of strings that never use  $s$ . We want strings outside the union of these bad sets. If we choose a particular set of  $j$  missing symbols, every position has  $k-j$  choices, so exactly  $(k-j)^n$  strings miss all selected symbols. There are  $C(k, j)$  choices for that selected set.

For  $n=4$ ,  $k=3$ , begin with all  $3^4=81$  strings. Subtract the strings missing one named symbol:  $3 * 2^4 = 48$ . A string using only one symbol was subtracted twice, so add back  $C(3,2) * 1^4 = 3$ . Missing all three symbols contributes  $0^4=0$  because  $n$  is positive. The answer is:

$$\begin{aligned} 3^4 - C(3,1)2^4 + C(3,2)1^4 - C(3,3)0^4 \\ = 81 - 48 + 3 - 0 \\ = 36 \end{aligned}$$

The sign alternates because a string missing exactly  $r$  symbols belongs to  $C(r, j)$  selected missing sets at level  $j$ . Its total coefficient is  $\sum_j (-1)^j C(r, j) = (1-1)^r$ : zero when  $r>0$ , and one when  $r=0$ .

### 109.4 Algorithm

```

answer = 0
for missing from 0 to k:
 term = C(k, missing) * (k - missing)^n modulo MOD
 if missing is even:
 answer += term
 else:
 answer -= term
print answer modulo MOD

```

Compute every power with binary exponentiation. Since  $k \leq 20$ , ordinary integer binomial coefficients fit comfortably before reduction; both languages can compute them directly.

## 109.5 Correctness proof

For a fixed selected set  $S$  of missing symbols, a string avoids every symbol in  $S$  exactly when all its positions use one of the other  $k - \text{missing}$  symbols. Thus it contributes  $(k - \text{missing})^n$  strings. Summing over the  $C(k, \text{missing})$  possible  $S$  counts each string with the usual inclusion-exclusion sign.

Consider any string that misses exactly  $r$  symbols. It occurs once for every selected subset of those  $r$  missed symbols, so its coefficient in the sum is  $\sum_{j=0}^r (-1)^j C(r, j)$ . This is 1 for  $r=0$  and 0 for  $r>0$ . Therefore every string using all symbols is counted once and every invalid string is cancelled, which is precisely the required answer. QED.

## 109.6 Complexity, implementation notes, and tests

There are  $k+1$  terms, and each modular power takes  $O(\log n)$ , for total time  $O(k \log n)$  and  $O(1)$  extra space. Keep the answer reduced after every addition/subtraction. In C++, use `long long` for modular products; in Python, use `pow(base, n, MOD)` rather than constructing `base**n`.

Test  $n < k$  (answer zero),  $k=1$  (answer one),  $n=k$ , and a very large  $n$  to make sure a linear-in- $n$  solution cannot slip through. A common error is to subtract  $k \cdot (k-1)^n$  without restoring strings that miss two symbols. Another is treating  $0^0$  casually; the local constraint  $n \geq 1$  makes the final term zero when every symbol is selected as missing.

## 109.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007LL;

long long mod_pow(long long base, long long exponent) {
 long long result = 1;
 while (exponent > 0) {
 if (exponent & 1LL) result = result * base % MOD;
 base = base * base % MOD;
 exponent >>= 1LL;
 }
}

```

```

 return result;
}

long long choose(int n, int r) {
 r = min(r, n - r);
 long long result = 1;
 for (int i = 1; i <= r; i++) result = result * (n - r + i) / i;
 return result;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 long long n;
 int k;
 if (!(cin >> n >> k)) return 0;

 long long answer = 0;
 for (int missing = 0; missing <= k; missing++) {
 long long term = choose(k, missing) * mod_pow(k - missing, n) % MOD;
 if (missing % 2 == 0) answer += term;
 else answer -= term;
 answer %= MOD;
 }
 cout << (answer + MOD) % MOD << '\n';
 return 0;
}

```

## 109.8 Full Python solution

```

import math
import sys

MOD = 1_000_000_007

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n, k = data

 answer = 0
 for missing in range(k + 1):
 term = math.comb(k, missing) * pow(k - missing, n, MOD)
 if missing % 2 == 0:
 answer += term
 else:
 answer -= term

```

```
print(answer % MOD)

if __name__ == "__main__":
 main()
```

## 109.9 D-specific stress testing

For tiny cases ( $n \leq 6$ ,  $k \leq 4$ ), enumerate all  $k^n$  strings directly and count those whose set of used symbols has size  $k$ . This oracle never uses the inclusion-exclusion formula, so it catches sign, combination, and overlap mistakes independently. Include  $n < k$ , where no string can use every symbol, and  $k = 1$ , where the one possible string is complete.

The separate scale case uses  $k = 20$  and an  $n$  near  $10^{18}$ . It verifies that the implementation uses logarithmic modular exponentiation for each term, rather than attempting a linear-in- $n$  enumeration, DP, or ordinary power.



## 110 Testing discussion

The mixed contest generators compare against direct combinatorics, brute-force gcd pair checks, and exact rational probability. This phase is especially prone to off-by-one and complement mistakes, so stress tests are part of the workflow.

## 111 What to carry forward

Math contest problems are often a composition of small tools. The main skill is recognizing the composition early: combinations plus subtraction, sieve plus inclusion-exclusion, or probability plus complement. # Extended upsolve workshop

**A. Blocked Path.** Subtract  $\text{ways}(\text{start}, \text{block}) * \text{ways}(\text{block}, \text{finish})$  from all monotone paths. Test a blocked start or finish, where every path is forbidden.

**B. Coprime Pairs.** For each value, inclusion-exclusion over its distinct prime factors counts earlier values sharing any factor; subtract from prior count. Test value one and repeated composite values.

**C. At Least One.** Compute  $1 - \text{product}(1 - p_i/q_i)$ . Directly adding event probabilities double-counts overlapping outcomes; test one certain event. # Full upsolve notes

**A. Blocked Path.** Every forbidden monotone path has a unique prefix ending at  $(R, C)$  and suffix from it, so subtract  $C(R+C-2, R-1) * C(H-R+W-C, H-R)$  from all paths. For a 3-by-3 grid blocked in the middle:  $6 - 2 * 2 = 2$ . Test a blocked start/end, where the answer is zero.

**B. Coprime Pairs.** Maintain  $\text{seen}[d]$ , the number of prior values divisible by square-free  $d$ . For 30 after  $[6, 10, 21]$ , inclusion-exclusion gives  $\text{seen}[2] + \text{seen}[3] + \text{seen}[5] - \text{seen}[6] - \text{seen}[10] - \text{seen}[15] + \text{seen}[30] = 3$ ; all earlier values share a factor. Count before updating the subset products. SPF plus at most 127 subsets per local value gives safe complexity. Test one, duplicates, and a three-factor value.

**C. At Least One.** Compute  $\text{none} = \text{product}((q-p) * \text{inverse}(q))$ , then answer  $1 - \text{none}$ . For  $1/2$  and  $1/3$ , this is  $1 - 1/2 * 2/3 = 2/3$ . Independence justifies the product; a certain event and a zero-probability event are essential tests.

**D. Complete Alphabet.** Let a bad event be that one named symbol is absent. For  $n=4, k=3$ , inclusion-exclusion is  $3^4 - 3 * 2^4 + 3 * 1^4 = 36$ . Test  $n < k$ , one symbol, and a huge exponent; the alternating signs repair all overlaps between absent-symbol events.

## 112 Problem-local upsolve checklist

### 112.1 A. Blocked Path

#### 112.1.1 Constraints and model

H and W are up to one million, so grid DP over  $H \times W$  cells is not viable. There is exactly one blocked cell. A monotone path is completely determined by the positions of its down moves, which makes binomial coefficients the correct model. Precompute factorials through  $H+W$  once.

#### 112.1.2 Trace and pseudocode

For  $H=3, W=4, R=2, C=2$ , all paths are  $C(5,2)=10$ . The blocked prefix count is  $C(2,1)=2$ ; the suffix count is  $C(3,1)=3$ ; subtract 6 to get 4.

```
all = choose(H+W-2, H-1)
through = choose(R+C-2, R-1) * choose(H-R+W-C, H-R)
answer = all - through modulo MOD
```

Every forbidden path has one and only one such prefix/suffix pair, proving the multiplication and subtraction. The time is  $O(H+W)$  preprocessing and  $O(1)$  work afterward. In C++ use `long long` products; Python uses the same factorial lists. Wrong answers usually forget that a blocked start or end makes the answer zero, or apply this one-block formula to multiple blocks. Test both endpoints, a one-row grid, and a block adjacent to an endpoint.

### 112.2 B. Coprime Pairs

#### 112.2.1 Constraints and state

The array has up to 200,000 values, each at most one million. Build SPF once. For each square-free divisor  $d$ , `seen[d]` is the number of already processed values divisible by  $d$ . Prime powers are deliberately absent: sharing a prime is the event being counted.

#### 112.2.2 Trace and pseudocode

When processing  $30=2 \times 3 \times 5$  after  $[6, 10, 21]$ , count shared values as:

```
+seen[2] + seen[3] + seen[5]
-seen[6] - seen[10] - seen[15]
+seen[30]
```

This is 3, hence it contributes  $3-3=0$  coprime pairs. Then increment every one of those seven `seen` entries for future values.

```

factors = distinct prime factors of x from SPF
shared = alternating sum seen[product(subset)]
answer += processed - shared
for every nonempty subset: seen[product(subset)] += 1

```

An earlier value is included in the alternating sum once iff it shares at least one prime with  $x$ ; otherwise it appears in no selected event. Therefore `processed-shared` is precisely the number of new coprime pairs. With at most seven distinct factors, each value creates at most 127 subsets. Complexity is  $O(M \log \log M + \sum 2^{\omega(a[i])})$  time and  $O(M)$  memory. Test 1 (no subsets, coprime with all prior values), repeated composites, and primes.

### 112.2.3 Implementation and failure modes

Generate subset products from distinct factors, not from the factorization with multiplicity. Count before updating `seen`. Use `long long` for the answer: there can be about  $n*(n-1)/2$  pairs. A brute-force oracle for random arrays of length at most 10 should compare every gcd directly.

## 112.3 C. At Least One

### 112.3.1 Constraints and model

The statement explicitly gives independent events and prime-modulus rational probabilities with nonzero denominators. Directly adding event probabilities overcounts outcomes where several events occur. The complement has one product: none of the events happens.

### 112.3.2 Trace, proof, and code shape

For probabilities  $1/2$ ,  $1/3$ ,  $1/4$ , the no-event probability is  $(1/2)*(2/3)*(3/4)=1/4$ , so the answer is  $3/4$ . In modular arithmetic compute each factor as  $(q-p) * \text{inverse}(q)$ , multiply them, and normalize  $1-\text{none}$ .

```

none = 1
for p, q:
 none = none * (q-p) * inverse(q) modulo MOD
print (1-none) modulo MOD

```

Independence proves the product; complementarity proves the final subtraction. The algorithm takes  $O(n \log \text{MOD})$  with Fermat inverses and  $O(1)$  extra memory. Test one event, a zero-probability event, and a certain event. Do not use this method if events are dependent or a denominator is zero modulo MOD.

## 112.4 36. 868F – Yet Another Minimization Problem: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/868/F> Local package: [problems/36\\_yet\\_another\\_minimization](#)

## 112.5 Restatement

An initially zero array receives interval additions. Between updates, queries ask for the sum over another interval. Both operation types must be processed online: a later operation sees every earlier update.

This is the local blind exercise. It combines the two one-sided patterns from the section, but the statement does not reveal how to make both the range update and the range-sum query logarithmic.

## 112.6 Constraints and traps

$n, q \leq 200,000$ , so literally adding to every position of a long interval is too slow. An operation sequence with 200,000 full-length updates would take quadratic time with a plain list.

The update amount can be negative. Sums may have magnitude up to  $200,000 * 200,000 * 10^6 = 4 * 10^{16}$ , which fits in signed 64-bit but not in `int`. Store Fenwick cells, update products, and answers in `long long` in C++.

The difference-array endpoint at  $r+1$  is real even when  $r=n$ . Allocate each Fenwick tree with an extra usable index. Omitting that index lets an update leak past the end of the array instead of cancelling there.

## 112.7 Observations

Let  $d$  be the difference array. A range add is small in that representation:

```
add x on [l,r] means d[l] += x, d[r+1] -= x.
```

The current value at position  $i$  is the prefix  $d[1] + \dots + d[i]$ . A single Fenwick tree over  $d$  therefore solves range-add/point-query, as in problem C. For a range sum, we need the prefix of those point values:

```
P(p) = value[1] + value[2] + ... + value[p]
 = sum(i=1..p) sum(j=1..i) d[j]
 = sum(j=1..p) d[j] * (p-j+1)
 = (p+1) * sum d[j] - sum j*d[j].
```

This last line is the key. Maintain two Fenwick trees:

```
B1 stores d[j] so B1.prefix(p) = sum d[j]
B2 stores j*d[j] so B2.prefix(p) = sum j*d[j]
```

Then:

```
P(p) = (p+1) * B1.prefix(p) - B2.prefix(p).
sum(l,r) = P(r) - P(l-1).
```

## 112.8 Sample walk-through

After add 3 on [2,4], the difference array has nonzero entries:

```
d[2] = +3, d[5] = -3.
```

For  $p=4$ , the two maintained prefixes are:

```
B1.prefix(4) = 3
B2.prefix(4) = 2*3 = 6
P(4) = (4+1)*3 - 6 = 9.
```

That is the sum of [0,3,3,3]. After adding -1 to [1,5], the query [2,3] is  $P(3)-P(1)=5-1=4$ . The final +10 at index three makes [2,3] into [2,12]; the final query [3,5] is  $12+2-1=13$ .

## 112.9 Algorithm

```
make two Fenwick trees B1 and B2 with indices 1..n+1

range_add(l, r, x):
 add B1 at l with x
 add B1 at r+1 with -x
 add B2 at l with l*x
 add B2 at r+1 with -(r+1)*x

prefix_sum(p):
 return (p+1) * B1.prefix(p) - B2.prefix(p)

for each operation:
 if it is type 1: range_add(l, r, x)
 otherwise: print prefix_sum(r) - prefix_sum(l-1)
```

`prefix_sum(0)` is zero because both Fenwick prefix queries at index zero are zero. This makes the  $l=1$  range-query boundary automatic.

### 112.10 Correctness proof

We prove that every printed query answer is correct.

**Lemma 1.** After all updates processed so far,  $B1.prefix(p)$  equals  $\sum_{j=1..p} d[j]$  and  $B2.prefix(p)$  equals  $\sum_{j=1..p} j*d[j]$  for the current difference array.

**Proof.** Each range update changes exactly  $d[l]$  by  $x$  and  $d[r+1]$  by  $-x$ . The algorithm applies those same changes to B1, and applies each change multiplied by its index to B2. A Fenwick prefix query is the sum of its stored point values, so both stated equalities hold after every update.  $\square$

**Lemma 2.** `prefix_sum(p)` returned by the algorithm equals the sum of the first  $p$  logical array values.

**Proof.** The logical value at position  $i$  is  $\sum_{j=1..i} d[j]$ . Swapping the order of summation gives:

```

sum(i=1..p) value[i]
= sum(j=1..p) d[j]*(p-j+1)
= (p+1)*sum(j=1..p)d[j] - sum(j=1..p)j*d[j].

```

By Lemma 1, the two Fenwick prefixes are exactly the two sums on the last line, which is the formula used by `prefix_sum`.  $\square$

**Theorem.** A type-2 query  $[l, r]$  is printed with its correct range sum.

**Proof.** By Lemma 2, `prefix_sum(r)` is the sum of positions  $1..r$ , and `prefix_sum(l-1)` is the sum of positions  $1..l-1$ . Their difference leaves exactly the positions  $l..r$ .  $\square$

## 112.11 Complexity

Each update uses four Fenwick point additions, and each query uses four Fenwick prefix sums:

```

Time: O(log n) per operation
Memory: O(n)

```

## 112.12 C++ implementation notes

Keep the trees indexed through  $n+1$ . In the weighted tree, write `delta * index` with `delta` as `long long`; otherwise `int` multiplication can overflow before it reaches the tree.

Lambdas for `range_add` and `prefix_sum` keep the four coordinated updates and the one formula together, reducing endpoint-copy mistakes.

## 112.13 Python implementation notes

Python integers safely hold all intermediate products. Parse the full input once and maintain an index into the token list; append answers to a list and write it once. The Fenwick loops still use ordinary integer indices.

## 112.14 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
 int n;
 vector<long long> bit;

 explicit Fenwick(int size) : n(size), bit(size + 1, 0) {}

 void add(int index, long long delta) {
 for (; index <= n; index += index & -index) bit[index] += delta;
 }
}

```

```

 long long sum(int index) const {
 long long result = 0;
 for (; index > 0; index -= index & -index) result += bit[index];
 return result;
 }
};

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, q;
 if (!(cin >> n >> q)) return 0;
 Fenwick coefficients(n + 1), weighted_coefficients(n + 1);

 auto add_range = [&](int left, int right, long long delta) {
 coefficients.add(left, delta);
 coefficients.add(right + 1, -delta);
 weighted_coefficients.add(left, delta * left);
 weighted_coefficients.add(right + 1, -delta * (right + 1));
 };

 auto prefix_sum = [&](int position) {
 long long coefficient_sum = coefficients.sum(position);
 long long weighted_sum = weighted_coefficients.sum(position);
 return (position + 1LL) * coefficient_sum - weighted_sum;
 };

 while (q--) {
 int type, left, right;
 cin >> type >> left >> right;
 if (type == 1) {
 long long delta;
 cin >> delta;
 add_range(left, right, delta);
 } else {
 cout << prefix_sum(right) - prefix_sum(left - 1) << '\n';
 }
 }
 return 0;
}

```

## 112.15 Full Python solution

```

import sys

class Fenwick:
 def __init__(self, size: int) -> None:
 self.size = size
 self.bit = [0] * (size + 1)

```



```

def add(self, index: int, delta: int) -> None:
 while index <= self.size:
 self.bit[index] += delta
 index += index & -index

def sum(self, index: int) -> int:
 result = 0
 while index > 0:
 result += self.bit[index]
 index -= index & -index
 return result

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n, q = data[0], data[1]
 coefficients = Fenwick(n + 1)
 weighted_coefficients = Fenwick(n + 1)

 def add_range(left: int, right: int, delta: int) -> None:
 coefficients.add(left, delta)
 coefficients.add(right + 1, -delta)
 weighted_coefficients.add(left, delta * left)
 weighted_coefficients.add(right + 1, -delta * (right + 1))

 def prefix_sum(position: int) -> int:
 return (position + 1) * coefficients.sum(position) - weighted_coefficients.sum(position)

 index = 2
 output: list[str] = []
 for _ in range(q):
 operation = data[index]
 left = data[index + 1]
 right = data[index + 2]
 index += 3
 if operation == 1:
 delta = data[index]
 index += 1
 add_range(left, right, delta)
 else:
 output.append(str(prefix_sum(right) - prefix_sum(left - 1)))
 sys.stdout.write("\n".join(output) + ("\n" if output else ""))

if __name__ == "__main__":
 main()

```

## 112.16 Common wrong answers

Using one tree over differences and treating its prefix as a range sum.  
Forgetting to multiply the B2 updates by their exact endpoint index.  
Using  $p \cdot B1.prefix(p) - B2.prefix(p)$  instead of  $(p+1) \cdot \dots$  .  
Not allocating the  $r+1$  endpoint when  $r=n$ .  
Using int for values or products after many updates.

## 112.17 Tests that matter

$n=1$  with positive and negative updates.  
Full-range updates where  $r=n$ , to exercise the cancellation endpoint.  
Queries with  $l=1$ , where  $prefix\_sum(0)$  is needed.  
Overlapping updates of opposite signs.  
200,000 updates and queries, to reject linear scans and test 64-bit totals.

## 113 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs with a literal mutable array, then the same expected files are used against both the C++ and Python submissions. For D, include full-range updates, negative deltas, and queries whose left endpoint is one.

# 114 Problem-specific upsolve notes

## 114.1 A. Point Add Range Sum

**Constraint implication and trap.** With a long sequence of updates and queries, scanning  $[1, r]$  per query is quadratic in the worst case. The update is an addition, not an assignment: call `add(i, x)`, never overwrite the old Fenwick contribution.

**Walk-through.** From  $[1, 2, 3, 4]$ , query  $[2, 4]$  is `prefix(4)-prefix(1)=9`. After `add(3, 10)`, the same query is 19. The tree's prefix invariant proves the subtraction removes exactly positions before 1.

```
build: for i=1..n: add(i, a[i])
type 1 i x: add(i, x)
type 2 l r: print prefix(r)-prefix(l-1)
```

Every `bit[j]` stores its lowbit block sum. Prefix blocks partition  $[1, r]$  and  $[1, l-1]$ , so their difference is exactly  $[l, r]$ . Build is  $O(n \log n)$  with these calls and each operation is  $O(\log n)$ ; memory is  $O(n)$ . In both C++ and Python, use 64-bit-capable sums. Test  $l=1$ ,  $l=r$ , negative delta, and a query after several additions to one position.

## 114.2 B. Inversion Count

**Observation.** Values can be large or negative, so compress them to ordered ranks. At index  $i$ , `i - prefix(rank)` is the number of earlier values that are strictly greater; using `prefix(rank-1)` would incorrectly count equals.

For  $[3, 1, 2, 5, 4]$ , the contributions are 0, 1, 1, 0, 1, total 3. The Fenwick invariant counts exactly the processed values of each rank, and compression preserves  $<$ , proving the accumulated contributions count every inversion once when its right endpoint arrives. Complexity is  $O(n \log n)$  time and  $O(n)$  memory. C++ needs `long long inv`; Python needs no special numeric type. Test sorted, reverse-sorted, all-equal, and values outside 32-bit range.

```
compress values to ranks
for i from 0 to n-1:
 answer += i - prefix(rank[a[i]])
 add(rank[a[i]], 1)
```

## 114.3 C. Range Add Point Query

**Observation.** Store `diff`, not the logical values. A range add changes only `diff[l]` and `diff[r+1]`; a point value is the prefix sum of differences. For  $[10, 20, 30, 40]$ , add 5 on  $[2, 3]$ : prefix at 3 changes from 30 to 35, but prefix at 4 returns to 40 because of `diff[4]=-5`.

The difference identity proves that every position in  $[1, r]$  sees  $+x$  once and every other position sees net zero. Each update and point query is  $O(\log n)$ , with  $O(n)$  memory. Allocate an extra usable index for  $r+1$ . In C++/Python, initialize by inserting  $a[i]-a[i-1]$ , not  $a[i]$ . Test a full-range add,  $r=n$ , and a negative update.

## 115 Stress-testing workflow

For A and C, keep a slow list and mutate it literally; for B, use a double loop on arrays of length at most eight. Compare after every random operation, then run `CP_TARGET=solution python3 sections/36_fenwick_trees/check.py`. Carry forward: Fenwick trees are prefix structures, and coordinate compression or a difference transform changes what that prefix means.

### 115.1 37. 1292C – Xenon’s Attack on the Gangs: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1292/C> Local package: [problems/37\\_xenons\\_attack](#)

### 115.2 Restatement and constraint implications

There are  $n$  desks, each with a non-negative capacity. A type 1  $i\ x$  operation replaces desk  $i$ ’s capacity by  $x$ . A type 2  $x$  operation must print the *smallest* desk index whose current capacity is at least  $x$ , or 0 if no such desk exists. There are up to 200,000 desks and operations, and capacities can be as large as  $10^9$ .

A direct scan from the first desk is easy to describe, but one query can then cost  $O(n)$ . In a sequence dominated by type-2 requests this becomes quadratic. We need a summary that can discard an entire interval as soon as it cannot contain a feasible desk.

### 115.3 Node invariant and trace

Store the **maximum capacity** in every tree node’s interval. The invariant is simple: `tree[v]` equals the maximum of all current capacities represented by node  $v$ . A node whose maximum is smaller than  $x$  contains no valid answer; a node whose maximum is at least  $x$  contains at least one valid answer.

For capacities  $[3, 0, 4, 2, 1]$ , a request for  $x=3$  starts at a root with maximum 4. Its left child covers  $[1,4]$  and has maximum 4, so the answer is in the left child. The next left child covers  $[1,2]$  and has maximum 3, so continue left. At the final split, position 1 has capacity 3, so it is the answer. A request for  $x=4$  first rejects  $[1,2]$  because its maximum is 3, then reaches position 3.

The important ordering rule is: inspect the left child first. Going right merely because both children are feasible would return some valid desk, but not necessarily the smallest one.

### 115.4 Algorithm

Build a power-of-two iterative segment tree using maximum as the merge. Pad leaves beyond  $n$  with  $-1$ ; all real capacities are non-negative and every request has  $x \geq 1$ , so padding cannot be selected.

```

assign(i, x):
 replace leaf i by x
 recompute maxima on the path to the root

first_at_least(x):
 if root maximum < x:
 return 0
 v = root
 while v is not a leaf:
 if maximum(left child of v) >= x:
 v = left child
 else:
 v = right child
 return index represented by v

```

## 115.5 Correctness proof

We first prove the node invariant. It is true for a leaf because its stored value is exactly that desk capacity. Every internal node is recomputed as the maximum of its two children, so if both child invariants are true, it equals the maximum capacity in their combined interval. Building bottom-up and recomputing every ancestor after an assignment therefore preserve the invariant.

For a request  $x$ , if the root maximum is below  $x$ , the invariant says every desk is below  $x$ , so returning zero is correct. Otherwise, at every internal node reached by the descent, its interval contains a qualifying desk. If the left child maximum is at least  $x$ , its interval contains one and every index there is smaller than every index in the right child, so descending left keeps the smallest possible answer. If it is below  $x$ , the left interval has no qualifying desk and the right interval must contain one, so descending right also keeps the smallest possible answer. At a leaf, that leaf is exactly the leftmost qualifying desk. Thus every printed answer is correct. QED.

## 115.6 Complexity, implementation notes, and tests

Building takes  $O(n)$ . Both an assignment and a search visit one root-to-leaf path, so they take  $O(\log n)$  time. The tree uses  $O(n)$  memory. Use `long long` in C++ for capacities even though the stated bound fits in an `int`; this makes the maximum state robust if the statement is later extended.

Test all capacities below a request, multiple equal feasible capacities, assignments that create a new earlier answer, and  $n=1$ . Do not use zero as padding: a request for zero would then select a fake leaf. The local contract uses requests at least one, but  $-1$  padding makes the intent unambiguous.

## 115.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

```

```

int n, q;
if (!(cin >> n >> q)) return 0;
int size = 1;
while (size < n) size <<= 1;

vector<long long> tree(2 * size, -1);
for (int i = 0; i < n; i++) cin >> tree[size + i];
for (int v = size - 1; v >= 1; v--) {
 tree[v] = max(tree[2 * v], tree[2 * v + 1]);
}

while (q--) {
 int type;
 cin >> type;
 if (type == 1) {
 int index;
 long long capacity;
 cin >> index >> capacity;
 int v = size + index - 1;
 tree[v] = capacity;
 for (v >>= 1; v > 0; v >>= 1) {
 tree[v] = max(tree[2 * v], tree[2 * v + 1]);
 }
 } else {
 long long need;
 cin >> need;
 if (tree[1] < need) {
 cout << 0 << '\n';
 continue;
 }
 int v = 1;
 while (v < size) {
 if (tree[2 * v] >= need) v = 2 * v;
 else v = 2 * v + 1;
 }
 cout << v - size + 1 << '\n';
 }
}
return 0;
}

```

## 115.8 Full Python solution

```

import sys

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n, q = data[0], data[1]

```



```

size = 1
while size < n:
 size *= 2

tree = [-1] * (2 * size)
tree[size : size + n] = data[2 : 2 + n]
for node in range(size - 1, 0, -1):
 tree[node] = max(tree[2 * node], tree[2 * node + 1])

pointer = 2 + n
output = []
for _ in range(q):
 operation = data[pointer]
 pointer += 1
 if operation == 1:
 index, capacity = data[pointer], data[pointer + 1]
 pointer += 2
 node = size + index - 1
 tree[node] = capacity
 node //= 2
 while node:
 tree[node] = max(tree[2 * node], tree[2 * node + 1])
 node //= 2
 else:
 need = data[pointer]
 pointer += 1
 if tree[1] < need:
 output.append("0")
 continue
 node = 1
 while node < size:
 if tree[2 * node] >= need:
 node *= 2
 else:
 node = 2 * node + 1
 output.append(str(node - size + 1))
print("\n".join(output))

if __name__ == "__main__":
 main()

```

## 116 Testing discussion

The fixed samples cover visible behavior. The original random tests exercise many short operation sequences. D additionally keeps a literal-array oracle: it scans from the first desk on tiny cases instead of invoking either reference implementation, then adds a 200,000-operation scale case with a directly known answer. The resulting expected files are used against both the C++ and Python submissions.

# 117 Problem-specific upsolve notes

## 117.1 A. Range Minimum Updates

**Constraint implication and trap.** Point assignments mean a sparse table is invalid after the first update; repeated direct range scans are too slow. Use `min` as the merge and `INF` as the padding identity for leaves beyond `n`.

**Trace and pseudocode.** In `[5,4,3,2,1]`, assign position 3 to 10. Rebuild only its ancestors; query `[2,4]` returns `min(4,10,2)=2`.

```
assign(p,x): set leaf p=x; recompute ancestors with min
query(l,r): merge the disjoint canonical nodes covering [l,r]
```

By induction from leaves, every node stores the minimum of its segment. The query nodes partition the requested interval, so their minimum is correct. Build is  $O(n)$ , each assignment/query is  $O(\log n)$ , memory  $O(n)$ . C++ uses `long long INF`; Python's `10**30` must exceed all inputs. Test singleton queries, updates at both ends, and all-positive values (which expose zero padding).

## 117.2 B. Maximum Subarray Updates

**Observation.** A node needs `(sum,pref,suff,best)`, all for non-empty subarrays. On merging `L` then `R`, a crossing answer is only `L.suff+R.pref`. For `[1,-2]` merged with `[3,4]`, candidates are left best 1, right best 7, crossing 6; the answer is 7.

```
merge(L,R):
 sum=L.sum+R.sum
 pref=max(L.pref,L.sum+R.pref)
 suff=max(R.suff,R.sum+L.suff)
 best=max(L.best,R.best,L.suff+R.pref)
```

Every non-empty subarray of the union lies fully left, fully right, or crosses the boundary, proving the formulas and root answer. A point assignment rebuilds one root path:  $O(\log n)$ ,  $O(n)$  memory. Use negative infinity in neutral padding, not zero. Test all-negative arrays, replacing the current best, and `n=1`.

## 117.3 C. Lazy Range Add Sum

**Invariant.** `seg[v]` already includes every update covering its entire interval; `lazy[v]` is the delta still owed to children. `apply(v,len,x)` adds `x*len` to its sum and adds `x` to its tag. On partial overlap, push before descending, recurse into intersecting children, then recompute the parent.

For `[1,2,3,4]`, adding 3 to `[1,4]` changes the root sum by 12 without visiting leaves. Query `[2,3]` first pushes the tag, then returns `5+6=11`. The invariant proves a fully covered node answers immediately, while pushing makes partial descendants correct. Build  $O(n)$ ; update/query  $O(\log n)$ ;

memory  $O(n)$ . Test a full update followed by partial query, overlapping updates, negative deltas, and an update ending at  $n$ .

## 117.4 D. First Available

**Invariant.** A node stores the maximum capacity in its interval. A request for  $x$  can discard any node whose maximum is below  $x$ ; if the root survives, always inspect the left child before the right child. For  $[3,0,4,2,1]$ , a request for 4 rejects the  $[1,2]$  interval and reaches index 3. This left-first descent is what turns an ordinary maximum tree into a leftmost-answer search.

```
if tree[1] < x: answer = 0
otherwise descend left when left.max >= x, else descend right
```

The maximum invariant proves rejection is safe and the left-first choice proves the leaf is the smallest feasible index. Both operations are  $O(\log n)$ . Test no feasible desk, ties, a new earlier capacity after an assignment, and the one-desk case. A literal array scan is an effective small random oracle.

## 118 Stress-testing workflow

Compare each operation to a literal list on short random cases. For B, recompute the answer with Kadane's algorithm after every assignment. Then run `CP_TARGET=solution python3 sections/37_segment_trees/check.py`. Carry forward: design the node state from the merge cases, and state the lazy-tag meaning before writing recursion.

### 118.1 38. 613D – Kingdom and its Cities: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/613/D> Local package: [problems/38\\_kingdom\\_cities](#)

### 118.2 Restatement

For each  $(v, k)$ , inspect  $v$ , its parent, and every ancestor through the  $k$ -th ancestor. Print their smallest label, or  $-1$  if that ancestor does not exist. This blind exercise looks like  $k$ th-ancestor queries, but it asks for an aggregate over the nodes crossed by the jumps.

### 118.3 Constraints and traps

$n, q \leq 200,000$ , so climbing one edge at a time is quadratic on a chain. Labels can be negative, so zero is not a valid missing-value sentinel. The path includes  $v$  itself: a query with  $k=0$  returns `label[v]`. A one-edge aggregate must include both  $v$  and its parent, not just the start node.

### 118.4 Observations

Keep the normal binary-lifting table:

```
up[t][v] = node exactly 2^t parent edges above v, or 0 if absent.
```

Also store an inclusive path aggregate whenever that jump exists:

```
best[t][v] = minimum label from v through up[t][v], inclusive.
```

For one edge:

```
best[0][v] = min(label[v], label[parent[v]]).
```

For a larger jump, let  $mid = up[t-1][v]$ . Two adjacent half-jumps form the whole path:

```
up[t][v] = up[t-1][mid]
best[t][v] = min(best[t-1][v], best[t-1][mid])
```

To answer  $(v, k)$ , start  $\text{answer} = \text{label}[v]$ . Decompose  $k$  into set bits. For each selected bit  $t$ , reject if  $\text{up}[t][v]$  is zero; otherwise merge  $\text{best}[t][v]$  and move  $v$  to  $\text{up}[t][v]$ . Consecutive segments may repeat an endpoint, but minimum is idempotent, so that overlap changes nothing.

## 118.5 Sample walk-through

For  $(5, 2)$ , the path is  $5 \rightarrow 2 \rightarrow 1$  with labels  $6, 5, 8$ . Bit one is set, so take a two-edge jump:

```
best[1][5] = min(best[0][5], best[0][2])
 = min(min(6, 5), min(5, 8)) = 5.
```

The answer is 5. For  $(4, 3)$ , the third parent edge would leave the root, so the answer is  $-1$ , not the minimum on the shorter existing prefix.

## 118.6 Algorithm

```
build up[0][v] = parent[v]
build best[0][v] = min(label[v], label[parent[v]])

for t = 1..LOG-1:
 mid = up[t-1][v]
 up[t][v] = up[t-1][mid]
 best[t][v] = min(best[t-1][v], best[t-1][mid])

for query (v, k):
 answer = label[v]
 for every set bit t of k:
 if up[t][v] is zero: print -1
 otherwise merge best[t][v], then v = up[t][v]
 print answer if all selected jumps existed
```

Use a large positive INF for aggregate entries involving sentinel node zero. The query checks the jump before using its aggregate, so INF never becomes a real answer.

## 118.7 Correctness proof

**Lemma 1.** If  $\text{up}[t][v]$  exists, it is the  $2^t$ -th ancestor of  $v$ , and  $\text{best}[t][v]$  is the minimum on the inclusive path between them.

**Proof.** At  $t=0$ , the table stores the parent and the minimum of its two endpoint labels. For larger  $t$ , the recurrence joins the path from  $v$  to  $\text{mid}$  and the adjacent path from  $\text{mid}$  to  $\text{up}[t-1][\text{mid}]$ . By induction both half aggregates are correct; their minimum is exactly the aggregate on the combined  $2^t$ -edge path.  $\square$

**Lemma 2.** After processing selected query bits totaling  $d$ , the current node is the  $d$ -th ancestor of the original node and **answer** is the minimum on the path to it.

**Proof.** It holds initially for  $d=0$ . Lemma 1 moves the node by each selected power-of-two distance and merges exactly the labels crossed. Induction over the bit loop proves the claim.  $\square$

**Theorem.** The algorithm prints -1 exactly for impossible queries and the requested path minimum for all other queries.

**Proof.** A missing selected jump proves that the requested ancestor is absent. Otherwise the selected powers sum to  $k$ , and Lemma 2 gives the minimum from the starting node through its  $k$ -th ancestor.  $\square$

## 118.8 Complexity

With  $L = \text{ceil}(\log_2(n+1))$ :

```
Preprocessing: $O(n L)$ time and memory
Each query: $O(L)$ time
```

## 118.9 C++ implementation notes

Use zero as the missing-parent sentinel and initialize `minimum[*][0]` to INF. Keep labels and table aggregates in `long long`. Before merging a selected jump, test `up[bit][node] != 0`; this distinguishes an impossible query from a valid path with a negative answer.

## 118.10 Python implementation notes

Use `10**30` as INF and an output list for all query answers. The loop that consumes  $k$  must advance its bit counter for both zero and one bits.

## 118.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, q;
 if (!(cin >> n >> q)) return 0;
 int log = 1;
 while ((1 << log) <= n) ++log;

 const long long INF = (1LL << 62);
 vector<long long> label(n + 1);
 for (int node = 1; node <= n; ++node) cin >> label[node];
```

```

vector<vector<int>> up(log, vector<int>(n + 1, 0));
vector<vector<long long>> minimum(log, vector<long long>(n + 1, INF));
for (int node = 2; node <= n; ++node) cin >> up[0][node];
for (int node = 1; node <= n; ++node) {
 minimum[0][node] = min(label[node], up[0][node] ? label[up[0][node]] : INF);
}

for (int bit = 1; bit < log; ++bit) {
 for (int node = 1; node <= n; ++node) {
 int middle = up[bit - 1][node];
 up[bit][node] = up[bit - 1][middle];
 minimum[bit][node] = min(minimum[bit - 1][node], minimum[bit - 1][middle]);
 }
}

while (q--) {
 int node;
 long long distance;
 cin >> node >> distance;
 long long answer = label[node];
 bool exists = true;
 for (int bit = 0; distance; ++bit, distance >>= 1LL) {
 if ((distance & 1LL) == 0) continue;
 if (bit >= log || up[bit][node] == 0) {
 exists = false;
 break;
 }
 answer = min(answer, minimum[bit][node]);
 node = up[bit][node];
 }
 cout << (exists ? answer : -1) << '\n';
}
return 0;
}

```

## 118.12 Full Python solution

```

import sys

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n, q = data[0], data[1]
 labels = [0] + data[2:2 + n]
 index = 2 + n
 log = max(1, n.bit_length())
 infinity = 10**30
 up = [[0] * (n + 1) for _ in range(log)]

```



```

minimum = [[infinity] * (n + 1) for _ in range(log)]
for node in range(2, n + 1):
 up[0][node] = data[index]
 index += 1
for node in range(1, n + 1):
 minimum[0][node] = min(labels[node], labels[up[0][node]] if up[0][node] else infinity)

for bit in range(1, log):
 previous_up = up[bit - 1]
 previous_minimum = minimum[bit - 1]
 for node in range(1, n + 1):
 middle = previous_up[node]
 up[bit][node] = previous_up[middle]
 minimum[bit][node] = min(previous_minimum[node], previous_minimum[middle])

output: list[str] = []
for _ in range(q):
 node = data[index]
 distance = data[index + 1]
 index += 2
 answer = labels[node]
 exists = True
 bit = 0
 while distance:
 if distance & 1:
 if bit >= log or up[bit][node] == 0:
 exists = False
 break
 answer = min(answer, minimum[bit][node])
 node = up[bit][node]
 distance >>= 1
 bit += 1
 output.append(str(answer if exists else -1))
sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__":
 main()

```

## 118.13 Common wrong answers

Storing only label[v] in best[0][v] and omitting the parent endpoint.  
 Returning a partial-path minimum after discovering a missing ancestor.  
 Using zero as the missing aggregate despite negative labels.  
 Forgetting that k=0 includes the starting node.  
 Choosing too few lifting rows for a deepest valid path.

## 118.14 Tests that matter

A one-node tree with  $k=0$  and positive  $k$ .

Negative labels along a path.

A query that reaches exactly the root and one edge beyond it.

A long chain with 200,000 queries, to reject literal climbing.

## 119 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

# 120 Problem-specific upsolve notes

## 120.1 A. Static Range Minimum

**Constraint implication and trap.** The array never changes, so pay  $O(n \log n)$  once for  $O(1)$  queries. Do not use this overlapping two-block query trick for sum: it is valid here only because min is idempotent.

For  $[5, 2, 7, 1, 3, 4]$ , query  $[2, 5]$  has length 4 and reads one  $2^2$  block; query  $[2, 6]$  uses blocks  $[2, 5]$  and  $[3, 6]$ , whose overlap cannot alter a minimum. Pseudocode is: build  $st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}])$ ; for  $[l, r]$ , set  $k = \lg[r-l+1]$  and take the min of the two length- $2^k$  blocks. Induction proves each table cell; the two blocks cover the query, proving the answer. Preprocessing/memory  $O(n \log n)$ , query  $O(1)$ . Test singleton and non-power-of-two ranges; convert input indices exactly once.

## 120.2 B. Static Range GCD

Use the identical table shape with gcd as merge. For  $[12, 18, 6, 10, 15]$ , query  $[2, 5]$  has gcd 1. Gcd is associative and idempotent, so the same overlapping-cover proof applies. Complexity is again  $O(n \log n)$  build and memory,  $O(1)$  per query. C++ uses `std::gcd`; Python uses `math.gcd`. Test a length-one range, equal values, and a range whose answer becomes one early.

## 120.3 C. Kth Ancestor

**Observation and trace.** Let  $up[k][v]$  be the exact  $2^k$ -th ancestor, with zero meaning absent. Build by composing two half jumps. To lift 4 by three in the sample tree, take bit 0 to reach 2, then bit 1 to reach zero, so output -1; for distance zero output 4 unchanged.

```
build up[0][v]=parent[v]
up[k][v]=up[k-1][up[k-1][v]]
for each set bit k of distance: v=up[k][v]
answer -1 if v=0 else v
```

Induction on  $k$  proves the table's jump meaning; binary expansion uses exactly the requested number of parent edges, proving each query. Build/memory are  $O(n \log n)$  and a query is  $O(\log n)$ . Choose LOG for the largest possible query distance, not just an arbitrary constant. Test root, zero distance, one-step, and a distance above depth.

## 121 Stress-testing workflow

Compare sparse-table answers to direct scans on small arrays. For ancestor queries, climb the parent array literally. Run `CP_TARGET=solution python3 sections/38_static_queries_binary_lifting/check.py`. Carry forward: overlap needs idempotence; binary lifting is the same power-of-two decomposition applied to functional-graph/tree jumps.

### 121.1 39. 1110F – Nearest Leaf: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1110/F> Local package: [problems/39\\_nearest\\_leaf](#)

### 121.2 Restatement and traps

Repeatedly merge two current files, paying their combined size, until one file remains. Minimize total cost. With  $n \leq 200000$ , trying merge orders is exponential; totals require `long long`. A one-file input costs zero.

### 121.3 Observation and trace

Always merge the two smallest current sizes. For 4,3,2,6, merge  $2+3=5$ , then  $4+5=9$ , then  $6+9=15$ , totaling 29.

View any merge plan as a binary tree whose leaves are original files. A file size is charged once for every ancestor merge, so smaller sizes should be at least as deep as larger sizes. Thus some optimal tree makes the two smallest files siblings at maximum depth; merge them first and recurse on their sum.

### 121.4 Algorithm

```
put every size in a min-heap
while heap has at least two values:
 pop a and b
 answer += a+b
 push a+b
print answer
```

## 121.5 Correctness proof

**Lemma.** There is an optimal merge plan whose first merge uses two smallest current sizes.

**Proof.** In an optimal merge tree, choose two sibling leaves at greatest depth. Swapping either of their leaf sizes with a no-larger smallest size can only decrease the weighted depth sum, so the two smallest sizes can occupy those sibling leaves without worsening the plan. Contracting that sibling pair is exactly merging them first.  $\square$

**Theorem.** The heap algorithm has minimum total cost.

**Proof.** By the lemma, its first merge is compatible with an optimal plan. After replacing the pair by their sum, the remaining problem has the same form. Induction on the number of files proves every greedy merge and therefore the total is optimal.  $\square$

## 121.6 Complexity

$n-1$  heap merges cost  $O(n \log n)$  time and  $O(n)$  memory.

## 121.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>>n))return 0;priority_q
```

## 121.8 Full Python solution

```
import heapq, sys
def main() -> None:
 data=list(map(int,sys.stdin.buffer.read().split()));q=data[1:];heapq.heapify(q);answer=0
 while len(q)>1:
 total=heapq.heappop(q)+heapq.heappop(q);answer+=total;heapq.heappush(q,total)
 print(answer)
if __name__ == "__main__": main()
```

## 121.9 Implementation notes and tests

Use a min-heap (`greater<long long>` in C++, `heapq` in Python). Do not sort once: merged sums must re-enter the current frontier. Test one file, equal sizes, a very large size beside small ones, and totals beyond 32-bit range.

## 121.10 Stress testing

For at most seven files, recursively try every pair of current files and take the least cost; this is independent of the greedy rule. The scale generator uses 200000 ones and a heap-computed expected total to exercise heap work and 64-bit accumulation.

## 122 What to carry forward

A heap represents the next useful frontier, not an arbitrary bag of values. State exactly what each heap item means, prove that its top is globally next, and check the boundary that removes or advances a frontier item.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

### 122.1 40. 587C – Duff in the Army: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/587/C> Local package: [problems/40\\_duff\\_in\\_the\\_army](#)

### 122.2 Restatement and constraint implications

There are  $n$  employees with current integer salaries. An operation  $! k x$  changes employee  $k$ 's salary to  $x$ . A query  $? a b$  asks how many current salaries lie in the inclusive *value* interval  $[a, b]$ . There are up to 200,000 employees and operations, and salaries can be negative or as large as  $10^9$  in magnitude.

The interval is about salary values, not employee indices. Scanning all employees per query is quadratic. A query needs only the number of occurrences of each ordered salary value, which is exactly what a Fenwick tree can store after coordinate compression.

### 122.3 Coordinate collection and trace

Read and retain all operations first. Collect every initial salary and every new  $x$  in an update, sort the distinct values, and give each one a one-based rank. This preserves order: for collected salaries  $u$  and  $v$ ,  $u < v$  if and only if  $\text{rank}(u) < \text{rank}(v)$ .

For initial salaries  $[10, 20, 30, 40, 50]$ , query  $? 15 45$  has four salaries at most 45 and one salary below 15, so its answer is  $4 - 1 = 3$ . After  $! 2 35$ , remove one count at rank 20 and add one at rank 35. Then  $? 30 35$  returns two. The bounds 15 and 45 need not be coordinates: binary search locates where they fall among actual salary values.

### 122.4 Algorithm

```
collect initial salaries and every update value; sort and deduplicate them
insert all current salary ranks into a Fenwick frequency tree
```

```
for each operation:
 if ! k x:
 remove the rank of current[k]
 set current[k] = x and add its rank
 else ? a b:
 right = number of coordinate values <= b
 left = number of coordinate values < a
 print BIT.prefix(right) - BIT.prefix(left)
```

Use `upper_bound(coords,b)/bisect_right(coords,b)` for `right`, and `lower_bound(coords,a)/bisect_left(coords,a)` for `left`.

## 122.5 Correctness proof

The Fenwick invariant is: the frequency at each rank equals the number of employees currently earning that corresponding salary. Initialization adds each employee once. An update removes exactly its old salary and adds exactly its new one, so the invariant remains true.

For `[a,b]`, `right` covers exactly salaries at most `b`, while `left` covers exactly salaries below `a`. The two Fenwick prefix sums therefore count those two employee sets under the invariant. Their difference counts exactly the employees with `a <= salary <= b`, so every query answer is correct. QED.

## 122.6 Complexity, implementation notes, and tests

Sorting coordinates costs  $O((n+q) \log(n+q))$ . Each update/query uses a constant number of binary searches and Fenwick operations, hence  $O(\log(n+q))$ ; memory is  $O(n+q)$ . Use `long long` for C++ salary values.

Test an update below every old salary, equal salaries, query endpoints absent from coordinates, and an empty value range. Common mistakes are using a lower bound for the inclusive upper endpoint, or adding the new salary without first removing the old one.

## 122.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
 vector<int> bit;
 explicit Fenwick(int n) : bit(n + 1, 0) {}

 void add(int index, int delta) {
 for (int n = (int)bit.size(); index < n; index += index & -index) {
 bit[index] += delta;
 }
 }
}
```



```

 int sum(int index) const {
 int result = 0;
 for (; index > 0; index -= index & -index) result += bit[index];
 return result;
 }
};

struct Operation {
 char type;
 int first;
 long long second;
};

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, q;
 if (!(cin >> n >> q)) return 0;
 vector<long long> salary(n + 1), coords;
 for (int i = 1; i <= n; i++) {
 cin >> salary[i];
 coords.push_back(salary[i]);
 }

 vector<Operation> operations;
 operations.reserve(q);
 for (int i = 0; i < q; i++) {
 char type;
 long long first, second;
 cin >> type >> first >> second;
 operations.push_back({type, (int)first, second});
 if (type == '!') coords.push_back(second);
 }

 sort(coords.begin(), coords.end());
 coords.erase(unique(coords.begin(), coords.end()), coords.end());
 auto rank_of = [&](long long value) {
 return int(lower_bound(coords.begin(), coords.end(), value) - coords.begin()) + 1;
 };

 Fenwick bit((int)coords.size());
 for (int i = 1; i <= n; i++) bit.add(rank_of(salary[i]), 1);

 for (const Operation& operation : operations) {
 if (operation.type == '!') {
 int employee = operation.first;
 bit.add(rank_of(salary[employee]), -1);
 salary[employee] = operation.second;
 bit.add(rank_of(salary[employee]), 1);
 } else {
 long long low = operation.first;

```

```

 long long high = operation.second;
 int before_low = int(lower_bound(coords.begin(), coords.end(), low) - coords.begin());
 int through_high = int(upper_bound(coords.begin(), coords.end(), high) - coords.begin());
 cout << bit.sum(through_high) - bit.sum(before_low) << '\n';
 }
}
return 0;
}

```

## 122.8 Full Python solution

```

import bisect
import sys

class Fenwick:
 def __init__(self, size: int) -> None:
 self.bit = [0] * (size + 1)

 def add(self, index: int, delta: int) -> None:
 while index < len(self.bit):
 self.bit[index] += delta
 index += index & -index

 def prefix_sum(self, index: int) -> int:
 total = 0
 while index:
 total += self.bit[index]
 index -= index & -index
 return total

def main() -> None:
 tokens = sys.stdin.buffer.read().split()
 if not tokens:
 return
 pointer = 0
 n = int(tokens[pointer])
 q = int(tokens[pointer + 1])
 pointer += 2
 salary = [0] + [int(value) for value in tokens[pointer : pointer + n]]
 pointer += n

 operations: list[tuple[bytes, int, int]] = []
 coords = salary[1:]
 for _ in range(q):
 kind = tokens[pointer]
 first = int(tokens[pointer + 1])
 second = int(tokens[pointer + 2])
 pointer += 3
 operations.append((kind, first, second))

```

```

 if kind == b"!":
 coords.append(second)

coords = sorted(set(coords))
bit = Fenwick(len(coords))

def rank_of(value: int) -> int:
 return bisect.bisect_left(coords, value) + 1

for value in salary[1:]:
 bit.add(rank_of(value), 1)

output: list[str] = []
for kind, first, second in operations:
 if kind == b"!":
 employee, new_salary = first, second
 bit.add(rank_of(salary[employee]), -1)
 salary[employee] = new_salary
 bit.add(rank_of(new_salary), 1)
 else:
 low, high = first, second
 before_low = bisect.bisect_left(coords, low)
 through_high = bisect.bisect_right(coords, high)
 output.append(str(bit.prefix_sum(through_high) - bit.prefix_sum(before_low)))
print("\n".join(output))

if __name__ == "__main__":
 main()

```

## 123 Testing discussion

The small D cases keep a literal salary list and answer queries by scanning it; they never invoke a reference solution. The scale case combines 100,000 updates with 100,000 range-count queries, so a linear scan cannot pass. The expected outputs are shared by the C++ and Python runners.

# 124 Detailed upsolve workshop

## 124.1 A. Hotel Queries

**Constraints and trap.**  $n, q$  are up to about  $2e5$ . A linear scan for each group is too slow; a plain maximum alone finds feasibility but not the first index. Store the maximum remaining capacity in each segment-tree node.

**Trace and pseudocode.** With capacities  $[3, 1, 4, 1, 5]$ , a request of four descends root  $\rightarrow$  left/right according to whether the left subtree maximum is at least four, reaches index three, and changes it to zero.

```
if root.max < x: output 0
otherwise descend: go left if left.max >= x, else right
at leaf i: subtract x; recompute max values on the path to root; output i
```

**Proof and complexity.** Every node is the maximum of exactly its leaves. A feasible left child contains an earlier valid hotel; an infeasible left child contains none, so the descent returns the first valid leaf. Each query changes and rebuilds one root-to-leaf path:  $O(\log n)$  time,  $O(n)$  memory. Use long long capacities. Test no feasible hotel, the first/last feasible hotel, and repeated requests to one hotel.

## 124.2 B. List Removals

**Constraints and trap.** Removing an element from an array/vector shifts all following values and becomes quadratic. Instead, original positions are fixed; a Fenwick tree stores one for alive positions and zero for removed positions.

**Trace and pseudocode.** For values  $[10, 20, 30, 40, 50]$ , rank two resolves to original index two, outputs 20, then adds -1. With counts  $[1, 0, 1, 1, 1]$ , rank three resolves to original index four, outputting 40.

```
initialize BIT with one at every index
for rank k:
 i = first index whose BIT prefix sum >= k
 output value[i]; BIT.add(i, -1)
```

**Proof and complexity.** The BIT prefix sum is the number of alive original positions at or before its index. Therefore its first prefix reaching  $k$  is the  $k$ -th current item. Fenwick binary lifting gives  $O(\log n)$  per removal and  $O(n)$  memory. The search uses one-based indices; test rank one, last rank, and duplicate values. A small mutable-list simulation is the stress oracle.

## 124.3 C. Mo Distinct Queries

**Constraints and trap.** Static array, many offline range queries, and no updates signal Mo's algorithm. Recomputing a set for every range is quadratic in the worst case. Coordinate-compress values before using an array frequency.

**Trace and pseudocode.** For  $[1,2,1,3,2,4]$ , activate  $[1,3]$  to obtain frequencies  $1:2, 2:1$  and answer two. Move to  $[2,5]$ : remove the first one, add 3 and 2, and obtain three.

```
sort queries by left block, alternating right direction
move [curL,curR] one endpoint at a time
add(pos): if freq[a[pos]] changes 0->1, distinct++
remove(pos): if freq[a[pos]] changes 1->0, distinct--
```

**Proof and complexity.** The frequency invariant exactly represents the current window, so `distinct` is correct when a query is reached. Block order limits endpoint movement to  $O((n+q)\sqrt{n})$  in the standard analysis; memory is  $O(n+q)$ . Convert endpoints once, use buffered I/O in Python, and test all equal values, all distinct values, singletons, and repeated ranges against `len(set(...))` on small random arrays.

## 125 What to carry forward

Choose a stored summary that answers the decision you actually need: subtree maximum for first-fit, alive prefix count for dynamic rank, and frequencies for an offline window. The invariant makes both the data structure and its proof mechanical.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

### 125.1 41. 786C – Till I Collapse: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/786/C> Local package: [problems/41\\_till\\_i\\_collapse](#)

### 125.2 Restatement

Each edge has a signed weight. For every pair of nodes, print the sum of the weights on their unique tree path. This blind exercise requires the same LCA primitive as unweighted distance, but the depth count alone is no longer the quantity to combine.

### 125.3 Constraints and traps

With  $n, q \leq 200,000$ , walking a path per query is too slow on a chain. A path sum can reach about  $2 * 10^{14}$ , so use `long long` in C++. Edge weights may be negative: do not use a shortest-path interpretation, and do not initialize a path aggregate to zero except for the empty path.

### 125.4 Observations

Root the tree at node 1. Let `root_sum[v]` be the sum of edge weights on the root-to-`v` path. It is available while reading this parent-before-child input:

```
root_sum[v] = root_sum[parent[v]] + edge_weight[v].
```

For  $c = \text{lca}(u, v)$ , the two root paths share exactly the root-to- $c$  prefix. Adding the two root sums counts that shared prefix twice, so remove it twice:

```
path_sum(u, v) = root_sum[u] + root_sum[v] - 2*root_sum[c].
```

The LCA table is the ordinary `up[t][v]` table. No edge aggregate table is needed because the prefix sums already summarize every root path.

## 125.5 Sample walk-through

For query (3,4),  $\text{lca}(3,4)=1$ . The root sums are 2 for node 3 and 16 for node 4, while the root sum at node 1 is zero:

$$2 + 16 - 2 \cdot 0 = 18.$$

For (2,2), the LCA is node 2, so its root sum is added twice and subtracted twice. The result is zero, as the empty path should be.

## 125.6 Algorithm

```
read parent and edge weight for every node
compute depth[v], root_sum[v], and up[0][v]
build the binary-lifting table

for each query (u,v):
 c = lca(u,v)
 print root_sum[u] + root_sum[v] - 2*root_sum[c]
```

The LCA routine first equalizes depths, then attempts simultaneous jumps from largest power to smallest while the proposed ancestors differ.

## 125.7 Correctness proof

**Lemma 1.**  $\text{root\_sum}[v]$  equals the sum of weights on the root-to- $v$  path.

**Proof.** The root has sum zero. Every other node appends exactly its parent edge to the already-correct parent path, so induction on input order proves the recurrence.  $\square$

**Lemma 2.** The binary-lifting routine returns  $\text{lca}(u,v)$ .

**Proof.** Each table entry is an exact power-of-two ancestor by induction. After depth equalization, simultaneous largest-first jumps keep both nodes below their LCA. When no such jump remains, they are its children, so their common parent is the LCA.  $\square$

**Theorem.** The algorithm prints the path-weight sum for every query.

**Proof.** By Lemma 2,  $c$  is the unique meeting point of the two root paths. By Lemma 1,  $\text{root\_sum}[u] + \text{root\_sum}[v]$  contains every edge on the  $u$ - $v$  path once and every edge on root-to- $c$  twice. Subtracting  $2 \cdot \text{root\_sum}[c]$  removes exactly that duplicated prefix.  $\square$

## 125.8 Complexity

```
Preprocessing: $O(n \log n)$ time and memory
Each query: $O(\log n)$ time
```



## 125.9 C++ implementation notes

Use `long long` for `root_sum` and the final expression. The `up` entries and depths remain ordinary integers. The parent-before-child input order means no recursive DFS is needed, avoiding recursion-depth issues on a chain.

## 125.10 Python implementation notes

Python integers naturally cover sums. Parse the input once, build output lines in a list, and use an iterative lifting loop; no recursion is needed.

## 125.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 ios::sync_with_stdio(false); cin.tie(nullptr);
 int n, q; if (!(cin >> n >> q)) return 0;
 int log = 1; while ((1 << log) <= n) ++log;
 vector<vector<int>> up(log, vector<int>(n + 1));
 vector<int> depth(n + 1); vector<long long> root_sum(n + 1);
 for (int v = 2; v <= n; ++v) { long long w; cin >> up[0][v] >> w; depth[v] = depth[up[0][v]] + w;
 for (int k = 1; k < log; ++k) for (int v = 1; v <= n; ++v) up[k][v] = up[k-1][up[k-1][v]];
 auto lift = [&](int v, int d) { for (int k = 0; k < log; ++k) if (d & (1 << k)) v = up[k][v];
 auto lca = [&](int a, int b) { if (depth[a] < depth[b]) swap(a, b); a = lift(a, depth[a] - depth[b]);
 while (q--) { int u, v; cin >> u >> v; int c = lca(u, v); cout << root_sum[u] + root_sum[v] << '\n';
}
```

## 125.12 Full Python solution

```
import sys

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data: return
 n, q = data[0], data[1]; log = max(1, n.bit_length()); index = 2
 up = [[0] * (n + 1) for _ in range(log)]; depth = [0] * (n + 1); root_sum = [0] * (n + 1)
 for node in range(2, n + 1):
 parent, weight = data[index], data[index + 1]; index += 2
 up[0][node] = parent; depth[node] = depth[parent] + 1; root_sum[node] = root_sum[parent] + weight
 for bit in range(1, log):
 for node in range(1, n + 1): up[bit][node] = up[bit - 1][up[bit - 1][node]]
 def lift(node: int, distance: int) -> int:
 for bit in range(log):
 if distance >> bit & 1: node = up[bit][node]
 return node

 for _ in range(q):
 u, v = data[index], data[index + 1]; index += 2
 c = lca(u, v)
 print(root_sum[u] + root_sum[v] - root_sum[c] * 2)
```

```

def lca(left: int, right: int) -> int:
 if depth[left] < depth[right]: left, right = right, left
 left = lift(left, depth[left] - depth[right])
 if left == right: return left
 for bit in range(log - 1, -1, -1):
 if up[bit][left] != up[bit][right]: left, right = up[bit][left], up[bit][right]
 return up[0][left]
output = []
for _ in range(q):
 left, right = data[index], data[index + 1]; index += 2; common = lca(left, right)
 output.append(str(root_sum[left] + root_sum[right] - 2 * root_sum[common]))
sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__": main()

```

## 125.13 Common wrong answers

Using edge count depth in place of weighted root sums.  
 Subtracting root\_sum[lca] once instead of twice.  
 Using int for a long chain of large weights.  
 Treating negative weights as an invalid input or shortest-path issue.  
 Forgetting that a query from a node to itself has answer zero.

## 125.14 Tests that matter

u=v, including the root.  
 An ancestor-descendant query.  
 Negative edges on one branch.  
 Two branches whose LCA is not the root.  
 A 200,000-node chain with 200,000 endpoint queries.

## 126 Testing discussion

Run `CP_TARGET=solution python3 sections/41_euler_tour_lca_binary_lifting/check.py`. For random trees of at most 20 vertices, build a path by parent walks and use it as an oracle for all three exercises. This catches depth, aggregate-order, and one-based-rank mistakes more reliably than samples.

## 127 What to carry forward

Binary lifting turns an ancestor relation into a powers-of-two table. Once the exact jump invariant is established, LCA, path distance, path aggregates, and path indexing are small variations on the same controlled lifts.

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

### 127.1 42. 1093G – Multidimensional Queries: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1093/G> Local package: [problems/42\\_multidimensional\\_queries](#)

### 127.2 Restatement and constraints

For every one of  $n$  closed intervals  $[l_i, r_i]$ , count two quantities:

1. how many *other* intervals it contains; and
2. how many *other* intervals contain it.

Interval  $i$  contains interval  $j$  exactly when  $l_i \leq l_j$  and  $r_j \leq r_i$ . Identical intervals therefore contain each other. With up to 200,000 intervals, comparing all ordered pairs is quadratic.

### 127.3 Ordering and worked trace

Sort interval records by (left ascending, right descending). The descending tie-break is essential: among intervals that begin together, a larger right endpoint can contain a smaller one.

For intervals

```
0: [1,6]
1: [2,5]
2: [2,5]
3: [3,4]
```

the contains counts are  $[3,2,2,0]$ : interval 0 contains all other intervals; each copy of  $[2,5]$  contains the other copy and  $[3,4]$ . The contained-by counts are  $[0,2,2,3]$ .

Compress right endpoints. To count **contained-by**, sweep the sorted list left to right. Earlier records have a left endpoint no larger than the current one. Among them, the number with right endpoint at least  $r$  is `processed - BIT.prefix(rank(r)-1)`.

To count **contains**, sweep the same ordering right to left. Already processed records have a left endpoint no smaller than the current one. The number with right endpoint at most  $r$  is `BIT.prefix(rank(r))`.

Equal pairs such as the two [2,5] records must be processed as a group in both sweeps. Compute the common answer against previously processed intervals, then add `group_size-1` because every duplicate contains, and is contained by, every other duplicate. Only then insert the whole group into the Fenwick tree.

## 127.4 Algorithm

```
sort records by (left increasing, right decreasing)
compress all right endpoints

for each equal-pair group from left to right:
 base = processed - BIT.prefix(rank(right) - 1)
 contained_by[index] = base + group_size - 1 for every index in group
 insert group_size copies of rank(right) into BIT

clear BIT
for each equal-pair group from right to left:
 base = BIT.prefix(rank(right))
 contains[index] = base + group_size - 1 for every index in group
 insert group_size copies of rank(right) into BIT
```

## 127.5 Correctness proof

Consider the forward sweep. Before a group is processed, the ordering ensures every inserted interval has left endpoint at most the group's left endpoint. The Fenwick tree counts their right endpoints. Subtracting the count strictly below the group's right endpoint from the number processed leaves exactly those with right endpoint at least it, which are exactly the previously inserted intervals containing this group. Equal-pair additions count every other identical interval, and delaying their insertion prevents double counting. Thus `contained_by` is correct.

For the reverse sweep, every inserted interval has left endpoint at least the current group's left endpoint. The Fenwick prefix through its right endpoint counts exactly those whose right endpoint is at most it, which are exactly the inserted intervals contained by this group. The same equal-pair argument adds the identical intervals exactly once. Thus `contains` is correct. QED.

## 127.6 Complexity, implementation notes, and tests

Sorting/compression cost  $O(n \log n)$ , and each Fenwick operation costs  $O(\log n)$ , so total time is  $O(n \log n)$  with  $O(n)$  memory. Keep original indices in the sorted records so answers can be printed in input order.

Test identical intervals, intervals sharing only one endpoint, disjoint intervals, and a fully nested chain. The common error is to update a duplicate immediately; that makes the first duplicate see none of its peers while later duplicates see too many.

## 127.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
 vector<int> bit;
 explicit Fenwick(int n) : bit(n + 1, 0) {}

 void add(int index, int delta) {
 for (int n = (int)bit.size(); index < n; index += index & -index) {
 bit[index] += delta;
 }
 }

 int sum(int index) const {
 int result = 0;
 for (; index > 0; index -= index & -index) result += bit[index];
 return result;
 }
};

struct Interval {
 long long left, right;
 int index;
};

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 if (!(cin >> n)) return 0;
 vector<Interval> ranges(n);
 vector<long long> rights;
 for (int i = 0; i < n; i++) {
 cin >> ranges[i].left >> ranges[i].right;
 ranges[i].index = i;
 rights.push_back(ranges[i].right);
 }
 sort(rights.begin(), rights.end());
 rights.erase(unique(rights.begin(), rights.end()), rights.end());
 sort(ranges.begin(), ranges.end(), [](const Interval& a, const Interval& b) {
 if (a.left != b.left) return a.left < b.left;
 return a.right > b.right;
 });
 auto rank_of = [&](long long right) {
 return int(lower_bound(rights.begin(), rights.end(), right) - rights.begin()) + 1;
 };

 vector<int> contains(n), contained_by(n);
 Fenwick bit((int)rights.size());
```

```

int processed = 0;
for (int start = 0; start < n;) {
 int end = start + 1;
 while (end < n && ranges[end].left == ranges[start].left &&
 ranges[end].right == ranges[start].right) {
 end++;
 }
 int size = end - start;
 int rank = rank_of(ranges[start].right);
 int base = processed - bit.sum(rank - 1);
 for (int i = start; i < end; i++) contained_by[ranges[i].index] = base + size - 1;
 bit.add(rank, size);
 processed += size;
 start = end;
}

bit = Fenwick((int)rights.size());
for (int end = n; end > 0;) {
 int start = end - 1;
 while (start > 0 && ranges[start - 1].left == ranges[end - 1].left &&
 ranges[start - 1].right == ranges[end - 1].right) {
 start--;
 }
 int size = end - start;
 int rank = rank_of(ranges[start].right);
 int base = bit.sum(rank);
 for (int i = start; i < end; i++) contains[ranges[i].index] = base + size - 1;
 bit.add(rank, size);
 end = start;
}

for (int i = 0; i < n; i++) cout << contains[i] << (i + 1 == n ? '\n' : ' ');
for (int i = 0; i < n; i++) cout << contained_by[i] << (i + 1 == n ? '\n' : ' ');
return 0;
}

```

## 127.8 Full Python solution

```

import bisect
import sys

class Fenwick:
 def __init__(self, size: int) -> None:
 self.bit = [0] * (size + 1)

 def add(self, index: int, delta: int) -> None:
 while index < len(self.bit):
 self.bit[index] += delta
 index += index & -index

```

```

def prefix_sum(self, index: int) -> int:
 total = 0
 while index:
 total += self.bit[index]
 index -= index & -index
 return total

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n = data[0]
 ranges = [(data[2 * i + 1], data[2 * i + 2], i) for i in range(n)]
 rights = sorted({right for _, right, _ in ranges})
 ranges.sort(key=lambda interval: (interval[0], -interval[1]))

 def rank_of(right: int) -> int:
 return bisect.bisect_left(rights, right) + 1

 contains = [0] * n
 contained_by = [0] * n

 bit = Fenwick(len(rights))
 processed = 0
 start = 0
 while start < n:
 end = start + 1
 while end < n and ranges[end][:2] == ranges[start][:2]:
 end += 1
 group_size = end - start
 rank = rank_of(ranges[start][1])
 base = processed - bit.prefix_sum(rank - 1)
 for _, _, index in ranges[start:end]:
 contained_by[index] = base + group_size - 1
 bit.add(rank, group_size)
 processed += group_size
 start = end

 bit = Fenwick(len(rights))
 end = n
 while end:
 start = end - 1
 while start > 0 and ranges[start - 1][:2] == ranges[end - 1][:2]:
 start -= 1
 group_size = end - start
 rank = rank_of(ranges[start][1])
 base = bit.prefix_sum(rank)
 for _, _, index in ranges[start:end]:
 contains[index] = base + group_size - 1
 bit.add(rank, group_size)
 end = start

```



```
print(" ".join(map(str, contains)))
print(" ".join(map(str, contained_by)))

if __name__ == "__main__":
 main()
```

## **128 What to carry forward**

## 129 Testing discussion

Run `CP_TARGET=solution python3 sections/42_advanced_data_structures_mixed/check.py`. For small random cases, compare A with a sorted multiset, B with a direct array, C with a direct point scan, and D with ordered-pair interval checks. These oracles expose incorrect binary lifting, off-by-one difference boundaries, compression errors, and duplicate-group mistakes.

## 130 What to carry forward

Fenwick trees are prefix-summary engines. Pair them when a range update needs an extra weighted correction, binary-lift them when a prefix count becomes an order statistic, and combine one with a sweep when one coordinate can be made monotone.

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

### 130.1 43. 321E – Ciel and Gondolas: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/321/E> Local package: [problems/43\\_ciel\\_and\\_gondolas](#)

### 130.2 Restatement and traps

For every prefix length  $L$ , count all substring occurrences of that prefix, including overlaps. With  $n \leq 200000$ , comparing every prefix at every start is quadratic. The whole string itself occurs once, and a singleton prefix can occur many times.

### 130.3 Observation and trace

Let  $pi[i]$  be the longest proper prefix ending at position  $i$ . Every prefix ending at  $i$  contributes one occurrence to length  $pi[i]$ ; that occurrence also belongs to every border of that prefix. Count direct  $pi$  values, then propagate counts from long lengths to their failure link:

```
for x in pi: count[x]++
for length from n-1 down to 1:
 count[pi[length-1]] += count[length]
answer[length] = count[length] + 1
```

The final  $+1$  counts the prefix occurrence beginning at position zero. Here is the complete small trace for `ababa`:

index $i$ :	0	1	2	3	4
character:	a	b	a	b	a
$pi[i]$ :	0	0	1	2	3

Directly incrementing the nonzero  $pi$  values gives:

length $L$ :	1	2	3	4	5
direct count:	1	1	1	0	0

Now propagate in decreasing length. Length 3 passes its one occurrence to its failure link `pi[2]=1`; the resulting relevant counts are:

```
after L=3: count[1]=2, count[2]=1, count[3]=1
```

Lengths 2 and 1 only pass counts to the zero-length sentinel. Adding the initial occurrence to every positive length gives 3 2 2 1 1. In particular, the occurrence of `aba` also contributes to its border `a`; that is exactly what the failure-link propagation preserves.

## 130.4 Correctness proof

**Lemma.** After propagation, `count[L]` is the number of non-initial occurrences of the prefix of length `L`.

**Proof.** Every occurrence ending at a position first contributes to its longest proper prefix match. Its failure-link chain lists exactly all shorter prefixes that also end there. Processing links from longer to shorter passes each occurrence to every such border once.  $\square$

**Theorem.** `count[L]+1` is the total occurrence count of prefix length `L`.

**Proof.** The lemma counts all occurrences ending after the initial prefix; the unmatched initial occurrence contributes the one added explicitly.  $\square$

## 130.5 Complexity

The prefix function and failure-link propagation are both  $O(n)$  time and use  $O(n)$  memory.

## 130.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;cin>>s;in
```

## 130.7 Full Python solution

```
import sys
def main():
 s=sys.stdin.buffer.readline().strip();n=len(s);pi=[0]*n;cnt=[0]*(n+1)
 for i in range(1,n):
 j=pi[i-1]
 while j and s[i]!=s[j]:j=pi[j-1]
 if s[i]==s[j]:j+=1
 pi[i]=j
 for x in pi:cnt[x]+=1
 for i in range(n-1,0,-1):cnt[pi[i-1]]+=cnt[i]
 print(*(cnt[i]+1 for i in range(1,n+1)))
if __name__=='__main__':main()
```

## 130.8 Tests and stress testing

Test `a`, `aaaa`, and a string with no repeated prefix. For tiny strings, the oracle directly compares every prefix against every legal substring start; this is independent of prefix links. The scale case is 200000 copies of `a`, whose answers are `n, n-1, ..., 1`, exposing quadratic matching.

## 130.9 44. 896C – Willem, Chtholly and Seniorious: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/896/C> Local package: [problems/44\\_willem\\_chtholly](#)

## 130.10 Restatement

For every input word, find the shortest nonempty prefix shared by exactly one input occurrence. Equal words are still separate occurrences, so no prefix can distinguish one copy from another. A word that is a prefix of a longer word may also have no answer.

This is the blind trie exercise for the section: the task asks for a property of all prefixes of each word, not merely a count for one supplied prefix.

## 130.11 Constraints and traps

There are up to 200,000 words and the same bound on total characters. Testing every prefix against every word is quadratic. The answer is an input-order answer, so duplicate words need repeated `-1` lines rather than being merged.

Do not mistake “the word ends here” for “the prefix is unique.” For `do` and `dog`, the prefix `do` is a prefix of both input words even though only one word ends at that trie node.

## 130.12 Observations and trace

In a trie, store `pass[node]`: the number of input words whose path visits the node. During insertion, increment it after every character. Then a prefix is unique exactly when the node reached by that prefix has `pass == 1`.

Insert the sample’s `cat`, `car`, `dog`, `do`, `cat`. The node for `ca` has `pass=3` because it serves both copies of `cat` and `car`. The node for `car` has `pass=1`, so `car` needs length three. The node for `do` has `pass=2`, and there is no later node for the shorter word `do`, so its answer is `-1`.

## 130.13 Algorithm

```
make a trie whose nodes store child links and pass counts
for every word:
 walk/create its character path
 increment pass at each reached node

for every word in input order:
```

```
walk its trie path from the root
print the first position whose node has pass = 1
print -1 if no such position occurs
```

## 130.14 Correctness proof

**Lemma.** For every trie node representing prefix  $p$ ,  $\text{pass}[\text{node}]$  equals the number of input words beginning with  $p$ .

**Proof.** A word visits exactly the nodes for its nonempty prefixes during insertion. The algorithm increments this node once for that word and never for a word that does not have the prefix. Summing over all input occurrences gives the stated count.  $\square$

**Theorem.** The algorithm prints the required shortest unique-prefix length for every word.

**Proof.** By the lemma, a scanned prefix is unique exactly when its node has  $\text{pass}=1$ . The second traversal checks prefixes in increasing length order, so the first such node is the shortest valid prefix. If none is found, every nonempty prefix is shared by at least two occurrences and  $-1$  is correct.  $\square$

## 130.15 Complexity

Let  $L$  be the total input length.

```
Time: O(L)
Memory: O(L)
```

## 130.16 C++ implementation notes

An `array<int,26>` gives fixed lowercase transitions. A node index of zero is the root; newly allocated children receive positive indices. Increment only the destination node's `pass` count, because the empty prefix is not allowed.

## 130.17 Python implementation notes

A list of dictionaries keeps the implementation direct. Use byte input for large files if desired; the reference decodes after splitting because input is restricted to lowercase ASCII.

## 130.18 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
struct Node { array<int,26> next{}; int pass = 0; };
int main() {
 ios::sync_with_stdio(false); cin.tie(nullptr);
 int n; if (!(cin >> n)) return 0; vector<string> words(n); vector<Node> trie(1);
 for (string &word : words) { cin >> word; int node = 0; for (char ch : word) { int c=ch-'a'
```

```
for (const string &word : words) { int node=0, answer=-1; for (int i=0;i<(int)word.size();
}
```

## 130.19 Full Python solution

```
import sys

def main() -> None:
 words = sys.stdin.buffer.read().split()
 if not words: return
 n = int(words[0]); words = [word.decode() for word in words[1:n + 1]]
 children = [{}]; passed = [0]
 for word in words:
 node = 0
 for char in word:
 if char not in children[node]:
 children[node][char] = len(children); children.append({}); passed.append(0)
 node = children[node][char]; passed[node] += 1
 output = []
 for word in words:
 node = 0; answer = -1
 for length, char in enumerate(word, 1):
 node = children[node][char]
 if passed[node] == 1: answer = length; break
 output.append(str(answer))
 sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__": main()
```

## 130.20 Common wrong answers

Counting only words ending at a node instead of every word passing through it.  
 Deduplicating equal strings before answering.  
 Returning a prefix for do when dog is also present.  
 Using the root count as an empty-prefix answer.  
 Scanning all words for every candidate prefix.

## 130.21 Tests that matter

Identical words: every copy must print -1.  
 A word that is a prefix of another word.  
 Words sharing a long prefix before branching.  
 A one-word input: answer is 1 for a nonempty word.  
 200,000 identical one-letter words: checks linear memory and output.



## 131 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

## 132 Testing discussion

Brute-force small trie prefixes by `word.startswith(prefix)`, palindrome queries by reverse comparison, and distinct substrings with a set of all slices. Then run:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```

### 132.1 45. 833B – The Bakery: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/833/B> Local package: [problems/45\\_bakery](#)

### 132.2 Restatement and reduction target

Each project has a possibly negative profit. A directed rule  $a \rightarrow b$  means that selecting project  $a$  requires selecting project  $b$ . Choose any feasible set of projects, including the empty set, with maximum total profit. There can be up to 200 projects and 1,000 rules, so enumerating all subsets is impossible.

This is a maximum-closure problem. A cut represents which projects are chosen: put a project on the source side exactly when it is selected. We will construct the network so that a finite cut is exactly a feasible choice, and its capacity is the profit lost by that choice.

### 132.3 Network construction and trace

Let  $P$  be the sum of all positive project profits. Build a source  $s$  and sink  $t$  around the project vertices:

```
profit w > 0: add s -> project with capacity w
profit w < 0: add project -> t with capacity -w
rule a requires b: add a -> b with capacity INF
```

Choose  $INF = P + 1$ . No optimum cut can afford an  $INF$  edge because cutting every positive source edge costs at most  $P$ . Therefore a finite minimum cut never puts  $a$  on the source side while  $b$  is on the sink side: it obeys every requirement.

For profits  $[9, -5, 7, -3]$  with rules  $1 \rightarrow 2$ ,  $3 \rightarrow 4$ , and  $3 \rightarrow 2$ , selecting all projects earns  $9 - 5 + 7 - 3 = 8$ . In the corresponding cut, no positive source edge is cut and the two negative project-to-sink edges cost  $5 + 3 = 8$ . Since  $P = 16$ ,  $P - \text{cut} = 8$ . Selecting only project 1 and its prerequisite 2 has profit 4 and cuts source edge for project 3 (cost 7) plus project 2's sink edge (cost 5), again  $16 - 12 = 4$ .

## 132.4 Algorithm

```
P = sum of positive profits
INF = P + 1
build the stated flow network
min_cut = maximum_flow(s, t)
print P - min_cut
```

The max-flow/min-cut theorem supplies the minimum cut value. Use `long long` for `P`, `INF`, capacities, and the result.

## 132.5 Correctness proof

Consider any feasible selected set  $S$  and place exactly its projects with the source. No prerequisite edge crosses from source to sink because feasibility ensures every requirement of a selected project is also selected. The cut crosses  $s \rightarrow v$  for every unselected positive project, and  $v \rightarrow t$  for every selected negative project. Its capacity is therefore

```
sum(positive profits of unselected projects)
+ sum(absolute negative profits of selected projects)
= P - profit(S).
```

Conversely, a minimum cut cannot cross an `INF` edge: cutting all source edges has cost  $P < INF$ . Thus its source-side projects satisfy every prerequisite, so they form a feasible set, and the same equality shows their profit is  $P - \text{cut\_capacity}$ . Minimizing cut capacity therefore maximizes feasible profit. By max-flow/min-cut, the computed maximum flow is that minimum capacity, so the algorithm prints the optimal profit. QED.

## 132.6 Complexity, implementation notes, and tests

The network has  $n+2$  vertices and  $n+m$  forward edges. It uses  $O(n+m)$  memory. With arbitrary integral capacities, Dinic has worst-case  $O(V^2 E)$  time; the deliberately moderate local bound ( $n \leq 200$ ,  $m \leq 1000$ ) makes this implementation appropriate in both supplied languages. For substantially larger arbitrary-capacity maximum-closure networks, use a flow implementation and limits whose performance guarantee has been analyzed for that setting. `INF` must be strictly larger than the total positive profit, not a small magic constant. Test all-negative projects (answer zero), a profitable project whose requirements make it unprofitable, independent profitable projects, and rules forming a cycle.

## 132.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Dinic {
 struct Edge {
```

```

 int to, reverse;
 long long capacity;
};

vector<vector<Edge>> graph;
vector<int> level, next_edge;

explicit Dinic(int n) : graph(n), level(n), next_edge(n) {}

void add_edge(int from, int to, long long capacity) {
 Edge forward{to, (int)graph[to].size(), capacity};
 Edge backward{from, (int)graph[from].size(), 0};
 graph[from].push_back(forward);
 graph[to].push_back(backward);
}

bool build_levels(int source, int sink) {
 fill(level.begin(), level.end(), -1);
 queue<int> queue;
 level[source] = 0;
 queue.push(source);
 while (!queue.empty()) {
 int vertex = queue.front();
 queue.pop();
 for (const Edge& edge : graph[vertex]) {
 if (edge.capacity > 0 && level[edge.to] == -1) {
 level[edge.to] = level[vertex] + 1;
 queue.push(edge.to);
 }
 }
 }
 return level[sink] != -1;
}

long long send_flow(int vertex, int sink, long long pushed) {
 if (vertex == sink) return pushed;
 for (int& edge_index = next_edge[vertex];
 edge_index < (int)graph[vertex].size(); edge_index++) {
 Edge& edge = graph[vertex][edge_index];
 if (edge.capacity == 0 || level[edge.to] != level[vertex] + 1) continue;
 long long flow = send_flow(edge.to, sink, min(pushed, edge.capacity));
 if (flow == 0) continue;
 edge.capacity -= flow;
 graph[edge.to][edge.reverse].capacity += flow;
 return flow;
 }
 return 0;
}

long long max_flow(int source, int sink) {
 long long total = 0;
 const long long INF = numeric_limits<long long>::max() / 4;
 while (build_levels(source, sink)) {

```

```

 fill(next_edge.begin(), next_edge.end(), 0);
 while (long long pushed = send_flow(source, sink, INF)) total += pushed;
 }
 return total;
}
};

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, m;
 if (!(cin >> n >> m)) return 0;
 vector<long long> profit(n);
 long long positive_total = 0;
 for (long long& value : profit) {
 cin >> value;
 if (value > 0) positive_total += value;
 }

 int source = n;
 int sink = n + 1;
 Dinic flow(n + 2);
 long long inf = positive_total + 1;
 for (int project = 0; project < n; project++) {
 if (profit[project] > 0) flow.add_edge(source, project, profit[project]);
 if (profit[project] < 0) flow.add_edge(project, sink, -profit[project]);
 }
 for (int i = 0; i < m; i++) {
 int project, prerequisite;
 cin >> project >> prerequisite;
 flow.add_edge(project - 1, prerequisite - 1, inf);
 }

 cout << positive_total - flow.max_flow(source, sink) << '\n';
 return 0;
}

```

## 132.8 Full Python solution

```

import sys
from collections import deque

class Dinic:
 def __init__(self, size: int) -> None:
 self.graph: list[list[list[int]]] = [[] for _ in range(size)]
 self.level = [-1] * size
 self.next_edge = [0] * size

 def add_edge(self, start: int, end: int, capacity: int) -> None:

```

```

 self.graph[start].append([end, len(self.graph[end]), capacity])
 self.graph[end].append([start, len(self.graph[start]) - 1, 0])

def build_levels(self, source: int, sink: int) -> bool:
 self.level = [-1] * len(self.graph)
 self.level[source] = 0
 queue = deque([source])
 while queue:
 vertex = queue.popleft()
 for target, _, capacity in self.graph[vertex]:
 if capacity > 0 and self.level[target] == -1:
 self.level[target] = self.level[vertex] + 1
 queue.append(target)
 return self.level[sink] != -1

def send_flow(self, vertex: int, sink: int, pushed: int) -> int:
 if vertex == sink:
 return pushed
 while self.next_edge[vertex] < len(self.graph[vertex]):
 edge = self.graph[vertex][self.next_edge[vertex]]
 target, reverse, capacity = edge
 if capacity > 0 and self.level[target] == self.level[vertex] + 1:
 flow = self.send_flow(target, sink, min(pushed, capacity))
 if flow:
 edge[2] -= flow
 self.graph[target][reverse][2] += flow
 return flow
 self.next_edge[vertex] += 1
 return 0

def max_flow(self, source: int, sink: int) -> int:
 total = 0
 while self.build_levels(source, sink):
 self.next_edge = [0] * len(self.graph)
 while pushed := self.send_flow(source, sink, 10**30):
 total += pushed
 return total

def main() -> None:
 tokens = list(map(int, sys.stdin.buffer.read().split()))
 if not tokens:
 return
 n, m = tokens[0], tokens[1]
 profit = tokens[2 : 2 + n]
 pointer = 2 + n
 positive_total = sum(value for value in profit if value > 0)
 source, sink = n, n + 1
 flow = Dinic(n + 2)
 inf = positive_total + 1
 for project, value in enumerate(profit):
 if value > 0:
 flow.add_edge(source, project, value)

```

```

 elif value < 0:
 flow.add_edge(project, sink, -value)
 for _ in range(m):
 project, prerequisite = tokens[pointer] - 1, tokens[pointer + 1] - 1
 pointer += 2
 flow.add_edge(project, prerequisite, inf)
 print(positive_total - flow.max_flow(source, sink))

if __name__ == "__main__":
 sys.setrecursionlimit(300_000)
 main()

```

## 133 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.



## 134 Testing discussion

For tiny random networks, compare with a slow augmenting-path implementation; for matching, compare with brute force over small left vertices. Verify cut output by checking every residual-reachable edge and summing original capacities leaving the reached set. For D, enumerate small project subsets and reject ones that violate a prerequisite before comparing their total profit. Run:

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
```

### 134.1 46. 311B – Cats Transport: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/311/B> Local package: [problems/46\\_cats\\_transport](#)

### 134.2 Restatement

For every vertex  $v$ , find the largest independent set that must include  $v$ . An independent set contains no edge whose two endpoints are both selected. Running a tree DP from scratch for each forced vertex would be quadratic.

### 134.3 Constraints and traps

With  $n=200000$ , one full DP per forced vertex is too slow. The state must say whether its current vertex is selected: that choice constrains every neighbour.

### 134.4 Observations and trace

Root the tree. For each vertex  $v$ , first compute values for its subtree only:

```
down_with[v] = best size in v's subtree when v is selected
down_without[v] = best size in v's subtree when v is not selected
```

For every child  $to$ , selecting  $v$  forces  $to$  not to be selected; leaving  $v$  unselected lets  $to$  choose its better state. Thus:

```
down_with[v] = 1 + sum(down_without[to])
down_without[v] = sum(max(down_with[to], down_without[to]))
```

These values omit the part above  $v$ . Store matching outside contributions:

```
outside_with[v] = best contribution outside v's subtree if v is selected
outside_without[v] = best contribution outside v's subtree if v is not selected
```

Both are zero at the root. To pass information from a parent  $v$  to a child  $to$ , remove  $to$  from the parent's child sums:

```
free_siblings = down_without[v] - max(down_with[to], down_without[to])
without_siblings = down_with[v] - 1 - down_without[to]

outside_with[to] = outside_without[v] + free_siblings
outside_without[to] = max(
 outside_without[v] + free_siblings,
 outside_with[v] + 1 + without_siblings
)
```

The first transition makes the parent unselected because  $to$  is selected. In the second, the parent may be selected or unselected. The requested answer is  $down\_with[v] + outside\_with[v]$ . In the sample, forcing vertex 3 excludes 1, 4, and 5; only vertex 2 can join it, so its answer is 2.

## 134.5 Algorithm

```
root the tree and record parent plus traversal order
process the order backwards to compute down_with and down_without
process the order forwards to compute outside_with and outside_without
for every v, print down_with[v] + outside_with[v]
```

## 134.6 Correctness proof

**Lemma 1.**  $down\_with[v]$  and  $down\_without[v]$  are optimal independent-set sizes in  $v$ 's subtree under their stated conditions.

**Proof.** At a leaf the values are one and zero. Child subtrees are disjoint. If  $v$  is selected, every child must be unselected; if  $v$  is unselected, every child may independently use its better state. The two recurrences therefore combine exactly the feasible optimal child choices. Induction proves the lemma.

**Lemma 2.**  $outside\_with[v]$  and  $outside\_without[v]$  are optimal contributions outside  $v$ 's subtree under their stated conditions.

**Proof.** Both values are zero at the root. For a parent-child edge, the transition enumerates the only two states of the parent. It combines the already optimal outside contribution with disjoint sibling subtrees, excluding the child itself. Induction in preorder proves the lemma.

**Theorem.** The printed value for every vertex  $v$  is the largest independent set size among sets that contain  $v$ .

**Proof.** Lemma 1 gives the optimal selected set inside  $v$ 's subtree and Lemma 2 gives the compatible optimum outside it. The regions are disjoint and their sum selects  $v$ ; every feasible global set splits into the same two regions. Hence the sum is feasible and optimal.

## 134.7 Complexity and implementation notes

Both passes inspect every edge a constant number of times:  $O(n)$  time and  $O(n)$  memory. Use an iterative traversal so a 200000-vertex chain cannot overflow recursion. Recomputing all sibling contributions for every child would be quadratic; subtract the child's known contribution from the parent sum.

## 134.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 if (!(cin >> n)) return 0;
 vector<vector<int>> graph(n);
 for (int edge = 0; edge + 1 < n; ++edge) {
 int left, right;
 cin >> left >> right;
 --left;
 --right;
 graph[left].push_back(right);
 graph[right].push_back(left);
 }

 vector<int> parent(n, -1), order{0};
 for (int index = 0; index < (int)order.size(); ++index) {
 int vertex = order[index];
 for (int neighbor : graph[vertex]) {
 if (neighbor != parent[vertex]) {
 parent[neighbor] = vertex;
 order.push_back(neighbor);
 }
 }
 }

 vector<int> down_without(n), down_with(n, 1);
 for (int index = n - 1; index >= 0; --index) {
 int vertex = order[index];
 for (int child : graph[vertex]) {
 if (parent[child] == vertex) {
 down_without[vertex] += max(down_without[child], down_with[child]);
 down_with[vertex] += down_without[child];
 }
 }
 }

 vector<int> outside_without(n), outside_with(n), answer(n);
```

```

for (int vertex : order) {
 answer[vertex] = down_with[vertex] + outside_with[vertex];
 for (int child : graph[vertex]) {
 if (parent[child] != vertex) continue;
 int siblings_free = down_without[vertex] - max(down_without[child], down_with[child]);
 int siblings_without = down_with[vertex] - 1 - down_without[child];
 outside_with[child] = outside_without[vertex] + siblings_free;
 outside_without[child] = max(
 outside_without[vertex] + siblings_free,
 outside_with[vertex] + 1 + siblings_without
);
 }
}

for (int vertex = 0; vertex < n; ++vertex) {
 cout << answer[vertex] << (vertex + 1 == n ? '\n' : ' ');
}
}

```

## 134.9 Full Python solution

```

import sys

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n = data[0]
 graph = [[] for _ in range(n)]
 for index in range(1, 2 * n - 1, 2):
 left, right = data[index] - 1, data[index + 1] - 1
 graph[left].append(right)
 graph[right].append(left)

 parent = [-1] * n
 order = [0]
 for vertex in order:
 for neighbor in graph[vertex]:
 if neighbor != parent[vertex]:
 parent[neighbor] = vertex
 order.append(neighbor)

 down_without = [0] * n
 down_with = [1] * n
 for vertex in reversed(order):
 for neighbor in graph[vertex]:
 if parent[neighbor] == vertex:
 down_without[vertex] += max(down_without[neighbor], down_with[neighbor])
 down_with[vertex] += down_without[neighbor]

```

```

outside_without = [0] * n
outside_with = [0] * n
answer = [0] * n
for vertex in order:
 answer[vertex] = down_with[vertex] + outside_with[vertex]
 for child in graph[vertex]:
 if parent[child] != vertex:
 continue
 siblings_free = down_without[vertex] - max(down_without[child], down_with[child])
 siblings_without = down_with[vertex] - 1 - down_without[child]
 outside_with[child] = outside_without[vertex] + siblings_free
 outside_without[child] = max(
 outside_without[vertex] + siblings_free,
 outside_with[vertex] + 1 + siblings_without,
)

print(" ".join(map(str, answer)))

if __name__ == "__main__":
 main()

```

## 134.10 Common wrong answers

Allowing a selected parent and selected child together.  
 Using a parent's full sum without subtracting the current child.  
 Combining `outside\_with` with a selected parent across the shared edge.  
 Returning the ordinary global maximum instead of forcing the queried vertex.

## 134.11 Tests that matter

n=1.  
 A star, where a forced center and a forced leaf behave very differently.  
 A chain, which also checks iterative traversal.  
 A branching tree where forcing an internal vertex excludes several neighbours.

## 135 Testing discussion

For A, compare each answer on tiny trees with a BFS from every start. For B, compare the diameter-endpoint answer with the maximum entry of a BFS distance row. For C, enumerate pairs and test whether their unique path contains the vertex. For D, enumerate all subsets on tiny trees, reject any subset containing both endpoints of an edge, and retain the best size for every included vertex. Then run:

```
CP_TARGET=solution python3 sections/46_rerooting_advanced_tree_dp/check.py
```

### 135.1 47. 55D – Beautiful Numbers: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/55/D> Local package: [problems/47\\_beautiful\\_numbers\\_divisibility](#)

### 135.2 Restatement and constraint implications

Choose a subset whose sum is exactly `target`. Among all such subsets, print the maximum number of selected items, or `-1` when no exact-sum subset exists. There are at most 36 values, which may be negative. This makes a target-indexed dynamic program impossible when values and target are large, while direct subset enumeration remains exponential in 36.

The split is the same as other meet-in-the-middle tasks, but the data retained for an equal sum changes. We do not merely need to know whether a right-half sum exists: we need the largest item count that achieves that sum.

### 135.3 State and worked trace

For values `[4,3,2,1,-1]` and target 5, split after the first two values. The left half contributes states `(sum,count)` including `(0,0)`, `(4,1)`, `(3,1)`, and `(7,2)`. The right half `[2,1,-1]` has, among others, sum 1 with maximum count 3, from selecting all three values. Pairing left sum 4 with right sum `5-4=1` gives four selected items: `[4,2,1,-1]`.

If several right subsets have the same sum, retaining only the largest count is safe. Any left state needing that sum benefits from the largest count; a smaller count can never improve the final objective.

### 135.4 Algorithm

```

enumerate (sum, count) for every subset of the right half
best_count[sum] = maximum count among its right-half states

answer = no solution
for every (left_sum, left_count) from the left half:
 if target - left_sum exists in best_count:
 answer = max(answer, left_count + best_count[target-left_sum])
print answer, or -1 if it was never updated

```

The enumeration must include (0,0) for each half, which represents taking no item from that half. Negative values require no pruning by partial sum.

## 135.5 Correctness proof

Every full subset has a unique left-half subset and right-half subset. Therefore it appears exactly once as a pair of enumerated states whose sums add to its total. For a fixed left state with sum  $s$ , an exact target is possible exactly when the right sum is  $\text{target}-s$ .

For that required right sum, `best_count` stores the maximum item count among all right subsets with that sum. Replacing any compatible right subset by that stored one preserves the exact total and cannot decrease item count. Thus the algorithm considers an item count at least as large as every valid full subset. It never combines sums other than `target`, so every candidate is valid. Consequently its maximum is precisely the largest cardinality of an exact-sum subset, or no candidate exists exactly when no such subset exists. QED.

## 135.6 Complexity, implementation notes, and tests

Each half has at most 18 elements, so its enumeration contains at most  $2^{18}$  states. Expected dictionary operations make the total time  $O(2^{(n/2)})$  and extra memory  $O(2^{(n/2)})$ . Use signed 64-bit integers for sums and targets in C++; Python integers already handle them.

Test target zero, all negative values, repeated values that create the same sum with different cardinalities, and an impossible target. A frequent bug is to overwrite a map entry with the last count rather than the maximum count.

## 135.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

using State = pair<long long, int>;

vector<State> enumerate_states(const vector<long long>& values) {
 vector<State> states{{0, 0}};
 states.reserve(1ULL << values.size());
 for (long long value : values) {
 int old_size = (int)states.size();
 for (int i = 0; i < old_size; i++) {

```

```

 states.push_back({states[i].first + value, states[i].second + 1});
 }
}
return states;
}

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 long long target;
 if (!(cin >> n >> target)) return 0;
 vector<long long> values(n);
 for (long long& value : values) cin >> value;

 vector<long long> left_values(values.begin(), values.begin() + n / 2);
 vector<long long> right_values(values.begin() + n / 2, values.end());
 vector<State> left_states = enumerate_states(left_values);
 vector<State> right_states = enumerate_states(right_values);

 unordered_map<long long, int> best_right_count;
 best_right_count.reserve(right_states.size() * 2);
 for (const State& state : right_states) {
 best_right_count[state.first] = max(best_right_count[state.first], state.second);
 }

 int answer = -1;
 for (const State& state : left_states) {
 auto it = best_right_count.find(target - state.first);
 if (it != best_right_count.end()) answer = max(answer, state.second + it->second);
 }
 cout << answer << '\n';
 return 0;
}

```

## 135.8 Full Python solution

```

import sys

def enumerate_states(values: list[int]) -> list[tuple[int, int]]:
 states = [(0, 0)]
 for value in values:
 states.extend((total + value, count + 1) for total, count in states[:])
 return states

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:

```



```

 return
 n, target = data[0], data[1]
 values = data[2 : 2 + n]
 middle = n // 2
 left_states = enumerate_states(values[:middle])
 right_states = enumerate_states(values[middle:])

 best_right_count: dict[int, int] = {}
 for total, count in right_states:
 previous = best_right_count.get(total, -1)
 if count > previous:
 best_right_count[total] = count

 answer = -1
 for total, count in left_states:
 other_count = best_right_count.get(target - total)
 if other_count is not None:
 answer = max(answer, count + other_count)
 print(answer)

if __name__ == "__main__":
 main()

```

## 136 Testing discussion

The proof is a bijection between full subsets and pairs of half-subsets. For small  $n$ , enumerate all masks as the stress oracle. In counting, use `upper_bound(X-left_sum)`; in optimization, verify the candidate immediately below that bound, never merely the first greater value.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.

For D, tiny cases instead enumerate every full-array mask, independently checking exact sum and item count. The scale case uses 36 power-of-two values, which makes a full enumeration impossible while giving a directly known answer.

### 136.1 48. 840D – Destiny: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/840/D> Local package: [problems/48\\_destiny](#)

### 136.2 Restatement and constraints

Choose a contiguous subarray whose length is in  $[L, R]$  and maximize its sum. Values may be negative, so an empty subarray is not permitted and initializing the answer to zero would be wrong. With  $n$  up to 200000, checking every start and end is quadratic.

### 136.3 Reduction and trace

Let `pref[i]` be the sum of the first  $i$  values. A subarray ending just before prefix position `right` and beginning after prefix position `left` has sum `pref[right] - pref[left]`. Its length is allowed exactly when `right - R <= left <= right - L`.

For a fixed `right`, `pref[right]` is fixed, so the best allowed subarray uses the smallest prefix sum in that sliding interval of `left` indices. With  $L=2$ ,  $R=4$ , and values  $-2, 3, -1, 5, -6, 4$ , endpoint `right=4` permits prefix indices 0, 1, 2, whose sums are 0, -2, 1. Choose index 1: the result is `pref[4]-pref[1]=5-(-2)=7`, from subarray  $3, -1, 5$ .

### 136.4 Monotone deque algorithm

Keep a deque of legal prefix indices in increasing `pref` order. At each endpoint `right`, insert `enter=right-L`, removing suffix indices with `pref[index] >= pref[enter]`; remove front indices below `right-R`; then update the answer with `pref[right]-pref[deque.front()]`. The entering index has just reached the minimum allowed length. A removed back index has a prefix sum no smaller than a newer index and expires earlier, so it can never be useful.

## 136.5 Correctness proof

**Lemma 1.** Before evaluating an endpoint `right`, the deque contains exactly the undominated prefix indices in `[right-R, right-L]`, in increasing prefix-sum order.

**Proof.** Index `right-L` is inserted when it becomes legal and indices below `right-R` are removed when they expire. If an older index has prefix sum at least a newer index's sum, the newer index is at least as good for every future endpoint and remains legal longer. Thus deleting the older index is safe; no other legal, undominated index is deleted.

**Lemma 2.** At endpoint `right`, the deque front gives the best allowed subarray ending there.

**Proof.** Lemma 1 makes the front the smallest legal `pref[left]`. Subtracting it from fixed `pref[right]` maximizes `pref[right]-pref[left]`.

**Theorem.** The algorithm prints the maximum sum of every allowed subarray.

**Proof.** Every allowed subarray has an endpoint `right`. Lemma 2 considers the best allowed one for that endpoint, and the algorithm takes the maximum over every endpoint.

## 136.6 Complexity and implementation notes

Every prefix index enters and leaves the deque at most once, so time is  $O(n)$  and memory is  $O(n)$  for the prefix array and deque. Use `long long` in C++: the answer can have magnitude  $2 * 10^{14}$ . Insert before querying so length exactly `L` is legal. Removing equal prefix sums from the back retains the newer index, which expires later.

## 136.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n, minimum_length, maximum_length;
 if (!(cin >> n >> minimum_length >> maximum_length)) return 0;
 vector<long long> prefix(n + 1);
 for (int index = 1; index <= n; ++index) {
 long long value;
 cin >> value;
 prefix[index] = prefix[index - 1] + value;
 }

 deque<int> candidates;
 long long answer = numeric_limits<long long>::lowest();
 for (int right = 1; right <= n; ++right) {
 int entering = right - minimum_length;
 if (entering >= 0) {
 while (!candidates.empty() && prefix[candidates.back()] >= prefix[entering]) {
 candidates.pop_back();
 }
 }
 }
```

```

 }
 candidates.push_back(entering);
 }
 while (!candidates.empty() && candidates.front() < right - maximum_length) {
 candidates.pop_front();
 }
 if (!candidates.empty()) {
 answer = max(answer, prefix[right] - prefix[candidates.front()]);
 }
}
cout << answer << '\n';
}

```

## 136.8 Full Python solution

```

import sys
from collections import deque

def main() -> None:
 data = list(map(int, sys.stdin.buffer.read().split()))
 if not data:
 return
 n, minimum_length, maximum_length = data[:3]
 values = data[3:]
 prefix = [0] * (n + 1)
 for index, value in enumerate(values, 1):
 prefix[index] = prefix[index - 1] + value

 candidates: deque[int] = deque()
 answer = -10**30
 for right in range(1, n + 1):
 entering = right - minimum_length
 if entering >= 0:
 while candidates and prefix[candidates[-1]] >= prefix[entering]:
 candidates.pop()
 candidates.append(entering)
 while candidates and candidates[0] < right - maximum_length:
 candidates.popleft()
 if candidates:
 answer = max(answer, prefix[right] - prefix[candidates[0]])

 print(answer)

if __name__ == "__main__":
 main()

```

## 136.9 Tests that matter

```
All values negative: the answer must still be nonempty.
L = R = 1: this becomes the maximum element.
L = 1 and R = n: every nonempty length is legal.
Optima at exactly L and exactly R, plus equal prefix sums.
```

The tiny oracle enumerates every subarray independently. The scale case has 200000 ones and a wide length interval, exposing a quadratic scan.

## 137 Testing discussion

For A, the input contract requires nonnegative values. This makes the squared segment-sum cost Monge, so the optimal previous cut is monotone and the divide-and-conquer search restriction is valid. Do not reuse this optimization unchanged for signed arrays: the monotone-optimum theorem can fail even though the unoptimized recurrence remains correct.

Keep a slow recurrence for random arrays. Compare every DP entry, not only the final answer: an incorrect opt-range often first corrupts an interior state. For window minima, test  $w=1$ ,  $w \geq n$ , and equal values; for Knuth, test one and two piles and verify interval iteration order.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.

### 137.1 49. 911G – Mass Change Queries: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/911/G> Local package: [problems/49\\_mass\\_change\\_queries](#)

### 137.2 Restatement

Lines are inserted online. At a query coordinate, print the smallest line value and, among equal values, the earliest inserted line. This blind exercise adds a deterministic tie rule to dynamic line minimum queries.

### 137.3 Observations

Compare a line at  $x$  by the pair  $(m \cdot x + b, \text{insertion\_id})$ . Lexicographic pair comparison exactly implements both the value objective and the tie rule. Read all operations first and compress the query coordinates; a Li Chao tree then needs nodes only for coordinates that will be queried.

At a segment, retain the better line at its midpoint. The other line can only win on one side, so recurse into that side after comparing endpoints. This is the Li Chao invariant, unchanged except that every comparison uses the pair.

### 137.4 Algorithm

```
collect and sort distinct query x values
for every inserted line, assign its insertion id
insert it into a Li Chao segment tree using pair comparisons at endpoints/mid
for a query x, take the minimum pair over tree nodes on x's root-to-leaf path
```

## 137.5 Correctness proof

**Lemma.** Every Li Chao node retains a line that is best at its midpoint among lines routed through that node, while any line that can improve another point is passed to the only child interval where it can do so.

**Proof.** The midpoint comparison retains the better pair. The difference of two linear values is linear; with lexicographic id resolving equality, the other line can beat the retained line on at most one side. The endpoint tests route it precisely there.  $\square$

**Theorem.** Each query returns the required value and earliest attaining id.

**Proof.** By the lemma, any inserted line that can win at the query coordinate is retained on some node of that coordinate's search path. Taking the minimum pair over that path considers every possible winner and chooses exactly the specified lexicographic optimum.  $\square$

## 137.6 Complexity

For  $K$  distinct query coordinates, insertion and query cost  $O(\log K)$ ; memory is  $O(K)$ . Products require long long under the local bounds.

## 137.7 Implementation notes

In C++, represent an empty line as  $(\text{INF}, \text{INT\_MAX})$ . In Python, use a matching large pair. Do not use only a value comparison on ties: a later equal line must not replace an earlier one.

## 137.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll INF=(1LL<<62);
struct Line{ll m=0,b=INF;int id=INT_MAX; pair<ll,int> at(ll x)const{return {m*x+b,id};}};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int q;if(!(cin>>q))return 0;vector<arr
```

## 137.9 Full Python solution

```
import sys
INF=10**30
def main():
 d=list(map(int,sys.stdin.buffer.read().split()));
 if not d:return
 q=d[0];i=1;ops=[];xs=[]
 for _ in range(q):
 t=d[i];a=d[i+1];i+=2
 if t==1:b=d[i];i+=1;ops.append((t,a,b))
 else:ops.append((t,a,0));xs.append(a)
 xs=sorted(set(xs));where={x:i for i,x in enumerate(xs)};tree=[None]*(4*len(xs))
 def val(line,x):return (line[0]*x+line[1],line[2]) if line else (INF,10**9)
```

```

def add(line,p,l,r):
 if tree[p] is None:tree[p]=line;return
 m=(l+r)//2
 if val(line,xs[m])<val(tree[p],xs[m]):tree[p],line=line,tree[p]
 if l==r:return
 if val(line,xs[l])<val(tree[p],xs[l]):add(line,p*2,l,m)
 elif val(line,xs[r])<val(tree[p],xs[r]):add(line,p*2+1,m+1,r)
def get(p,l,r,k):
 ans=val(tree[p],xs[k]);
 if l==r:return ans
 m=(l+r)//2;return min(ans,get(p*2,l,m,k) if k<=m else get(p*2+1,m+1,r,k))
out=[];id=0
for t,a,b in ops:
 if t==1:id+=1;add((a,b,id),1,0,len(xs)-1)
 else:out.append('%d %d'%get(1,0,len(xs)-1,where[a]))
print('\n'.join(out))
if __name__=='__main__':main()

```

## 137.10 Tests that matter

Several identical lines: always select the first id.  
 Lines intersecting at a query coordinate.  
 Negative x and negative slopes.  
 100,000 equal insertions followed by 100,000 queries.



## 138 Testing discussion

Stress every line query against evaluating all inserted lines. Equal slopes need an explicit keep-better-intercept rule. For quadratic DP, expand  $(x_i - x_j)^2$  before choosing slope and intercept; a sign error changes min to max behavior. Test repeated  $x$ , negative values, and a single line.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.

### 138.1 50. 1100F – Ivan and Burgers: local companion

External Codeforces task: <https://codeforces.com/problemset/problem/1100/F> Local package: [problems/50\\_ivan\\_and\\_burgers](#)

### 138.2 Restatement and observation

For a huge inclusive interval  $[1, r]$ , sum Fibonacci values modulo MOD. The blind step is recognizing the telescoping identity:

$$F(0) + F(1) + \dots + F(t) = F(t+2) - 1.$$

Therefore the requested result is:

$$F(r+2) - F(1+1) \bmod \text{MOD}.$$

### 138.3 Algorithm

Use fast doubling to obtain  $(F(n), F(n+1))$  in  $O(\log n)$ :

$$\begin{aligned} F(2k) &= F(k) * (2F(k+1) - F(k)) \\ F(2k+1) &= F(k)^2 + F(k+1)^2. \end{aligned}$$

Apply it twice per query and normalize the subtraction.

### 138.4 Correctness proof

The prefix identity follows by induction from  $F(i+2) = F(i+1) + F(i)$ . Subtracting the prefix through  $1-1$  from the prefix through  $r$  gives exactly  $[1, r]$ . Fast doubling identities compute the required Fibonacci values by splitting the index in half, so the printed normalized difference is correct.  $\square$

## 138.5 Complexity and implementation

Each query costs  $O(\log r)$  time and  $O(\log r)$  recursion stack; values are always reduced modulo 1,000,000,007. Use unsigned 64-bit indices in C++ for the  $10^{18}$  bound and Python integers in Python.

## 138.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll MOD=1000000007;
pair<ll,ll> fib(unsigned long long n){
 if (!n) return {0, 1};
 auto [a, b] = fib(n / 2);
 ll c=a*((2*b%MOD-a+MOD)%MOD)%MOD, d=(a*a%MOD+b*b%MOD)%MOD;
 return n&1?make_pair(d,(c+d)%MOD):make_pair(c,d);
}
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int q;cin>>q;while(q--){unsigned long
```

## 138.7 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
 if n==0:return 0,1
 a,b=fib(n//2); c=a*((2*b-a)%MOD)%MOD; d=(a*a+b*b)%MOD
 return (d,(c+d)%MOD) if n&1 else (c,d)
def main():
 data=list(map(int,sys.stdin.buffer.read().split())); out=[]
 for i in range(1,len(data),2): out.append(str((fib(data[i+1]+2)[0]-fib(data[i]+1)[0])%MOD))
 print('\n'.join(out))
if __name__=='__main__':main()
```

## 138.8 Tests that matter

[0,0], [0,1], one-element ranges, and a range with  $l > 0$ .  
Large endpoints: never iterate through the interval.

## 139 Testing discussion

Use a slow mex DFS on tiny DAGs, brute-force congruence search on small moduli, and iterative Fibonacci as independent oracles. Check CRT normalization into  $[0,1\text{cm})$ , zero Fibonacci, and a DAG vertex with no outgoing edges.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.